Studio Nx

PRD (Phase 0~6) & DB 설계 · ERD 통합본

2026-10-02

목차

1. PRD/00_PRD_Overview.md
2. PRD/01_Phase1_Account_Pages.md
3. PRD/02_Phase2_Sequencer_Core.md
4. PRD/03_Phase3_Audio_Export.md
5. PRD/04_Phase4_Stem_Separation.md
6. PRD/05_Phase5_AI_Reference_Analysis.md
7. PRD/06_Phase6_Collaboration_Chat.md
8. DB.md
9. ERD.md

Studio Nx — PRD 00. 전체 개요

항목	내용
문서 버전	v0.1 (초안)
작성일	2026-10-01
원본 기획	`StudioNx_CLAUDE.md`
DB 문서	docs/DB.md

---

1. 제품 비전

Studio Nx는 설치 없이 브라우저에서 바로 쓰는 웹 기반 DAW(Digital Audio Workstation) 시퀀서입니다.

샘플을 끌어다 놓고, MIDI를 찍고, 웹 기반 가상악기/이펙트로 곡을 만든 뒤 WAV/MP3로 내보낼 수 있습니다. 여기에 다음 두 가지 차별점을 더합니다.

1. **AI 레퍼런스 분석** — 참고할 곡(YouTube 링크/파일)과 장르를 주면, AI가 분석해 synth·플러그인 프리셋과 믹싱·마스터링 설정을 추천합니다.
2. **실시간 협업** — Google Docs처럼 여러 회원이 하나의 프로젝트를 동시에 편집하고, 단체 채팅으로 소통합니다.

2. 대상 사용자

페르소나	특징	핵심 니즈
입문 프로듀서	DAW 설치·구매가 부담, 믹싱 지식 부족	브라우저에서 바로 시작, "이런 느낌" 내는 법을 AI가 알려줌
비트메이커 / 취미 작곡가	샘플 위주 작업, 여러 기기에서 작업	샘플 라이브러리 관리, 어디서나 같은 프로젝트 열기
협업 팀 (밴드, 작곡 스터디)	파일 주고받기가 번거로움	같은 프로젝트를 동시에 편집, 채팅으로 피드백

3. 핵심 기능 ↔ Phase 매핑

#	기획서 핵심 기능	Phase	문서
1	회원 등 기본 CRUD	Phase 1	01_Phase1_Account_Pages.md
2	시퀀서 (샘플 드래그앤드롭·라이브러리, MIDI 클릭/외부 악기 입력, 웹 기반 synth·plugin)	Phase 2	02_Phase2_Sequencer_Core.md
5	WAV/MP3 등 포맷별 출력	Phase 3	03_Phase3_Audio_Export.md
4	샘플 악기 분리 (instrument separation)	Phase 4	04_Phase4_Stem_Separation.md
3	레퍼런스 AI 분석 → 프리셋·믹싱·마스터링 추천	Phase 5	05_Phase5_AI_Reference_Analysis.md

#	기획서 핵심 기능	Phase	문서
6	회원 초대 실시간 협업 + 단체톡	Phase 6	06_Phase6_Collaboration_Chat.md
+	시퀀서 캐릭터 Nyxi(Fruity Dance식, 외형은 외부 AI 플랫폼 제작), 트랙 아코디언(Ableton 체인 + FL Patcher), **Nyxi 카드형 디바이스 UI + 상세 설정 모달**, 토글형 하단 패널	Phase 2	02 §4.7~4.9
+	오토메이션, 스케일·코드 도우미, 오디오 녹음, 패턴·클립 런처, 템플릿(확인창), 튜토리얼, UI 커스터마이징·커맨드 팔레트, 로컬 캐시	Phase 2	02 §4.2, §4.10~4.16
+	저장 용량·사용량 관리	Phase 1	01 §4.4
+	Nyxi 도우미 (FL Studio Gopher식 AI 도우미, 캐릭터가 얼굴), **Nyxi 팁(상황 인지, 프로젝트 5개 시 유지 여부 확인창)**	Phase 5	05 §4.4~4.5
+	타임라인 코멘트, 버전 스냅샷·복원, 트랙 잠금·담당자	Phase 6	06 §4.5~4.7
+	오디오 회귀 테스트, Sentry 활용 확장	공통	이 문서 §5, §7.1

> `+` 행은 2026-10-01 추가 요구사항입니다. 채택 경위는 Ai feedback.md §0, 상세 설명은 Ai feedback_moreExplain.md Phase 순서는 기획서 번호가 아니라 **의존성** 기준입니다. 시퀀서(Phase 2)가 있어야 출력(3)이 가능하고, 샘플 라이브러리가 있어야 분리(4)가 가능하며, 플러그인·믹서 구조가 있어야 AI 추천값(5)을 적용할 수 있습니다. 실시간 협업(6)은 프로젝트 데이터 구조가 안정된 뒤에 얹는 것이 안전합니다.

4. 로드맵

```
Phase 1  계정·기본 페이지   ──▶  회원가입/로그인, 마이페이지, 프로젝트 목록
Phase 2  시퀀서 코어        ──▶  studioNx.html 핵심 (MVP 완성 시점)
Phase 3  오디오 출력        ──▶  WAV / MP3 / (FLAC) 내보내기
Phase 4  악기 분리          ──▶  샘플 → stem 분리 → 라이브러리 등록
Phase 5  AI 레퍼런스 분석   ──▶  분석 + 프리셋·믹싱·마스터링 추천
Phase 6  협업·단체톡        ──▶  초대, 동시 편집, 채팅
```

- **MVP = Phase 1 + 2 + 3의 핵심** (혼자서 곡을 만들고 파일로 내보낼 수 있는 상태)
- Phase(기능 묶음)와 별개로, 실제 **구현 순서는 MVP·중요도 기준 3단계**로 나눴습니다 → Implementation_Plan.md (구현 목록 + 간트차트)

5. 기술 스택

영역	기술	비고
Frontend	HTML5 + CSS + JavaScript (Vanilla)	오디오: Web Audio API, AudioWorklet / 입력: Web MIDI API, Drag and Drop API
Backend	Python **FastAPI**	REST API + WebSocket(Phase 6), 무거운 작업은 백그라운드 작업 큐
Database	**MySQL 8.0** (database `studionx`)	기획서 규칙: Oracle 설치가 필요하면 MySQL로 대체 → 현재 PC에 MySQL 8.0이 설치되어 있어 MySQL 사용. 상세는 DB.md
C++	DSP / 가상악기 엔진 (JUCE 등 학습 목적)	브라우저에서 쓰려면 **WebAssembly(Emscripten)** 로 빌드해 AudioWorklet에서 호출하는 구조가 유력. 연동 지점만 PRD에 정의하고 구현은 사용자가 직접

영역	기술	비고
파일 저장	서버 로컬 `storage/` (추후 S3 등 교체 가능)	DB에는 경로만 저장
오류 추적 · 모니터링	**Sentry**	Frontend(`@sentry/browser`)와 Backend(`sentry-sdk` FastAPI 연동) 모두 적용. 예외·로그 오류 수집, 스택 트레이스, 릴리스·환경(dev/prod) 구분, 성능 추적(Tracing). 백그라운드 워커(인코딩·분리·AI 분석)도 동일하게 연동

5.1 시스템 구성도 (개념)

```
[Browser]
  ├ HTML/CSS 페이지들
  ├ JS 앱 (시퀀서 UI, 상태 관리)
  ├ Web Audio 그래프 ── AudioWorklet ── (C++ → WASM DSP 모듈)
  └ Web MIDI (외부 MIDI 키보드)
        │  REST (JSON, multipart)        │ WebSocket (Phase 6)
        ▼                                 ▼
[FastAPI 서버] ── 작업 큐(Worker) ── 인코딩 / 악기 분리 / AI 분석
        │
        ├ MySQL (studionx)
        └ File Storage (samples, exports, stems, references)

[Browser] · [FastAPI 서버] · [Worker]  ──(오류·성능 이벤트)──►  [Sentry]
```

6. 페이지 구성

페이지	설명	Phase
`home.html`	서비스 첫 화면 — 서비스 소개, 기능 설명, 문의 창	1
`login.html`	로그인	1
`join.html`	회원가입	1
`myPage.html`	마이페이지 대시보드 (최근 프로젝트, 라이브러리 요약, 받은 초대)	1 (+6 초대 위젯)
`userInfo.html`	회원 정보 조회·수정·탈퇴, **환경 설정(UI 커스터마이징)** 탭	1 (+2 환경 설정)
`myProject.html`	나의 프로젝트 목록 (생성/이름변경/삭제/열기, 공유된 프로젝트)	1 (+6 공유 목록)
`studioNx.html`	시퀀서 본체 — 트랙 아코디언(디바이스·노트·오토메이션), 어레인지/런처 뷰, 토글형 하단 패널(톡방·Nyxi), 캐릭터 Nyxi, 설정·커맨드 팔레트	2~6

```
home ─┬─ login ──► myPage ─┬─ userInfo
      └─ join              ├─ myProject ──► studioNx
                           └─ (초대 수락) ──► studioNx
```

7. 공통 비기능 요구사항

구분	요구사항
브라우저	최신 Chrome / Edge 우선 지원 (Web MIDI·AudioWorklet 지원). Firefox·Safari는 가능한 범위에서

구분	요구사항
보안	비밀번호 해시 저장(bcrypt/argon2), HTTPS, 모든 API에서 **본인/권한 검사**, 업로드 파일 형식·용량 검증
성능	시퀀서 재생 중 UI 조작으로 오디오가 끊기지 않을 것 (오디오 처리는 AudioWorklet 스레드)
반응형	일반 페이지는 모바일 대응. `studioNx.html`은 데스크톱(1280px 이상) 우선
로그 · 오류 추적	서버 오류·작업 실패는 원인(`error_message`)을 DB에 남기고, 예외·오류 로그는 **Sentry**로 수집·추적. 프론트 JS 오류(오디오 엔진, WebSocket 등)도 Sentry로 전송. 비밀번호·토큰 등 민감정보는 Sentry 전송 전 제거(`before_send` 필터)
모니터링 지표 (Sentry 확장)	오류뿐 아니라 **서비스 품질 지표**도 Sentry 커스텀 이벤트·성능 측정으로 수집: 오디오 언더런(끊김) 횟수, AudioContext 출력 지연, 프로젝트 로딩 시간(캐시 적중 여부), WebSocket 재접속 횟수·연산 거부율, AI 분석·Nyxi 응답 시간, 출력·분리 작업 소요 시간. Session Replay는 입력값·채팅 내용 마스킹 후 오류 발생 세션에만 제한적으로 사용
저장 용량	사용자별 저장 한도(기본 5GB, 설정값)와 사용량 표시. 오래된 출력 파일·자동 스냅샷은 보관 정책에 따라 자동 정리
시간	서버·DB는 UTC 저장, 화면에서 로컬 시간으로 표시

7.1 품질 보증 — 오디오 회귀 테스트 (골든 파일)

> DAW에서 가장 위험한 버그는 **오류 없이 소리만 조용히 달라지는 것**입니다. Sentry로는 잡히지 않으므로 별도 테스트를 둡니다.

항목	내용
방법	테스트용 프로젝트 세트(악기별·이펙트별·패처 병렬·오토메이션·템플릿)를 **오프라인 렌더링**해 기준 WAV(골든 파일)와 비교
비교 기준	샘플 단위 차이의 최대값·RMS 차이·LUFS 차이가 허용 오차 이내 (예: RMS 차이 -80 dBFS 이하)
실행 시점	오디오 엔진·플러그인·라우팅·오토메이션 코드를 바꿀 때마다 (CI 또는 로컬 명령 1개). Headless Chrome에서 실행
기준 갱신	의도적으로 소리를 바꾼 경우에만, 변경 이유를 기록하고 골든 파일 갱신
보관 위치	`tests/audio/projects/` (테스트 프로젝트 JSON), `tests/audio/golden/` (기준 WAV)
WASM 비교	같은 플러그인의 Web Audio 버전과 C++/WASM 버전 결과를 비교하는 용도로도 사용

8. AI 활용 규칙 (기획서 원칙 정리)

모든 Phase 문서의 "AI 역할 구분" 절은 이 규칙을 따릅니다.

영역	AI 역할	비고
HTML / CSS	적극 활용 (AI가 작성)	단, 작성 전 관련 디렉토리의 **기존 레퍼런스(스타일·구조)를 먼저 확인**하고 그에 맞출 것
JavaScript	**템플릿만 제공** (파일 구조, 함수 시그니처, TODO 주석)	구현은 사용자가 직접. **요청할 때만** AI가 구현
Backend (FastAPI)	**템플릿만 제공** (라우터, 스키마, 함수 뼈대)	위와 동일
DB	새 database(`studionx`)에 테이블 생성, 구조 문서(DB.md) 유지	스키마 변경 시 DB.md를 반드시 함께 갱신
C++	**최대한 건드리지 않음**	JUCE 등 외부 라이브러리 학습 목적. 인터페이스(입출력 규격) 문서화까지만

영역	AI 역할	비고
문서화	AI가 구현한 부분은 **반드시 별도 문서에 상세 설명**	프로젝트 관련 질문(면접·발표 등)에 대비. 예: `docs/ai-notes/<기능명>.md`

9. 용어집

용어	설명
DAW	Digital Audio Workstation. 음악 녹음·편집·믹싱 소프트웨어
시퀀서	시간축(타임라인) 위에 오디오/MIDI를 배치하고 재생하는 도구
트랙 / 클립	트랙은 가로 한 줄(악기 하나), 클립은 트랙 위에 놓인 오디오/MIDI 덩어리
샘플	짧은 오디오 파일(드럼 한 방, 루프 등)
MIDI	음 높이·세기·길이 등 연주 정보. 소리 자체가 아님
피아노롤	MIDI 노트를 격자 위에 찍어 편집하는 화면
가상악기 / Synth	MIDI를 받아 소리를 만드는 악기 플러그인
이펙트 플러그인	EQ, 컴프레서, 리버브 등 소리를 가공하는 플러그인
프리셋	플러그인 파라미터 값을 저장한 묶음
믹싱 / 마스터링	트랙별 밸런스·음색 조정 / 최종 결과물의 음량·톤 마무리
Stem	곡을 악기별로 나눈 오디오 (보컬, 드럼, 베이스 등)
LUFS	체감 음량 단위. 스트리밍 플랫폼 기준값(예: -14 LUFS) 비교에 사용
Tick / PPQ	시퀀서 시간 단위. PPQ=960이면 4분음표 1개 = 960 tick
AudioWorklet	브라우저의 별도 오디오 스레드에서 실행되는 사용자 정의 DSP 코드
WASM	WebAssembly. C++ 등을 브라우저에서 실행 가능한 형태로 컴파일한 것
트랙 아코디언	트랙 바로 밑으로 펼쳐지는 편집 영역. 디바이스·피아노롤을 여기서 편집 (하단 고정 패널 대체)
디바이스 체인	Ableton Live 방식. 악기/이펙트 카드를 왼쪽→오른쪽으로 나열해 직렬 처리
패처 (Patcher)	FL Studio 방식. 디바이스를 노드로 두고 선으로 연결해 병렬·분기 라우팅
하단 패널	버튼을 누를 때만 나타나는 패널. [톡방]과 [Nyxi] 탭
Nyxi (닉시)	Studio Nx의 오리지널 캐릭터. ① FL Studio **Fruity Dance**(FL Chan)처럼 화면에서 박자에 맞춰 춤추고 ② **AI 도우미의 아이콘·얼굴** ③ **디바이스 카드 아트**의 바탕. 외형·애니메이션은 **별도 AI 플랫폼에서 제작**하며 이 프로젝트는 연동만 담당
Nyxi 팁	Nyxi가 묻기 전에 먼저 알려주는 상황 인지 팁 (클리핑, 저역 겹침 등). 끄기 / 중요한 것만 / 모두
디바이스 카드 / 상세 설정 모달	이펙터·악기를 Nyxi 기반 카드로 보여주고 핵심 노브만 노출, 세부 설정은 카드 클릭 시 여는 모달에서 조작
Nyxi 도우미	FL Studio의 Gopher 같은 대화형 AI. Studio Nx 사용법·작곡/믹싱 질문에 답함 (Phase 5). "Gopher"는 FL Studio 서비스명이라 사용하지 않음
오토메이션	볼륨·필터 등 파라미터를 시간에 따라 자동으로 바꾸는 선(레인)
패턴 / 링크 클립	여러 클립이 같은 노트 묶음(패턴)을 공유. 하나를 고치면 모두 바뀜
클립 런처 (세션 뷰)	트랙 × 씬 격자에서 클립을 즉석 재생·조합하는 화면 (Ableton 방식)
스냅샷	특정 시점의 프로젝트 전체 상태 저장본. 미리듣기·복원·복제 가능
커맨드 팔레트	Ctrl+P로 모든 명령을 검색해 실행하는 입력창

Studio Nx — PRD 01. Phase 1: 계정 · 기본 페이지

항목	내용
기획서 기능	1) 기본적인 회원 등의 CRUD 기능
선행 Phase	없음
관련 페이지	`home.html`, `login.html`, `join.html`, `myPage.html`, `userInfo.html`, `myProject.html`
전체 개요	00_PRD_Overview.md

---

1. 목표 / 배경

- 서비스의 입구(홈)와 회원 체계를 만들어, 이후 모든 기능(샘플 라이브러리, 프로젝트, 협업)이 **"누구의 데이터인가"** 를 기준으로 동작할 수 있게 합니다.
- 프로젝트 목록 CRUD까지 이 단계에서 만들어, Phase 2 시퀀서가 "프로젝트를 열고 저장"하는 대상이 미리 존재하도록 합니다.

2. 범위

In scope

- 회원가입 / 로그인 / 로그아웃 / 회원정보 조회·수정 / 비밀번호 변경 / 회원 탈퇴
- 인증 상태 유지 (JWT 기반, 아래 FR-105 참고)
- 홈 화면 서비스 소개 + 문의 접수
- 마이페이지 대시보드
- 나의 프로젝트 목록: 생성, 이름·설명 수정, 삭제, 열기(→ `studioNx.html` 이동)

Out of scope (이후 Phase 또는 추후 검토)

- 소셜 로그인(Google 등), 이메일 인증, 비밀번호 찾기 메일 발송 → 추후 검토
- 관리자 문의 답변 화면 → 추후 검토 (DB 컬럼은 미리 준비)
- 공유받은 프로젝트 목록 → Phase 6

3. 사용자 스토리

ID	사용자 스토리
US-101	방문자로서, 홈에서 Studio Nx가 무엇인지 보고 바로 가입하고 싶다.
US-102	방문자로서, 궁금한 점을 문의 창으로 보내고 싶다.
US-103	회원으로서, 이메일·비밀번호로 로그인하고 로그인 상태가 유지되길 원한다.
US-104	회원으로서, 닉네임·프로필 이미지·비밀번호를 바꾸고 싶다.
US-105	회원으로서, 원하면 탈퇴할 수 있어야 한다.
US-106	회원으로서, 마이페이지에서 최근 작업한 프로젝트를 바로 열고 싶다.
US-107	회원으로서, 새 프로젝트를 만들고, 이름을 바꾸고, 지울 수 있어야 한다.

4. 기능 요구사항

4.1 회원 (users)

ID	요구사항	우선순위
FR-101	회원가입: 이메일, 비밀번호(+확인), 닉네임 입력. 이메일·닉네임 중복 검사(입력 중 실시간 확인 API 제공)	필수
FR-102	비밀번호 규칙: 8자 이상, 영문+숫자 포함. 서버에서 bcrypt/argon2로 해시 저장	필수
FR-103	로그인: 이메일+비밀번호 확인 후 토큰 발급, `last_login_at` 갱신. 실패 시 "이메일 또는 비밀번호가 올바르지 않습니다"로 통일된 메시지	필수
FR-104	로그아웃: 클라이언트 토큰 삭제 (+ 리프레시 토큰 쿠키 삭제)	필수
FR-105	인증 방식: **Access Token(JWT, 짧은 만료) + Refresh Token(HttpOnly 쿠키)**. 로그인이 필요한 페이지는 토큰이 없으면 `login.html`로 이동	필수
FR-106	회원정보 조회/수정: 닉네임, 프로필 이미지 변경. 이메일은 변경 불가	필수
FR-107	비밀번호 변경: 현재 비밀번호 확인 후 변경	필수
FR-108	회원 탈퇴: 비밀번호 재확인 → `status='DELETED'`, `deleted_at` 기록(소프트 삭제). 탈퇴 계정은 로그인 불가	필수

4.2 홈 / 문의 (inquiries)

ID	요구사항	우선순위
FR-111	홈 화면: 서비스 소개(히어로), 핵심 기능 6가지 소개, 시작하기(가입/로그인) 버튼	필수
FR-112	문의 창: 이름, 이메일, 제목, 내용 입력 → 저장. 로그인 상태면 이름·이메일 자동 채움, `user_id` 연결	필수
FR-113	문의 남용 방지: 같은 IP/이메일 기준 짧은 시간 내 반복 전송 제한	권장

4.3 마이페이지 / 프로젝트 (projects)

ID	요구사항	우선순위
FR-121	마이페이지 대시보드: 프로필 요약, 최근 수정 프로젝트 5개, 샘플 라이브러리 개수(Phase 2 이후 표시), **저장 용량·AI 사용량 위젯**(§4.4)	필수
FR-122	프로젝트 생성: 제목(필수), 설명, BPM(기본 120), 박자(기본 4/4) 입력. 생성 시 `project_members`에 본인을 `OWNER`로 함께 등록하고, **MASTER 트랙 1개를 자동 생성**	필수
FR-127	생성 방식 선택: [빈 프로젝트] / [템플릿으로 시작](PRD 02 §4.14). 템플릿 선택 시 **확인창**에 BPM·트랙 구성 등 생성 내용 요약 후 [만들기]	필수 (2단계부터)
FR-123	프로젝트 목록: 최근 수정순 정렬, 제목 검색, 페이지네이션	필수
FR-124	프로젝트 수정: 제목·설명 변경	필수
FR-125	프로젝트 삭제: 확인 모달 후 소프트 삭제(`deleted_at`). OWNER만 가능	필수
FR-126	프로젝트 열기: `studioNx.html?projectId={id}` 로 이동	필수

4.4 저장 용량 · 사용량 관리

ID	요구사항	우선순위
FR-131	사용자별 저장 한도(`users.storage_quota_bytes`, 기본 5GB)와 사용량(`storage_used_bytes`) 관리. 샘플·녹음·출력·분리 결과·스냅샷 파일이 사용량에 포함	필수
FR-132	사용량은 파일 생성·삭제 시 증감, 하루 1회 실제 파일 크기로 재계산(보정)	필수

ID	요구사항	우선순위
FR-133	대시보드 위젯: 용량 막대(종류별 비율: 샘플/출력/stem/스냅샷), 80% 이상이면 경고 색	필수
FR-134	[용량 정리] 화면: 큰 파일 순 목록, 사용 안 하는 샘플(어떤 클립에도 안 쓰임), 오래된 출력 파일 → 선택 삭제(확인창)	권장
FR-135	한도 초과 시 새 업로드·녹음·출력·분리를 막고 안내 (기존 프로젝트 열기·재생은 가능)	필수
FR-136	AI 사용량 위젯: 오늘/이번 달 레퍼런스 분석·Nyxi 질문 횟수와 남은 한도 (PRD 05 §4.3.1)	필수
FR-137	자동 정리 정책: 출력 파일 30일, 자동 스냅샷은 보관 규칙(PRD 06 §4.6)에 따라 정리. 정리 7일 전 대시보드 알림	권장

4.5 환경 설정 (UI 커스터마이징)

ID	요구사항	우선순위
FR-141	`userInfo.html`에 **[환경 설정] 탭**: 테마, 강조 색, UI 배율, 글자 크기, 캐릭터, 단축키 등 — 시퀀서의 ⚙ 설정 창과 **같은 설정**(PRD 02 §4.16)을 편집	필수 (2단계부터)
FR-142	테마(다크/라이트/시스템)는 시퀀서뿐 아니라 **모든 페이지**(home, myPage 등)에 적용. 비로그인 상태에서는 브라우저 설정을 따름	필수

5. 비기능 요구사항

- 비밀번호·토큰은 로그에 남기지 않습니다.
- 모든 프로젝트 API는 **요청자가 해당 프로젝트의 멤버인지** 확인합니다(`project_members` 기준).
- 프로필 이미지: jpg/png/webp, 최대 2MB.
- 입력값 검증은 프론트(UX용)와 백엔드(보안용) **양쪽 모두** 수행.

6. 화면 구성

페이지	주요 요소
`home.html`	상단 네비(로고, 로그인/가입 or 마이페이지), 히어로 섹션, 기능 소개 카드 6개, 문의 폼, 푸터
`login.html`	이메일, 비밀번호, 로그인 버튼, 회원가입 링크
`join.html`	이메일(중복확인), 닉네임(중복확인), 비밀번호/확인, 가입 버튼
`myPage.html`	좌측 메뉴(대시보드 / 회원정보 / 나의 프로젝트), 프로필 카드, 최근 프로젝트 카드 목록, 저장 용량·AI 사용량 위젯
`userInfo.html`	탭: [회원 정보](프로필 이미지, 닉네임, 비밀번호 변경, 회원 탈퇴) · [환경 설정](UI 커스터마이징) · [용량 관리]
`myProject.html`	"새 프로젝트" 버튼(빈 프로젝트 / 템플릿으로 시작 → 확인창), 검색창, 프로젝트 카드(썸네일, 제목, 수정일, 메뉴: 열기/이름변경/삭제)

7. API 초안 (FastAPI)

> 경로 prefix: `/api/v1`. 구현 없이 라우터·스키마 템플릿만 제공 예정.

Method	Path	설명	인증
POST	`/auth/signup`	회원가입	-

Method	Path	설명	인증
GET	`/auth/check-email?email=`	이메일 중복 확인	-
GET	`/auth/check-nickname?nickname=`	닉네임 중복 확인	-
POST	`/auth/login`	로그인 → access token + refresh 쿠키	-
POST	`/auth/refresh`	access token 재발급	refresh 쿠키
POST	`/auth/logout`	로그아웃	✓
GET	`/users/me`	내 정보	✓
PATCH	`/users/me`	닉네임 등 수정	✓
POST	`/users/me/profile-image`	프로필 이미지 업로드 (multipart)	✓
PUT	`/users/me/password`	비밀번호 변경	✓
DELETE	`/users/me`	회원 탈퇴 (body: 비밀번호)	✓
POST	`/inquiries`	문의 등록	선택
GET	`/projects?query=&page=&size=`	내 프로젝트 목록	✓
POST	`/projects`	프로젝트 생성	✓
GET	`/projects/{project_id}`	프로젝트 기본 정보	✓ 멤버
PATCH	`/projects/{project_id}`	제목·설명 수정	✓ OWNER/EDITOR
DELETE	`/projects/{project_id}`	프로젝트 삭제	✓ OWNER
GET	`/me/dashboard`	대시보드 요약 (용량·AI 사용량 포함)	✓
GET	`/me/storage`	저장 용량 상세 (종류별 사용량, 큰 파일 목록)	✓
GET	`/users/me/settings` · PATCH	환경 설정 조회·저장 (PRD 02 §7)	✓

8. 사용 테이블

> 컬럼 상세는 DB.md §3.1 참고

- `users`
- `inquiries`
- `projects`
- `project_members` (프로젝트 생성 시 OWNER 행 등록용 — 협업 기능 자체는 Phase 6)
- `tracks` (프로젝트 생성 시 MASTER 트랙 자동 생성)
- `user_settings` (환경 설정), `users.storage_quota_bytes` / `storage_used_bytes` (용량)

9. AI 역할 구분

영역	이 Phase에서 AI가 하는 일
HTML / CSS	6개 페이지 마크업·스타일 **작성** (작성 전 기존 레퍼런스 디렉토리 확인)
JavaScript	폼 검증, API 호출, 토큰 관리 모듈의 **템플릿**(함수 시그니처 + TODO)만
Backend	`auth`, `users`, `inquiries`, `projects` 라우터·Pydantic 스키마·서비스 함수 **템플릿**만
DB	`studionx` database 및 Phase 1 테이블 생성, DB.md 갱신

영역	이 Phase에서 AI가 하는 일
문서	AI가 작성한 부분은 `docs/ai-notes/`에 설명 문서 작성

10. 완료 기준 (Acceptance Criteria)

- [ ] 회원가입 → 로그인 → 마이페이지 진입이 정상 동작한다.
- [ ] 중복 이메일/닉네임으로는 가입할 수 없다.
- [ ] DB에 비밀번호가 평문으로 저장되지 않는다.
- [ ] 로그인하지 않은 상태로 `myPage.html` 등에 접근하면 `login.html`로 이동한다.
- [ ] 회원정보·비밀번호 수정, 탈퇴가 동작하고, 탈퇴한 계정은 로그인되지 않는다.
- [ ] 홈 문의 폼 제출 시 `inquiries`에 저장된다.
- [ ] 프로젝트 생성/목록/수정/삭제가 동작하고, 생성 시 `project_members`에 OWNER 행이 생긴다.
- [ ] 다른 회원의 프로젝트는 API로도 조회·수정·삭제할 수 없다.
- [ ] 대시보드에 저장 용량과 AI 사용량이 표시되고, 한도를 넘으면 업로드가 막힌다.
- [ ] 환경 설정에서 테마를 바꾸면 모든 페이지에 적용되고 다음 로그인에도 유지된다.

Studio Nx — PRD 02. Phase 2: 시퀀서 코어

항목	내용
기획서 기능	2) 기본적인 시퀀서 기능 — 샘플 드래그앤드롭·라이브러리 저장, MIDI 클릭/외부 악기 입력, 웹 기반 plugin·synth·가상악기
선행 Phase	Phase 1 (회원, 프로젝트)
관련 페이지	`studioNx.html`
전체 개요	00_PRD_Overview.md
구현 단계	이 문서의 기능은 1~3단계에 나뉘어 구현됩니다 → Implementation_Plan.md

---

1. 목표 / 배경

- 브라우저만으로 **곡을 만들 수 있는 DAW**를 완성합니다. 이 Phase의 핵심(타임라인·피아노롤·악기·이펙트·저장)이 Studio Nx MVP의 중심입니다.
- 이후 Phase(출력, 분리, AI 추천, 협업)가 모두 이 시퀀서의 데이터 구조(트랙·클립·패턴·노트·디바이스·오토메이션)를 기반으로 동작하므로, **데이터 모델을 안정적으로 설계**하는 것이 가장 중요합니다.

2. 범위

In scope

- 트랜스포트: 재생/정지/일시정지, 루프 구간, BPM·박자 설정, 메트로놈
- 타임라인(어레인지 뷰): 트랙 추가/삭제/순서변경, 클립 배치·이동·길이조절·복제·삭제, 그리드 스냅
- 샘플 라이브러리: 외부 파일 드래그앤드롭 업로드 → 개인 라이브러리 저장, 검색·태그, 브라우저 로컬 캐시
- MIDI 입력: 피아노롤 클릭 입력, **Web MIDI API로 외부 MIDI 키보드** 실시간 연주/녹음, `.mid` 파일 가져오기
- **스케일·코드 도우미** (피아노롤)
- **오디오 녹음** (마이크/오디오 인터페이스)
- 웹 기반 가상악기(synth, 샘플러) 및 이펙트 플러그인(EQ, 컴프레서, 리버브, 딜레이 등)
- **트랙 아코디언**: 각 트랙 바로 밑으로 펼쳐지는 편집 영역. Ableton Live식 **디바이스 체인 뷰** + FL Studio Patcher식 **패처(노드 연결) 뷰**를 융합 (하단 고정 패널은 두지 않음)
- **오토메이션 레인** (아코디언의 [오토메이션] 탭)
- **패턴(링크 클립)** 과 **클립 런처(세션 뷰)**
- 믹서: 트랙 헤더의 볼륨·팬·뮤트·솔로·미터, 마스터 트랙
- **하단 패널(토글형)**: 버튼을 눌렀을 때만 나타나는 패널. 이 Phase에서는 틀(컨테이너)만 만들고, 내용인 **Nyxi 도우미(AI)** 는 Phase 5, **협업 톡방**은 Phase 6에서 채움
- **시퀀서 캐릭터 Nyxi(닉시)**: FL Studio의 Fruity Dance(FL Chan)처럼 화면 안에서 춤추는 캐릭터. 동시에 **AI 도우미의 아이콘·얼굴** 역할. **외형·애니메이션 에셋은 별도 AI 플랫폼에서 제작**하고, 이 프로젝트는 연동만 담당
- **Nyxi 카드형 디바이스 UI**: 이펙터·가상악기를 Nyxi 캐릭터 기반 카드로 표시하고, 세부 설정은 카드를 클릭해 여는 **상세 설정 모달**에서 조작
- **프로젝트 템플릿** (적용 전 확인창 필수)
- **튜토리얼 모드** (Nyxi가 안내)

- **UI 커스터마이징**(테마·색·크기·레이아웃·단축키)과 **커맨드 팔레트**
- 프로젝트 저장/불러오기, 실행 취소/다시 실행(Undo/Redo)

Out of scope

- VST 등 네이티브 플러그인 로드 → 지원하지 않음 (웹 기반만)
- 파일 출력 → Phase 3
- **Nyxi 캐릭터 외형·애니메이션·카드 아트 제작** → 별도 AI 플랫폼에서 제작해 제공받음. 이 프로젝트에서는 만들지 않고 **연동 규격**(§4.9 FR-288)만 정의
- 상황 인지 팁(Nyxi 팁) → PRD 05 §4.5 (이 문서에서는 설정 항목만 다룸)

3. 사용자 스토리

ID	사용자 스토리
US-201	사용자로서, 내 PC의 샘플 파일을 화면에 끌어다 놓으면 라이브러리에 저장되어 다음에도 쓰고 싶다.
US-202	사용자로서, 라이브러리의 샘플을 타임라인으로 끌어 원하는 박자에 배치하고 싶다.
US-203	사용자로서, 피아노롤에서 마우스 클릭으로 멜로디를 찍고 싶다.
US-204	사용자로서, USB MIDI 키보드를 연결해 연주하고 그대로 녹음하고 싶다.
US-205	사용자로서, MIDI 트랙에 신스를 고르고 노브를 돌려 소리를 바꾸고 싶다.
US-206	사용자로서, 트랙에 EQ·리버브 같은 이펙트를 순서대로 걸고 싶다.
US-207	사용자로서, 작업을 저장하고 나중에 같은 상태로 다시 열고 싶다.
US-208	사용자로서, 실수하면 Ctrl+Z로 되돌리고 싶다.
US-209	사용자로서, 트랙을 펼치면 바로 밑에서 이펙트를 왼쪽→오른쪽으로 끼워 넣고(체인), 필요하면 선을 연결해 병렬·분기 라우팅(패처)도 하고 싶다.
US-210	사용자로서, 작업 화면이 넓도록 평소엔 하단 패널이 숨겨져 있다가, 버튼을 누르면 AI 도우미나 톡방이 올라오길 원한다.
US-211	사용자로서, 곡을 재생하면 화면 구석의 캐릭터가 박자에 맞춰 춤추는 재미를 느끼고 싶다.
US-212	사용자로서, 빌드업 구간에서 필터를 서서히 여는 것처럼 파라미터를 시간에 따라 바꾸고 싶다. (오토메이션)
US-213	입문자로서, 키(스케일)를 고르면 맞는 음만 보이게 하고, 코드를 한 번에 찍고 싶다.
US-214	사용자로서, 마이크로 보컬·기타를 바로 녹음하고 싶다.
US-215	비트메이커로서, 드럼 패턴 하나를 고치면 곡 전체의 같은 패턴이 함께 바뀌길 원한다. 또 여러 패턴을 즉석에서 켜고 끄며 조합해 보고 싶다.
US-216	사용자로서, 장르 템플릿으로 시작하되, 적용 전에 무엇이 바뀌는지 확인하고 싶다.
US-217	처음 온 사용자로서, 캐릭터가 단계별로 사용법을 알려주길 원한다.
US-218	사용자로서, 화면 테마·색·글자 크기·단축키를 내 취향대로 바꾸고 싶다.
US-219	숙련 사용자로서, 마우스 없이 Ctrl+P로 명령을 검색해 바로 실행하고 싶다.
US-220	사용자로서, 트랙의 이펙터들을 한눈에 알아보기 쉬운 카드로 보고, 핵심 노브는 바로 돌리되 자세한 설정은 카드를 눌러 큰 창에서 하고 싶다.

4. 기능 요구사항

4.1 트랜스포트 / 타임라인

ID	요구사항	우선순위
FR-201	재생/정지/일시정지, 스페이스바 단축키, 재생 헤드 표시 및 클릭으로 위치 이동	필수
FR-202	BPM(20~300), 박자(분자/분모) 설정 → `projects`에 저장	필수
FR-203	메트로놈 on/off, 카운트인(1마디)	권장
FR-204	루프 구간 지정 및 반복 재생	필수
FR-205	시간 단위는 **tick (PPQ=960)** 으로 관리. 화면 표시는 `마디.박.틱` 형식	필수
FR-206	그리드 스냅(1/1 ~ 1/32, 셋잇단 포함), Alt 키로 스냅 해제	필수
FR-207	트랙 추가(AUDIO / MIDI), 이름·색상 변경, 순서 드래그 변경, 삭제	필수
FR-208	클립 이동, 좌우 끝 드래그로 길이 조절(오디오는 트리밍 = `offset_tick`), 복제(Ctrl+D), 삭제, 분할	필수
FR-209	가로 줌(시간축), 세로 줌(트랙 높이)	필수
FR-210	오디오 클립에 파형 표시	필수

4.2 샘플 라이브러리 · 로컬 캐시

ID	요구사항	우선순위
FR-221	브라우저 밖에서 파일을 화면(라이브러리 패널 또는 타임라인)에 드롭하면 업로드 → `samples` 저장. 타임라인에 드롭했으면 해당 위치에 클립도 생성	필수
FR-222	지원 형식: WAV, MP3, FLAC, OGG, AIFF. 파일당 최대 100MB(설정값)	필수
FR-223	업로드 시 길이·샘플레이트·채널 수 추출, 파형 미리보기 데이터 생성	필수
FR-224	라이브러리 패널: 목록, 이름 검색, 태그 필터, 클릭 시 미리듣기	필수
FR-225	샘플 이름 변경, 태그 추가/삭제, 삭제(사용 중인 클립이 있으면 경고 후 소프트 삭제)	필수
FR-226	여러 파일 동시 드롭, 업로드 진행률 표시	권장
FR-227	**저장 용량 확인**: 업로드 전 남은 용량(`users.storage_quota_bytes - storage_used_bytes`) 확인, 초과 시 업로드 거부 + 정리 안내 (PRD 01 §4.4)	필수
FR-228	**로컬 캐시**: 한 번 받은 샘플 오디오·파형 데이터를 브라우저(OPFS, 미지원 시 IndexedDB)에 저장. 프로젝트 재오픈 시 캐시 우선 사용 → 로딩 시간 단축	권장
FR-229	캐시 관리: 캐시 용량 상한(예: 2GB, 설정 가능), 오래 안 쓴 것부터 삭제(LRU), 설정 화면에 [캐시 비우기]. 서버 파일이 바뀌면(삭제·교체) 캐시 무효화	권장
FR-230	오프라인 열기: 네트워크가 끊겨도 캐시에 있는 프로젝트는 열어서 재생 가능(저장은 재연결 후 동기화)	추후

4.3 MIDI 입력

ID	요구사항	우선순위
FR-231	피아노롤 에디터: MIDI 클립 더블클릭 시 **해당 트랙 아코디언의 [노트] 탭**에서 열림(최대화 버튼으로 크게 보기 가능). 클릭으로 노트 추가, 드래그로 이동/길이조절, 우클릭(또는 Delete)으로 삭제, 다중 선택	필수
FR-232	노트 벨로시티 편집(하단 벨로시티 레인)	필수
FR-233	퀀타이즈(선택 노트를 그리드에 맞춤)	권장
FR-234	**Web MIDI API**: 연결된 MIDI 입력 장치 목록 표시·선택, 선택된 트랙의 가상악기로 실시간 연주	필수

ID	요구사항	우선순위
FR-235	MIDI 녹음: 녹음 버튼 → 재생하면서 연주한 노트를 새 MIDI 클립으로 저장	필수
FR-236	화면 건반 / PC 키보드(A~K 등)로 연주 (MIDI 장치가 없을 때)	권장
FR-237	`.mid` 파일 가져오기: 트랙별로 MIDI 클립 생성	권장
FR-238	Web MIDI 미지원 브라우저에서는 안내 메시지 표시, 클릭 입력은 정상 동작	필수

4.4 가상악기 / 이펙트 플러그인 (웹 기반)

ID	요구사항	우선순위
FR-241	플러그인은 서버의 `plugins` 테이블에 등록된 목록을 불러와 표시. 각 플러그인은 `code`로 프론트 모듈과 연결	필수
FR-242	기본 제공 가상악기: **Subtractive Synth**(OSC 2개, 필터, ADSR, LFO), **Sampler**(샘플 1개를 음정별로 재생), **Drum Rack**(패드별 샘플)	필수 (Synth, Sampler) / 권장 (Drum Rack)
FR-243	기본 제공 이펙트: EQ(3밴드 이상), Compressor, Reverb, Delay, Gain/Utility, Limiter(마스터용)	필수
FR-244	플러그인 UI는 `param_schema`(이름, 범위, 기본값, 단위)를 읽어 **노브/슬라이더를 자동 생성**하는 공통 UI 사용	필수
FR-245	트랙 이펙트 배치·라우팅은 **트랙 아코디언**(§4.7)에서 편집 → `track_plugins`, `track_plugin_links`	필수
FR-246	프리셋: 시스템 프리셋 불러오기, 내 프리셋 저장/불러오기 → `plugin_presets`	필수
FR-247	플러그인 구현 방식: Web Audio 기본 노드 조합(`engine='WEBAUDIO'`) 또는 **AudioWorklet + WASM**(`engine='WASM'`)	필수
FR-248	**C++ 연동 지점**: WASM 모듈은 공통 인터페이스(아래 §4.6)를 따르며, C++ DSP 코드 자체는 사용자가 직접 작성	필수

4.5 믹서 / 저장

ID	요구사항	우선순위
FR-251	트랙 헤더에 볼륨(dB), 팬, 뮤트, 솔로, 레벨 미터 (별도 믹서 탭 없이 트랙 헤더 + 아코디언에서 처리)	필수
FR-252	트랙 목록 맨 아래 **마스터 트랙**(`tracks.track_type='MASTER'`, 프로젝트당 1개, 삭제·이동 불가): 볼륨, 미터, 아코디언으로 마스터 디바이스(Limiter 등) 편집	필수
FR-253	프로젝트 저장: 수동 저장(Ctrl+S) + 자동 저장(변경 후 일정 시간, 예: 30초)	필수
FR-254	프로젝트 불러오기: 트랙·클립·패턴·노트·디바이스·오토메이션 상태를 한 번에 로드하고 오디오 그래프 재구성	필수
FR-255	Undo/Redo: 클라이언트에서 명령(Command) 단위로 이력 관리 (최소 100단계)	필수
FR-256	저장하지 않은 변경이 있을 때 페이지를 떠나면 경고	필수

4.6 WASM 플러그인 인터페이스 (C++ 연동 규격 초안)

> C++ 구현은 사용자가 직접 합니다. AI는 아래 **입출력 규격**만 문서로 정의합니다.

항목	규격 (초안)
빌드	Emscripten으로 `.wasm` + JS glue 생성, AudioWorkletProcessor에서 로드

항목	규격 (초안)
초기화	`init(sampleRate, maxBlockSize)`
파라미터	`setParam(paramIndex, value)` — 인덱스는 `param_schema` 순서와 동일 (오토메이션도 이 함수로 전달)
MIDI (악기)	`noteOn(pitch, velocity)`, `noteOff(pitch)`
오디오 처리	`process(inPtrL, inPtrR, outPtrL, outPtrR, numFrames)` — 128 frame 블록 기준
메모리	입출력 버퍼는 WASM 힙에 미리 할당, 매 블록마다 새로 할당하지 않음

4.7 트랙 아코디언 — 디바이스 체인(Ableton식) + 패처(FL Studio식) 융합

> 하단 고정 패널 대신, **각 트랙 바로 밑으로 펼쳐지는 아코디언**에서 그 트랙의 소리 만들기를 모두 처리합니다. 기본은 Ableton Live처럼 **왼쪽→오른쪽 디바이스 체인**으로 쉽게 쓰고, 복잡한 라우팅이 필요할 때 FL Studio Patcher처럼 **노드를 선으로 연결**하는 뷰로 전환합니다. 두 뷰는 같은 데이터를 다르게 보여주는 것입니다.

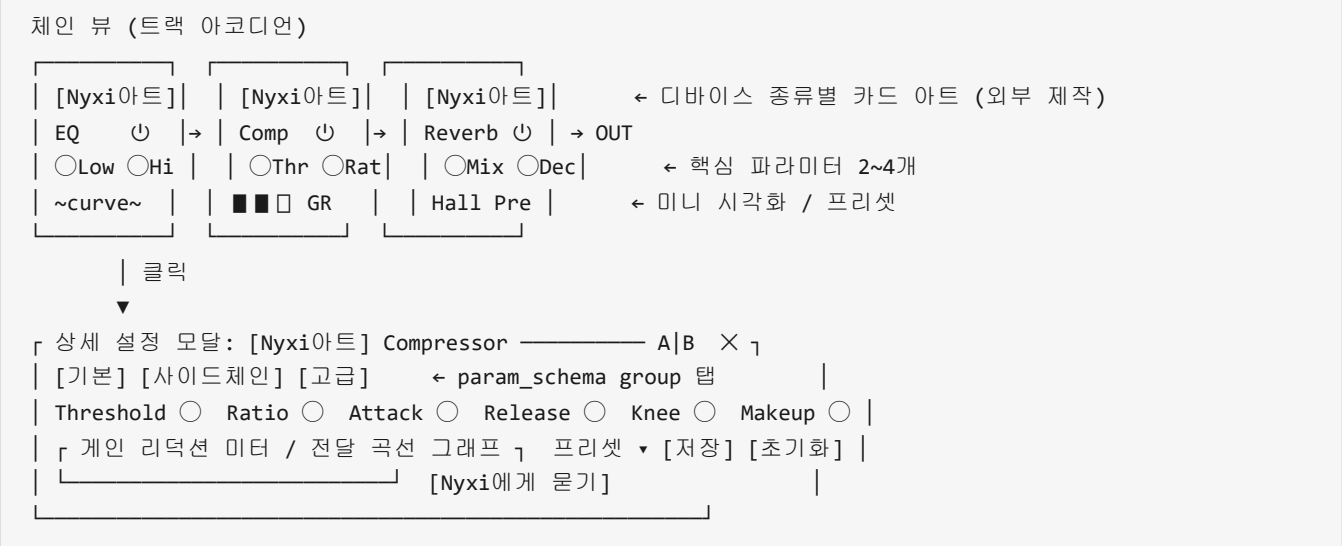
ID	요구사항	우선순위
FR-261	트랙 헤더의 ▼ 버튼(또는 트랙 이름 더블클릭)으로 아코디언 펼치기/접기. 여러 트랙 동시 펼침 가능, 펼침 상태는 브라우저에 기억	필수
FR-262	아코디언 상단 탭: **[디바이스]** · **[노트]**(MIDI 트랙: 피아노롤) / **[클립]**(오디오 트랙: 게인·트리밍·파형) · **[오토메이션]**(§4.10)	필수
FR-263	**체인 뷰(기본, Ableton식)**: 소스(가상악기 또는 오디오) → 디바이스 카드들 → 트랙 출력이 가로로 나열. 브라우저에서 디바이스를 드래그해 원하는 위치에 끼워 넣기, 카드 드래그로 순서 변경, 카드별 접기/바이패스/삭제/프리셋	필수
FR-264	디바이스는 **Nyxi 기반 카드**(§4.7.1)로 표시. 카드에서는 **핵심 파라미터만** 바로 조작하고, 전체 파라미터·시각화는 카드를 클릭해 여는 **상세 설정 모달**에서 조작	필수
FR-265	**패처 뷰(FL Studio Patcher식)**: [체인 ⇄ 패처] 토글. 디바이스가 노드로 표시되고, 출력 포트→입력 포트를 드래그해 선으로 연결. 고정 노드 `소스`·`트랙 출력` 존재	필수
FR-266	패처에서 가능한 것: **병렬 처리**(한 출력을 여러 디바이스로 분기 후 합치기), 연결선별 게인, 사이드체인 입력(포트 1), 노드 위치 저장	필수 (병렬·게인) / 권장 (사이드체인)
FR-267	모드 전환 규칙: 체인 → 패처 전환 시 현재 순서대로 연결선 자동 생성. 패처에서 **직렬 한 줄**이면 체인 뷰로 그대로 돌아갈 수 있고, 분기가 있으면 체인 뷰에서는 "패처 라우팅 사용 중" 표시 + 읽기 전용	필수
FR-268	순환 연결 금지(연결 시 즉시 검사), 연결 안 된 노드는 흐리게 표시하고 소리 처리에서 제외	필수
FR-269	저장: 라우팅 방식은 `tracks.routing_mode`, 노드 위치는 `track_plugins.node_x/node_y`, 연결선은 `track_plugin_links`	필수
FR-270	키보드 조작: 펼친 트랙에서 Tab으로 디바이스 간 이동, Del로 삭제, Ctrl+G로 선택 디바이스를 패처 병렬 그룹으로 묶기	권장



4.7.1 디바이스 카드 UI (Nyxi 기반) · 상세 설정 모달

> 이펙터·가상악기를 **Nyxi 캐릭터를 바탕으로 한 카드**로 보여줍니다. 카드는 "한눈에 알아보고 빠르게 조작", 모달은 "자세히 다듬기" 역할입니다. 카드 아트(디바이스 종류별 Nyxi 일러스트)는 **별도 AI 플랫폼에서 제작**해 제공받습니다(§4.9 FR-288). 에셋이 오기 전에는 기본(플레이스홀더) 아트로 개발합니다.

ID	요구사항	우선순위
FR-2181	**카드 구성**: ① 상단 — 디바이스 종류별 **Nyxi 카드 아트**(예: EQ용, 컴프용, 리버브용, 신스용), 디바이스 이름, 바이패스 토글 ② 중단 — **핵심 파라미터 2~4개** 노브(`param_schema`에서 `primary: true`인 항목) ③ 하단 — 현재 프리셋 이름, 미니 시각화(레벨 미터 또는 작은 EQ 곡선)	필수
FR-2182	카드 상태 표현: 바이패스 시 카드 흐림 + 아트의 "쉬는" 변형, 선택된 카드는 강조 테두리(테마 강조 색), 오토메이션이 걸린 노브는 색 테두리	필수
FR-2183	카드 아트 반응(선택): 재생 중 입력 레벨에 맞춰 아트가 살짝 반응(크기 펄스 등). 성능 설정에서 끌 수 있음	권장
FR-2184	**카드 클릭 → 상세 설정 모달** 열기 (노브·토글 위 클릭은 조작으로 처리하고, 카드의 빈 영역·아트·이름 클릭 또는 ⤢ 버튼이 모달을 엶)	필수
FR-2185	**모달 내용**: 상단 Nyxi 아트 + 디바이스 이름 / **전체 파라미터**(`param_schema`의 `group`별 탭 또는 섹션) / **전용 시각화**(EQ 곡선 드래그 편집, 컴프 게인 리덕션 미터, 리버브 디케이 그래프, 신스 엔벨로프 등) / 프리셋 탐색·저장 / **A/B 비교** / 초기화 / [Nyxi에게 이 디바이스 묻기]	필수 (시각화는 디바이스별 권장)
FR-2186	**모달 동작**: 재생을 멈추지 않음, 값 변경 즉시 소리에 반영, 드래그로 위치 이동, Esc·바깥 클릭·✕로 닫기, 한 번에 하나만(다른 카드 클릭 시 내용 전환), 모달 안에서도 노브 우클릭 → 오토메이션 레인 만들기, 모든 변경은 Undo 대상	필수
FR-2187	렌더링 방식: 카드·모달 모두 `param_schema`로 **자동 생성**이 기본. 전용 시각화가 필요한 디바이스만 `plugins.code` 기준으로 커스텀 모달 컴포넌트 등록	필수
FR-2188	같은 디자인 체계를 **패처 뷰 노드**(아트 아이콘 + 이름의 축소 카드)와 **트랙의 가상악기**에도 적용. 노드 클릭 시 같은 모달	필수
FR-2189	접근성: 모달은 포커스 트랩·Tab 순서·키보드로 노브 조절(↑ ↓, Shift=미세 조정), 아트에는 대체 텍스트	필수
FR-2190	협업(Phase 6): 다른 멤버가 같은 디바이스 모달을 열고 있으면 모달 상단에 그 멤버 아바타 표시, 값 변경은 실시간 동기화	권장



4.8 하단 패널 (토글형)

ID	요구사항	우선순위
FR-271	기본 상태는 **숨김**. 상단바의 버튼(💬 톡방, **Nyxi 아이콘**)을 누르면 화면 아래에서 올라옴. 같은 버튼을 다시 누르거나 Esc로 닫힘	필수
FR-272	탭 구조: **[톡방]**(Phase 6 협업 채팅) · **[Nyxi]**(Phase 5 AI 도우미). 아직 구현되지 않은 탭은 버튼을 숨김. 공유되지 않은 개인 프로젝트에서는 톡방 버튼 숨김	필수
FR-273	높이 드래그 조절, 마지막 높이·탭은 `user_settings.bottom_panel_state`에 저장	권장
FR-274	패널이 닫혀 있어도 새 채팅 메시지·Nyxi 답변 도착 시 버튼에 배지 표시	필수
FR-275	단축키: Ctrl+J(톡방), Ctrl+K(Nyxi) — §4.16 단축키 설정에서 변경 가능	권장

4.9 시퀀서 캐릭터 Nyxi (Fruity Dance식 · AI 도우미의 얼굴)

> FL Studio의 **Fruity Dance**(FL Chan이 춤추는 기능)처럼 시퀀서 화면 안에서 캐릭터가 춤춥니다. 같은 캐릭터를 **AI 도우미(Phase 5)의 아이콘이자 얼굴**로 써서, 캐릭터와 도우미를 하나의 경험으로 묶습니다. **오리지널 캐릭터**를 사용합니다(FL Chan 등 타사 캐릭터·에셋 사용 금지). 이름은 **Nyxi(닉시)** 로 확정했습니다(상표 검색 후 최종 확정). **캐릭터의 외형·애니메이션·카드 아트는 별도 AI 플랫폼에서 만들어 제공받습니다.** 이 프로젝트에서는 캐릭터를 디자인하거나 그리지 않고, 아래 FR-288의 **연동 규격**에 맞는 에셋을 받아 표시만 합니다.

ID	요구사항	우선순위
FR-281	캐릭터는 브라우저 페이지 **안의 떠 있는 창**(플로팅 레이어)으로 표시. 드래그로 위치 이동, 모서리 드래그로 크기 조절, 투명 배경	필수
FR-282	표시/숨김·위치·크기는 `user_settings`(`character_enabled`, `character_state`)에 저장되어 다음 접속에도 유지	필수
FR-283	**박자 동기 춤(Fruity Dance식)**: 재생 중에는 현재 BPM에 맞춰 춤 애니메이션 속도를 맞추고(1루프 = 1박 또는 1마디), 정지하면 대기(idle) 동작	필수
FR-284	춤 동작 선택(여러 개) 및 "춤 끄기" 옵션	권장
FR-285	**이벤트 반응**: 저장 완료, 출력 완료(Phase 3), 분리 완료(Phase 4), 새 채팅·코멘트(Phase 6) 시 짧은 리액션 + 말풍선	권장
FR-286	**아이콘 역할**: 상단바의 AI 도우미 버튼 아이콘 = Nyxi 얼굴(작은 크기, 재생 중엔 살짝 리듬을 탐). 캐릭터 창을 숨겨도 아이콘은 남음	필수
FR-287	**도우미와 연결**: 상태별 동작 — 대기(idle) / 춤(재생 중) / **생각 중**(AI 답변 생성 중) / **말하기**(답변 요약 말풍선). 말풍선 클릭 → 하단 패널 [Nyxi] 탭 열림. 캐릭터 클릭 → 짧은 질문 입력창(빠른 질문)	필수 (Phase 5 이후 활성)
FR-288	**에셋 연동 규격** (외부 AI 플랫폼 제작물 수령 기준): ① 상태별 애니메이션 — idle / dance / thinking / talking / react / guide ② 형식 — Lottie(JSON) 또는 스프라이트 시트(PNG) + 프레임 정보 JSON, 투명 배경 ③ dance는 **1루프 = 1박** 기준 길이 명시(BPM에 맞춰 재생 속도 조절) ④ 아이콘 24·32·48px ⑤ 디바이스 카드 아트(디바이스 종류별, §4.7.1) + 바이패스 변형 ⑥ 모든 파일 목록·상태 매핑을 담은 `manifest.json` ⑦ 보관 위치 `static/characters/{code}/`	필수
FR-288a	에셋 로더: `manifest.json`만 읽으면 동작하도록 구현 → 외부에서 에셋이 갱신되어도 코드 수정 없이 교체. 에셋이 없거나 깨지면 기본(플레이스홀더) 표시	필수
FR-289	성능 보호: `requestAnimationFrame`으로 렌더링, 탭이 비활성/캐릭터가 숨김일 때 애니메이션 정지, **오디오 스레드에 영향 없음**	필수

ID	요구사항	우선순위
FR-290	캐릭터/스킨 교체 (`character_code`)	추후

4.10 오토메이션 레인

ID	요구사항	우선순위
FR-2101	트랙 아코디언 **[오토메이션] 탭**에서 레인 추가: 대상 = 트랙 볼륨 / 팬 / 가상악기 파라미터 / 디바이스 파라미터 (마스터 트랙 포함)	필수
FR-2102	디바이스 노브 우클릭 → "오토메이션 레인 만들기"로 바로 생성	권장
FR-2103	레인 편집: 클릭으로 포인트 추가, 드래그 이동, 더블클릭 삭제, 구간 선택 후 일괄 이동·삭제, 포인트 사이 곡선 정도(`curve`) 조절	필수
FR-2104	그리기 도구(연필)로 자유 곡선 입력 → 포인트로 단순화 저장	권장
FR-2105	재생 시 Web Audio `AudioParam` 스케줄링(`setValueAtTime`, `linearRampToValueAtTime` 등)으로 샘플 단위 정확도, WASM 디바이스는 `setParam`으로 블록 단위 전달	필수
FR-2106	오토메이션이 있는 파라미터는 노브에 표시(색 테두리). 재생 중 노브를 손으로 움직이면 해당 레인 일시 해제(override) + [다시 켜기] 버튼	권장
FR-2107	레인 켜기/끄기, 접힌 트랙에서도 클립 영역에 얇은 오토메이션 선 미리보기	권장
FR-2108	저장: `automation_lanes`, `automation_points` (값은 파라미터 실제 단위로 저장)	필수

4.11 스케일 · 코드 도우미 (피아노롤)

ID	요구사항	우선순위
FR-2111	프로젝트 키(`music_key`)와 스케일(`scale_mode`: major, minor, dorian, mixolydian, 펜타토닉 등) 선택	필수
FR-2112	피아노롤에서 스케일 밖 음 줄을 흐리게 표시, 루트음 강조	필수
FR-2113	"스케일 스냅" 켜면 클릭·드래그한 노트가 스케일 안의 가장 가까운 음으로 맞춰짐	필수
FR-2114	**코드 스탬프**: 코드 종류(메이저/마이너/7th/sus 등) 선택 후 클릭 한 번에 코드 입력, 전위(inversion) 선택	필수
FR-2115	스케일 안 다이아토닉 코드 목록(Ⅰ~Ⅶ) 버튼 제공	권장
FR-2116	Phase 5 레퍼런스 분석 결과의 키를 [이 키 적용] 버튼으로 바로 설정	권장

4.12 오디오 녹음

ID	요구사항	우선순위
FR-2121	오디오 트랙 헤더의 녹음 대기(●) 버튼 → 입력 장치 선택(`getUserMedia`, 마이크 권한 요청), 입력 레벨 미터	필수
FR-2122	녹음: 재생과 동시에 녹음 → 정지 시 WAV로 업로드, `samples.source='RECORDING'`으로 저장, 해당 위치에 오디오 클립 생성	필수
FR-2123	입력 모니터링 on/off (헤드폰 사용 권장 안내, 하울링 방지)	필수
FR-2124	**레이턴시 보정**: 오디오 입출력 지연(`AudioContext.outputLatency` + 입력 지연)을 측정·설정값으로 저장해 녹음 클립 위치 자동 보정. 수동 보정값(ms) 입력 가능	필수
FR-2125	루프 녹음(테이크 여러 개) 및 테이크 선택	추후
FR-2126	녹음 중 용량 확인(FR-227), 브라우저 권한 거부 시 안내	필수

4.13 패턴(링크 클립) · 클립 런처

> **패턴**: FL Studio 패턴처럼, MIDI 노트 묶음을 "패턴"으로 만들고 여러 클립이 같은 패턴을 가리키게 합니다. 패턴 하나를 고치면 그 패턴을 쓰는 모든 클립이 같이 바뀝니다. **클립 런처**: Ableton 세션 뷰처럼, 트랙 × 씬(행) 격자에 클립을 넣고 즉석에서 재생·조합해 봅니다.

ID	요구사항	우선순위
FR-2131	모든 MIDI 클립은 패턴(`patterns`)을 가리킴. 노트는 패턴에 속함(`midi_notes.pattern_id`)	필수
FR-2132	복제 두 가지: **링크 복제**(Ctrl+D, 같은 패턴 공유) / **독립 복제**(Ctrl+Shift+D, 패턴 복사). 링크된 클립은 같은 색 + 링크 아이콘	필수
FR-2133	[고유하게 만들기]: 링크된 클립 하나를 패턴 복사해 분리	필수
FR-2134	브라우저 패널에 프로젝트 패턴 목록 → 드래그로 타임라인 배치, 패턴 이름·색 변경	권장
FR-2135	**클립 런처 뷰**: [어레인지 ⇄ 런처] 전환. 트랙 × 씬 격자, 슬롯 클릭으로 해당 클립 루프 재생(다음 마디 경계에서 시작·정지 = 퀀타이즈 런치), 씬 행 단위 동시 재생	권장
FR-2136	런처에서 연주한 결과를 어레인지(타임라인)에 녹음하는 기능	추후
FR-2137	저장: `patterns`, `clips.pattern_id`, `launcher_scenes`, `clips.scene_id`(NULL이면 타임라인 클립)	필수

4.14 프로젝트 템플릿 (적용 전 확인 필수)

ID	요구사항	우선순위
FR-2141	시스템 장르 템플릿(예: Lo-fi, House, Trap, Pop Ballad): BPM·박자·키, 트랙 구성, 디바이스 체인, 기본 패턴(선택) 포함 → `project_templates`	필수
FR-2142	내 템플릿 저장: 현재 프로젝트 구성을 템플릿으로 저장(샘플·노트 포함 여부 선택)	권장
FR-2143	**적용 전 확인창(Confirm) 필수** — 사용처 3곳: ① 새 프로젝트를 템플릿으로 만들 때 ② 기존 프로젝트에 템플릿 적용(트랙 추가/교체) ③ AI 레퍼런스 기반 새 프로젝트(PRD 05). 확인창에는 **무엇이 생성·변경되는지 요약**(BPM 변경, 추가될 트랙 N개, 교체될 디바이스 등)과 [적용] [취소] 버튼	필수
FR-2144	기존 프로젝트에 적용할 때는 [추가(기존 유지)] / [교체(기존 트랙 대체)] 선택. **교체**는 경고 문구 + 적용 직전 자동 스냅샷(`BEFORE_TEMPLATE`, PRD 06 §4.6) 생성	필수
FR-2145	적용은 Undo 한 번으로 전체 되돌리기 가능	필수

4.15 튜토리얼 모드 (Nyxi 안내)

ID	요구사항	우선순위
FR-2151	첫 접속 시 "튜토리얼을 시작할까요?" (건너뛰기 가능) → Nyxi가 말풍선으로 단계 안내	필수
FR-2152	기본 코스: ① 샘플 끌어오기 → ② MIDI 트랙 만들고 노트 찍기 → ③ 트랙 펼쳐 이펙트 넣기 → ④ 재생·저장 → ⑤ WAV 출력. 각 단계는 **실제 동작을 감지**해 다음으로 진행(해당 UI 하이라이트)	필수
FR-2153	추가 코스(선택): 패처 라우팅, 오토메이션, 스케일·코드 도우미, AI 레퍼런스 분석, 협업 초대	권장
FR-2154	튜토리얼 전용 연습 프로젝트(템플릿)로 진행, 실제 작업 프로젝트를 건드리지 않음	필수
FR-2155	진행 상태 저장(`user_settings.tutorial_state`), 설정 또는 커맨드 팔레트에서 다시 보기	필수

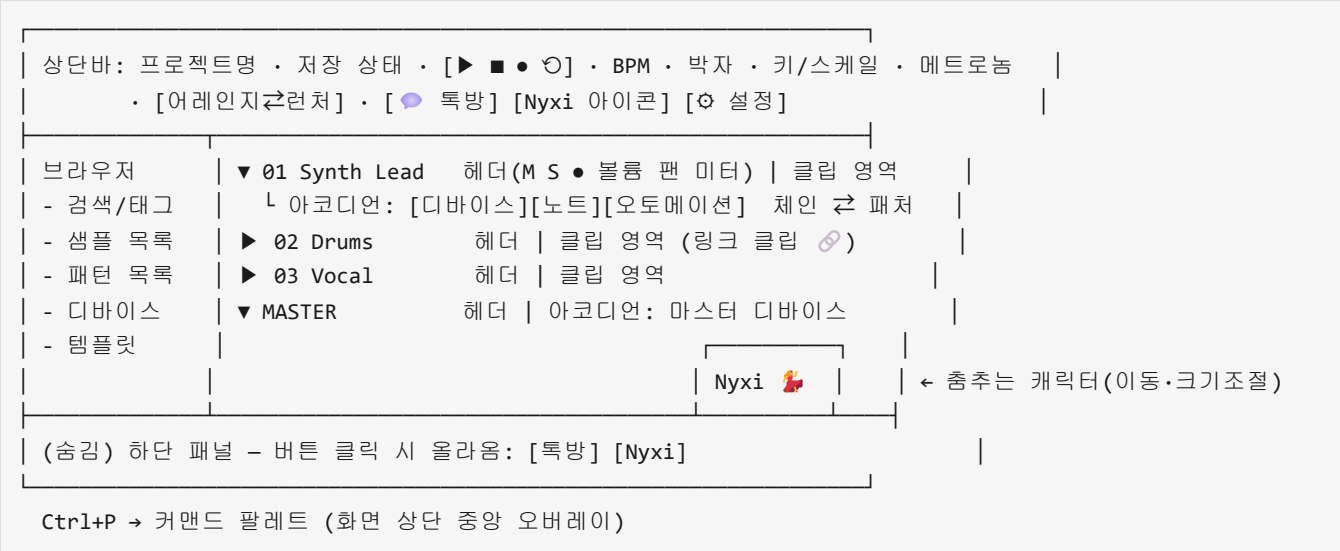
4.16 UI 커스터마이징 · 단축키 · 커맨드 팔레트

ID	요구사항	우선순위
FR-2161	**환경 설정 창**: `studioNx.html` 상단바 ⚙ 버튼 + `userInfo.html` [환경 설정] 탭. 변경 즉시 미리보기, 계정에 저장(`user_settings`)되어 다른 기기에서도 동일	필수
FR-2162	**테마**: 다크 / 라이트 / 시스템 따라가기, **강조 색(accent)** 선택, 트랙 기본 색 팔레트	필수
FR-2163	**크기·가독성**: UI 배율(80~150%), 글자 크기, 트랙 기본 높이, 고대비 모드	필수
FR-2164	**레이아웃**: 브라우저 패널 위치(좌/우)·너비, 아코디언 기본 탭, 하단 패널 기본 높이, 파형 표시 방식(채움/선), 미터 스타일	권장
FR-2165	**편집 기본값**: 기본 스냅 단위, 새 트랙 기본 악기, 자동 저장 간격, 메트로놈 소리, 오디오 버퍼(지연) 설정	권장
FR-2166	**캐릭터 설정**: 표시, 춤 on/off, 말풍선 on/off, 크기, 카드 아트 반응 on/off, 스킨(추후)	필수
FR-2166a	**Nyxi 팁 설정**: 끄기 / 중요한 것만(기본) / 모두, [다시 보지 않기] 한 팁 목록 초기화 → `user_settings.tips_level`, `tips_state` (팁 동작은 PRD 05 §4.5)	필수
FR-2167	**단축키 사용자 지정**: 모든 명령의 단축키 목록·검색·변경, 충돌 감지 경고, 기본값으로 되돌리기(확인창) → `user_settings.keybindings`	필수
FR-2168	**커맨드 팔레트**(Ctrl+P): 모든 명령(트랙 추가, 디바이스 삽입, 출력, 패처 열기, 튜토리얼 등)을 이름으로 검색·실행, 최근 사용 명령 우선, 각 명령 옆에 단축키 표시	필수
FR-2169	명령 체계 통일: 메뉴·단축키·커맨드 팔레트·**Nyxi 도우미 액션 버튼**(PRD 05 FR-535)이 같은 명령 레지스트리(명령 ID)를 사용	필수
FR-2170	설정 내보내기/가져오기(JSON), 전체 초기화(확인창)	권장
FR-2171	언어 설정(한국어 기본, 영어 추후)	추후

5. 비기능 요구사항

구분	요구사항
오디오 성능	DSP는 AudioWorklet 스레드에서 처리. 16트랙 + 트랙당 이펙트 3개 + 오토메이션 레인 10개 재생 시 끊김 없음(데스크톱 Chrome 기준)
지연	외부 MIDI 키보드 입력 → 소리 출력 지연 체감 30ms 이하 목표
녹음 정확도	레이턴시 보정 후 녹음 클립 위치 오차 ±5ms 이내
UI 성능	타임라인·피아노롤·오토메이션은 Canvas 렌더링 권장, 노트 수천 개에서도 스크롤·줌이 부드러울 것
저장	저장 실패 시 사용자에게 알리고 로컬(브라우저 저장소)에 임시 백업
보안	업로드 파일은 확장자 + MIME + 실제 헤더 검사, 파일명은 서버에서 UUID로 변경. 마이크 권한은 녹음 시점에만 요청
브라우저	오디오 재생은 사용자 클릭 이후 `AudioContext.resume()` (자동재생 정책 대응)
라우팅 변경	패처 연결을 바꿀 때 오디오 그래프는 재생을 멈추지 않고 교체(짧은 페이드로 클릭 노이즈 방지)
캐릭터	캐릭터 표시 중에도 CPU 사용 증가가 체감되지 않을 것. 캐릭터 기능 오류가 시퀀서 동작을 막지 않도록 분리
디바이스 모달	모달을 열고 닫아도 오디오가 끊기지 않을 것. 모달 열기 100ms 이내, 시각화는 60fps 목표(오디오 스레드와 분리)
접근성	고대비 모드, 키보드만으로 주요 기능 사용 가능(커맨드 팔레트), 색에만 의존하지 않는 상태 표시
품질	오디오 엔진·플러그인 변경 시 **오디오 회귀 테스트**(PRD 00 §7.1) 통과 필수

6. 화면 구성 (`studioNx.html`)



- **하단 고정 패널(피아노롤/믹서/플러그인 탭)은 두지 않습니다.** 피아노롤·디바이스·오토메이션은 트랙 아코디언, 믹서는 트랙 헤더 + 마스터 트랙에서 처리합니다.
- 하단 패널은 버튼을 누를 때만 나타나며 내용은 Phase 5(Nyxi 도우미), Phase 6(톡방)에서 채웁니다.
- Phase 5의 "AI 레퍼런스 분석" 패널, Phase 6의 "코멘트·스냅샷" 패널은 우측 사이드 패널(토글)로 추가됩니다.

7. API 초안 (FastAPI)

Method	Path	설명
GET	`/projects/{id}/full`	프로젝트 전체 상태(트랙, 클립, 패턴, 노트, 디바이스, 연결선, 오토메이션, 씬) 한 번에 조회
PUT	`/projects/{id}/full`	프로젝트 전체 상태 저장 (트랜잭션으로 일괄 반영)
PATCH	`/projects/{id}/settings`	BPM, 박자, 키, 스케일
POST	`/samples`	샘플 업로드 (multipart, 다중 파일, 녹음 포함 `source`) — 용량 초과 시 413
GET	`/samples?query=&tag=&page=`	내 라이브러리 목록
GET	`/samples/{id}/file`	오디오 파일 스트리밍 (본인/프로젝트 멤버만, ETag로 캐시 검증)
GET	`/samples/{id}/waveform`	파형 데이터
PATCH	`/samples/{id}`	이름 변경
DELETE	`/samples/{id}`	소프트 삭제
PUT	`/samples/{id}/tags`	태그 설정
GET	`/tags?category=`	태그 목록
GET	`/plugins`	사용 가능한 플러그인 목록 + `param_schema`
GET	`/plugins/{id}/presets`	시스템 + 내 프리셋
POST	`/plugins/{id}/presets`	내 프리셋 저장
DELETE	`/presets/{preset_id}`	내 프리셋 삭제
POST	`/projects/{id}/import-midi`	`.mid` 파일 → 트랙/패턴/클립/노트 생성 (프론트 파싱으로 대체 가능)
GET	`/templates`	시스템 + 내 템플릿 목록

Method	Path	설명
GET	`/templates/{id}/preview`	템플릿 적용 시 변경 요약 (**확인창 내용**)
POST	`/templates`	현재 프로젝트를 내 템플릿으로 저장
POST	`/projects/{id}/apply-template`	템플릿 적용 (body: template_id, mode=ADD/REPLACE, `confirmed=true` 필수)
DELETE	`/templates/{id}`	내 템플릿 삭제
GET	`/users/me/settings`	UI 설정(테마·단축키·캐릭터·하단 패널·튜토리얼) 조회
PATCH	`/users/me/settings`	UI 설정 부분 저장 (캐릭터 위치 이동 등은 디바운스 후 전송)

> 패처 라우팅·오토메이션·패턴·씬은 `/projects/{id}/full` 저장·조회에 함께 포함됩니다. 저장 방식: Phase 2는 **전체 저장(PUT full)** 으로 단순하게 시작하고, Phase 6에서 변경 단위(operation) 저장으로 확장합니다. 템플릿 적용 API는 `confirmed=true`가 없으면 400을 반환합니다 — 프론트 확인창을 거치지 않은 적용을 서버에서도 막습니다.

8. 사용 테이블

> 컬럼 상세는 DB.md §3.2 참고 (시간 단위 규칙은 §1.4)

- `samples`, `tags`, `sample_tags`
- `plugins`, `plugin_presets`
- `tracks` (track_type MASTER, routing_mode), `track_plugins` (node_x/node_y), `track_plugin_links`
- `patterns`, `clips` (pattern_id, scene_id), `midi_notes` (pattern_id), `launcher_scenes`
- `automation_lanes`, `automation_points`
- `project_templates`
- `projects` (bpm, time_sig_*, music_key, scale_mode, template_id)
- `users` (storage_quota_bytes, storage_used_bytes)
- `user_settings` (테마·배율·단축키·캐릭터·하단 패널·튜토리얼)

9. AI 역할 구분

영역	이 Phase에서 AI가 하는 일
HTML / CSS	`studioNx.html` 레이아웃, 트랙 헤더·아코디언, Nyxi 디바이스 카드·상세 설정 모달 레이아웃(아트 자리는 플레이스홀더), 패처 노드/연결선, 오토메이션 레인, 런처 격자, 하단 패널, 캐릭터 창, 설정 창, 커맨드 팔레트, 템플릿 확인창, **테마 CSS 변수 체계** 등 **작성**
JavaScript	오디오 엔진, 스케줄러, 타임라인/피아노롤, 체인·패처 라우팅 그래프, 오토메이션 스케줄링, 녹음·레이턴시 보정, 패턴/런처, Web MIDI, Undo 관리자, 명령 레지스트리, 로컬 캐시, 캐릭터 애니메이션, 튜토리얼 진행 엔진 등 **모듈 구조와 함수 시그니처 템플릿**만. 구현은 사용자 (요청 시에만 AI)
캐릭터 에셋	**만들지 않음.** Nyxi 외형·애니메이션·카드 아트는 별도 AI 플랫폼에서 제작. 이 프로젝트의 AI는 FR-288 연동 규격과 플레이스홀더만 다룸
Backend	`samples`, `plugins`, `templates`, `projects/full`, `users/me/settings` 라우터·스키마 **템플릿**만
C++	**작성하지 않음.** §4.6 인터페이스 규격 문서까지만

영역	이 Phase에서 AI가 하는 일
DB	Phase 2 테이블 생성, 기본 `plugins`·`tags`·시스템 프리셋·시스템 템플릿 시드 데이터
문서	AI가 작성한 부분은 `docs/ai-notes/`에 설명

10. 완료 기준 (Acceptance Criteria)

- [ ] 외부 오디오 파일을 드롭하면 라이브러리에 저장되고, 새로고침 후에도 남아 있다.
- [ ] 라이브러리 샘플을 타임라인에 끌어 놓고 재생하면 정확한 박자 위치에서 소리가 난다.
- [ ] 피아노롤에서 클릭으로 입력한 노트가 선택된 신스로 재생된다.
- [ ] 외부 MIDI 키보드 연주가 실시간으로 들리고, 녹음하면 MIDI 클립으로 남는다.
- [ ] 트랙에 이펙트 3개를 걸고 순서를 바꾸거나 바이패스하면 소리에 반영된다.
- [ ] BPM을 바꿔도 클립·노트·오토메이션 포인트의 마디 위치가 유지된다.
- [ ] 저장 → 페이지 새로고침 → 불러오기 시 모든 트랙·클립·패턴·노트·디바이스·오토메이션 값이 동일하다.
- [ ] Undo/Redo가 클립 이동, 노트 추가/삭제, 파라미터 변경, 템플릿 적용에 대해 동작한다.
- [ ] 다른 회원의 샘플 파일 URL로 직접 접근해도 받을 수 없다.
- [ ] 트랙을 펼치면 바로 밑에 아코디언이 열리고, 디바이스를 드래그해 체인에 끼워 넣을 수 있다.
- [ ] 패처 뷰에서 병렬 분기(예: 원음 + 디스토션 혼합)를 연결하면 소리에 반영되고, 저장 후 다시 열어도 연결이 유지된다. 순환 연결은 만들 수 없다.
- [ ] 하단 패널은 기본적으로 숨겨져 있고, 버튼을 누를 때만 나타난다.
- [ ] Nyxi가 페이지 안에 떠 있고, 재생 시 BPM에 맞춰 춤추며, 숨김/위치 설정이 다음 접속에도 유지된다. 상단바 도우미 버튼 아이콘이 Nyxi 얼굴이다.
- [ ] 필터 컷오프 오토메이션을 그리면 재생 시 소리가 그에 맞게 변하고, 출력(Phase 3) 결과에도 반영된다.
- [ ] C minor 스케일을 고르면 스케일 밖 음이 흐려지고, 스케일 스냅·코드 스탬프가 동작한다.
- [ ] 마이크 녹음 클립이 레이턴시 보정되어 박자에 맞게 놓인다.
- [ ] 링크 복제한 패턴 하나를 고치면 같은 패턴의 모든 클립이 바뀌고, [고유하게 만들기] 후에는 따로 바뀐다.
- [ ] 클립 런처에서 씬을 실행하면 다음 마디 경계에서 해당 클립들이 동시에 재생된다.
- [ ] 템플릿 적용 시 **반드시 확인창**이 뜨고, [취소]하면 아무것도 바뀌지 않는다. 교체 적용 전에는 자동 스냅샷이 생긴다.
- [ ] 첫 접속 시 튜토리얼을 진행할 수 있고, 중간에 나갔다 와도 이어서 할 수 있다.
- [ ] 테마·UI 배율·단축키를 바꾸면 즉시 적용되고, 다른 브라우저에서 로그인해도 유지된다. 단축키 충돌 시 경고가 뜬다.
- [ ] Ctrl+P로 "트랙 추가"를 검색해 실행할 수 있다.
- [ ] 디바이스가 Nyxi 카드로 표시되고 핵심 노브를 카드에서 바로 돌릴 수 있으며, 카드를 클릭하면 상세 설정 모달이 열려 전체 파라미터를 조작할 수 있다(재생은 끊기지 않음).
- [ ] 외부에서 받은 에셋 폴더를 교체하면 코드 수정 없이 캐릭터·카드 아트가 바뀐다. 에셋이 없으면 플레이스홀더가 보인다.
- [ ] 두 번째로 연 프로젝트는 샘플을 캐시에서 읽어 첫 오픈보다 빨리 열린다.
- [ ] 저장 용량을 넘으면 업로드가 거부되고 안내가 표시된다.

Studio Nx — PRD 03. Phase 3: 오디오 출력 (Export)

항목	내용
기획서 기능	5) wav 및 mp3 등 오디오 파일 형식에 따라 output하는 기능
선행 Phase	Phase 2 (시퀀서, 믹서, 플러그인)
관련 페이지	`studioNx.html` (출력 모달), `myProject.html` (출력 이력, 선택)
전체 개요	00_PRD_Overview.md

---

1. 목표 / 배경

- 시퀀서에서 만든 곡을 **WAV / MP3 (+FLAC)** 파일로 내보내 실제로 쓸 수 있게 합니다. 이 Phase까지 끝나면 **MVP 완성**입니다.
- 출력 결과는 "시퀀서에서 들리는 소리와 동일"해야 하므로, 실시간 재생과 같은 오디오 그래프를 사용해 렌더링합니다.

2. 범위

In scope

- 전체 곡 / 루프 구간 / 지정 구간 출력
- 포맷: WAV, MP3, FLAC(권장)
- 옵션: 샘플레이트, 비트뎁스(WAV/FLAC), 비트레이트(MP3), 정규화(Normalize) 여부, 끝부분 꼬리(리버브 tail) 길이
- 출력 진행률 표시, 완료 후 다운로드, 출력 이력 조회
- 출력 결과를 내 샘플 라이브러리에 추가(선택)

Out of scope

- 트랙별 개별 출력(Stem Export) → 추후 검토 (구조상 `export_jobs`에 옵션 컬럼 추가로 확장 가능)
- 영상 포맷, 스트리밍 플랫폼 직접 업로드 → 지원하지 않음

3. 사용자 스토리

ID	사용자 스토리
US-301	사용자로서, 완성한 곡을 WAV로 받아 다른 DAW나 마스터링에 쓰고 싶다.
US-302	사용자로서, 친구에게 보낼 용도로 MP3 320kbps로 가볍게 받고 싶다.
US-303	사용자로서, 루프 구간만 잘라 내보내 샘플로 다시 쓰고 싶다.
US-304	사용자로서, 출력이 얼마나 진행됐는지 보고 싶다.

4. 기능 요구사항

ID	요구사항	우선순위
FR-301	출력 모달: 포맷, 범위(전체/루프/직접 지정), 샘플레이트(44.1k/48k), 비트뎁스(16/24/32f), MP3 비트레이트(128/192/320), 정규화, tail 길이(초)	필수
FR-302	**렌더링 방식 — 기본: 클라이언트 렌더링(`render_mode='CLIENT'`)** — `OfflineAudioContext`에 동일한 오디오 그래프(악기·이펙트·믹서)를 구성해 실시간보다 빠르게 렌더링	필수
FR-303	WAV 인코딩: 클라이언트에서 PCM → WAV 헤더 작성	필수
FR-304	MP3 / FLAC 인코딩: 렌더링한 WAV를 서버로 업로드 → 서버에서 인코딩(예: ffmpeg) → 결과 저장. 또는 클라이언트 인코더(WASM) 사용 — 구현 시 선택	필수
FR-305	출력 작업을 `export_jobs`에 기록: 상태(PENDING → PROCESSING → DONE/FAILED), 진행률, 결과 경로, 오류 메시지	필수
FR-306	진행률 표시: 클라이언트 렌더링 진행률 + 서버 인코딩 상태(폴링, 예: 1초 간격)	필수
FR-307	완료 시 다운로드 버튼. 파일명 기본값: `{프로젝트명}_{YYYYMMDD_HHmm}.{ext}`	필수
FR-308	"라이브러리에 추가" 옵션: 결과를 `samples`에 `source='EXPORT'`로 등록	권장
FR-309	출력 이력: 프로젝트별 최근 출력 목록, 재다운로드, 삭제	권장
FR-310	렌더링 중 시퀀서 편집은 막거나(모달), 렌더링 시작 시점의 상태 스냅샷을 사용	필수
FR-311	출력 결과 클리핑 감지: 피크가 0dBFS를 넘으면 경고 표시	권장
FR-313	렌더링에 **오토메이션·패처 라우팅·링크 클립(패턴)** 이 모두 반영됨 (클립 런처 상태는 반영하지 않음 — 타임라인 기준)	필수
FR-314	출력 시작 전 저장 용량 확인(PRD 01 FR-135). 출력 파일은 30일 후 자동 정리(이력은 남고 파일만 삭제, 정리 7일 전 알림)	필수
FR-312	(확장) 서버 렌더링(`render_mode='SERVER'`) — 저사양 기기 대응. C++/WASM 엔진을 서버에서도 쓸 수 있을 때 검토	추후

5. 비기능 요구사항

- 3분 길이, 16트랙 프로젝트 기준 WAV 출력이 데스크톱에서 실제 재생 시간보다 빠르게 끝날 것.
- 출력 결과와 시퀀서 재생 결과가 들어서 구별되지 않을 것(같은 그래프, 같은 파라미터).
- 출력 렌더러는 **오디오 회귀 테스트**(PRD 00 §7.1)의 렌더링 엔진으로도 재사용.
- 서버 인코딩은 백그라운드 작업으로 처리해 API 응답을 막지 않음.
- 결과 파일은 일정 기간(예: 30일) 후 정리 가능하도록 `created_at` 기준 정책 마련.
- 다운로드는 요청자 또는 프로젝트 멤버만 가능.

6. 화면 구성

- `studioNx.html` 상단바 **[Export]** 버튼 → 출력 모달
  - 옵션 영역 → [출력 시작] → 진행 바 → 완료 시 [다운로드] [라이브러리에 추가]
  - 하단에 "최근 출력" 목록 (포맷, 일시, 크기, 다운로드)

7. API 초안 (FastAPI)

Method	Path	설명
POST	`/projects/{id}/exports`	출력 작업 생성 (옵션 저장, `export_id` 반환)
POST	`/exports/{export_id}/upload`	클라이언트 렌더링 WAV 업로드 → 필요 시 서버 인코딩 시작
GET	`/exports/{export_id}`	상태·진행률 조회
GET	`/exports/{export_id}/download`	결과 파일 다운로드
GET	`/projects/{id}/exports`	출력 이력
DELETE	`/exports/{export_id}`	결과 삭제
POST	`/exports/{export_id}/to-library`	결과를 `samples`에 등록

8. 사용 테이블

> 컬럼 상세는 DB.md §3.3 참고

- `export_jobs`
- `samples` (라이브러리 추가 시, `source='EXPORT'`)
- `projects`, `tracks`, `track_plugins`, `clips`, `midi_notes` (렌더링 입력, 읽기 전용)

9. AI 역할 구분

영역	이 Phase에서 AI가 하는 일
HTML / CSS	출력 모달, 진행 바, 출력 이력 목록 **작성**
JavaScript	오프라인 렌더러, WAV 인코더, 진행률 폴링 모듈 **템플릿**만
Backend	`exports` 라우터, 백그라운드 인코딩 작업 함수 **템플릿**만
C++	작성하지 않음 (서버 렌더링 검토 시 인터페이스만 문서화)
DB	`export_jobs` 생성, DB.md 갱신

10. 완료 기준 (Acceptance Criteria)

- [ ] 같은 프로젝트를 WAV / MP3 / FLAC으로 각각 출력해 다운로드할 수 있다.
- [ ] 출력 파일의 길이·샘플레이트·비트뎁스(비트레이트)가 선택한 옵션과 일치한다.
- [ ] 출력 결과가 시퀀서 재생 소리와 동일하게 들린다(이펙트, 볼륨, 팬 반영).
- [ ] 루프 구간 출력 시 해당 구간만 담긴다.
- [ ] 진행률이 표시되고, 실패 시 원인 메시지가 표시되며 `export_jobs.status='FAILED'`로 기록된다.
- [ ] "라이브러리에 추가"한 결과를 다시 타임라인에 배치할 수 있다.

Studio Nx — PRD 04. Phase 4: 악기 분리 (Instrument / Stem Separation)

항목	내용
기획서 기능	4) 샘플 등을 별도로 instrument separation하는 기능
선행 Phase	Phase 2 (샘플 라이브러리)
관련 페이지	`studioNx.html` (라이브러리 패널, 분리 모달)
전체 개요	00_PRD_Overview.md

---

1. 목표 / 배경

- 라이브러리에 있는 오디오(완성곡, 루프, 샘플)를 **보컬·드럼·베이스·기타 등 악기별 stem으로 분리**해, 각각을 새 샘플로 쓸 수 있게 합니다.
- 예: 드럼 루프만 뽑아 쓰기, 보컬을 빼고 반주만 남기기, 레퍼런스 곡의 베이스 라인만 들어보기.
- 분리 결과는 **일반 샘플과 똑같이** 라이브러리에 등록되므로, 시퀀서·출력·AI 분석 기능을 그대로 재사용합니다.

2. 범위

In scope

- 라이브러리 샘플 선택 → 분리 모드 선택 → 서버 비동기 분리 → 결과 stem들을 라이브러리에 자동 등록
- 분리 모드: 2-stem(보컬/반주), 4-stem(보컬/드럼/베이스/기타), 6-stem(+기타/피아노)
- 작업 진행 상태 표시, 작업 목록, 실패 시 재시도
- 분리 결과를 타임라인에 바로 트랙으로 펼치기(선택)

Out of scope

- 실시간(재생 중) 분리 → 지원하지 않음
- 드럼 내부 세부 분리(킥/스네어/하이햇) → 추후 검토
- 분리 모델 직접 학습 → 지원하지 않음 (사전 학습 모델 사용)

3. 사용자 스토리

ID	사용자 스토리
US-401	사용자로서, 라이브러리의 곡을 우클릭해 "악기 분리"를 누르면 stem들이 생기길 원한다.
US-402	사용자로서, 분리가 오래 걸려도 다른 작업을 계속하고, 끝나면 알림을 받고 싶다.
US-403	사용자로서, 분리된 stem들을 한 번에 타임라인 트랙들로 펼치고 싶다.

4. 기능 요구사항

ID	요구사항	우선순위
FR-401	라이브러리 샘플 컨텍스트 메뉴 → "악기 분리" → 모드 선택 모달(2/4/6-stem, 예상 소요시간 안내)	필수
FR-402	작업 생성 시 `separation_jobs`에 `PENDING`으로 저장 후 **백그라운드 작업 큐**에서 처리 (API는 즉시 응답)	필수
FR-403	분리 엔진: 사전 학습 모델(후보: **Demucs / HT-Demucs** 계열) 사용, `model_name`에 기록	필수
FR-404	진행 상태: `PROCESSING` 중 진행률 갱신, 클라이언트는 폴링(또는 Phase 6 이후 WebSocket 알림)	필수
FR-405	완료 시 stem별 파일을 저장하고 각각 `samples`에 `source='SEPARATION'`으로 등록, `separation_stems`에 연결. 이름 규칙: `{원본이름} - {Vocals}`	필수
FR-406	결과 stem들에 자동 태그(예: INSTRUMENT 카테고리의 `Vocals`, `Drums`) 부여	권장
FR-407	"타임라인에 펼치기": 각 stem을 새 AUDIO 트랙 + 같은 시작 위치 클립으로 배치	권장
FR-408	작업 목록: 진행 중/완료/실패, 실패 시 오류 메시지와 [재시도]	필수
FR-409	제한: 원본 길이 최대 10분, 사용자당 동시 작업 1~2개 (설정값)	필수
FR-410	원본 샘플이 삭제(소프트)되어도 이미 만들어진 stem은 유지	필수
FR-411	작업 생성 전 **예상 결과 용량**(원본 크기 × stem 수)을 남은 저장 용량과 비교, 부족하면 생성 거부 + [용량 정리] 안내 (PRD 01 §4.4)	필수

5. 비기능 요구사항

구분	요구사항
처리 환경	분리는 연산량이 크므로 **별도 워커 프로세스**에서 실행. GPU가 있으면 사용, 없으면 CPU (느림을 UI에 안내)
처리 시간	3분 곡 4-stem 기준 CPU에서 수 분 이내 목표 (환경에 따라 측정 후 조정)
안정성	워커가 죽어도 작업이 `PROCESSING`에 영원히 머무르지 않도록 타임아웃 → `FAILED` 처리
품질	결과 stem의 샘플레이트·길이는 원본과 동일
저장공간	stem은 원본 크기 × stem 수만큼 용량이 늘어나므로, 사용자별 저장 용량 표시 고려
권리	업로드/분리 대상 음원의 저작권 책임은 사용자에게 있음을 이용약관·UI에 안내

6. 화면 구성

- 라이브러리 패널: 샘플 우클릭 → "악기 분리"
- 분리 모달: 모드 선택(그림으로 stem 구성 표시) → [분리 시작]
- 라이브러리 상단 "작업" 배지: 진행 중 작업 수 표시, 클릭 시 작업 목록(진행률 바)
- 완료 토스트: "OO 분리가 완료되었습니다 [타임라인에 펼치기]"

7. API 초안 (FastAPI)

Method	Path	설명
POST	`/separations`	분리 작업 생성 (body: `source_sample_id`, `stem_mode`)
GET	`/separations?status=`	내 분리 작업 목록
GET	`/separations/{id}`	상태·진행률·결과 stem 목록

Method	Path	설명
POST	`/separations/{id}/retry`	실패 작업 재시도
DELETE	`/separations/{id}`	작업 기록 삭제 (stem 샘플은 유지)

워커 측 (내부 함수, 템플릿만):

- `run_separation(separation_id)` — 원본 로드 → 모델 추론 → stem 저장 → `samples`/`separation_stems` INSERT → 상태 갱신

8. 사용 테이블

> 컬럼 상세는 DB.md §3.4 참고

- `separation_jobs`
- `separation_stems`
- `samples` (원본 조회 + 결과 등록 `source='SEPARATION'`)
- `tags`, `sample_tags` (자동 태그)
- `tracks`, `clips` (타임라인에 펼치기)

9. AI 역할 구분

영역	이 Phase에서 AI가 하는 일
HTML / CSS	분리 모달, 작업 목록, 토스트 **작성**
JavaScript	작업 생성·폴링·펼치기 로직 **템플릿**만
Backend	`separations` 라우터, 워커 함수 **템플릿**만. 모델 연동 코드는 요청 시 AI 구현
C++	해당 없음
DB	`separation_jobs`, `separation_stems` 생성, DB.md 갱신
문서	사용 모델·라이선스·처리 흐름을 `docs/ai-notes/`에 정리

10. 완료 기준 (Acceptance Criteria)

- [ ] 라이브러리 샘플을 4-stem 모드로 분리하면 4개의 새 샘플이 라이브러리에 생긴다.
- [ ] 분리 진행 중에도 시퀀서 편집·재생이 가능하다.
- [ ] stem 4개를 동시에 재생하면 원본과 거의 같은 소리가 난다.
- [ ] 실패한 작업은 오류 메시지가 보이고 재시도할 수 있다.
- [ ] 다른 회원의 샘플로는 분리 작업을 만들 수 없다.
- [ ] "타임라인에 펼치기"로 stem 수만큼 트랙이 생성된다.

Studio Nx — PRD 05. Phase 5: AI 레퍼런스 분석 · 설정 추천

항목	내용
기획서 기능	3) 레퍼런스 트랙의 link(youtube 등)나 파일, 주 장르를 입력하거나 태그에서 클릭 → AI가 분석 → synth·plugin 프리셋 추천 설정, 믹싱 및 마스터링 설정
선행 Phase	Phase 2 (플러그인·믹서 구조), Phase 4 (분리 — 악기별 분석에 활용, 선택)
관련 페이지	`studioNx.html` (우측 AI 분석 패널, 하단 패널 [Nyxi] 탭)
전체 개요	00_PRD_Overview.md

1. 목표 / 배경

- "이 곡 같은 느낌을 내고 싶은데 어떻게 설정해야 할지 모르겠다"는 문제를 해결합니다.
- 사용자가 **레퍼런스(링크 또는 파일)** 와 **장르(직접 입력 또는 태그 클릭)** 를 주면, AI가 음향 특징을 분석하고 Studio Nx의 플러그인·믹서에 **바로 적용 가능한 설정값**으로 추천합니다.
- 추천은 "정답"이 아니라 **출발점**입니다. 적용 후 사용자가 자유롭게 수정할 수 있어야 합니다.

2. 범위

In scope

- 레퍼런스 등록: 링크(YouTube 등) / 오디오 파일 업로드 / 라이브러리 샘플 선택
- 장르 지정: 텍스트 직접 입력 또는 장르 태그 클릭(복수 선택)
- 분석: 템포, 키, 라우드니스(LUFS), 트루피크, 다이내믹 레인지, 대역별 스펙트럼 밸런스, 스테레오 폭, 악기 구성 추정
- 추천 4종: **Synth 프리셋**, **Plugin(이펙트) 프리셋**, **믹싱 설정**(트랙별 볼륨/팬/EQ/컴프), **마스터링 설정**(마스터 체인, 목표 LUFS)
- 추천 적용: 미리보기(A/B) → 적용 → Undo 가능
- 내 프로젝트 vs 레퍼런스 비교(라우드니스, 스펙트럼)
- **Nyxi 도우미 (AI)**: FL Studio의 Gopher 같은 대화형 도우미. 하단 패널에서 Studio Nx 사용법·작곡/믹싱 질문에 답함. 시퀀서 캐릭터 **Nyxi가 도우미의 아이콘·얼굴** (§4.4)
- **Nyxi 팁 (상황 인지)**: 사용자가 묻기 전에 Nyxi가 클리핑·저역 겹침 등 문제를 먼저 알려줌. 기본은 "중요한 것만", **프로젝트가 5개쯤 되면 확인창으로 계속 받을지 물어봄** (§4.5)
- **레퍼런스 기반 새 프로젝트**: 분석 결과로 BPM·키·트랙 구성·추천 체인을 갖춘 새 프로젝트 생성 (적용 전 확인창 필수)
- AI 사용량 표시·제한

Out of scope

- 레퍼런스 곡의 멜로디·코드 자동 채보 → 추후 검토
- 자동 작곡/생성 → 지원하지 않음
- 저작권 있는 원본 오디오의 영구 보관·재배포 → 하지 않음 (아래 §5 참고)

3. 사용자 스토리

ID	사용자 스토리
US-501	사용자로서, YouTube 링크를 붙여넣고 장르 태그 "Lo-fi"를 클릭한 뒤 분석을 누르고 싶다.
US-502	사용자로서, 분석 결과(BPM, 키, 음량, 톤 밸런스)를 한눈에 보고 싶다.
US-503	사용자로서, 추천된 신스 프리셋을 선택한 MIDI 트랙에 바로 적용하고 싶다.
US-504	사용자로서, 추천 마스터링 체인을 적용하기 전·후를 비교해 듣고 싶다.
US-505	사용자로서, 마음에 드는 추천 프리셋을 내 프리셋으로 저장하고 싶다.
US-506	사용자로서, 내 곡이 레퍼런스보다 얼마나 작거나 저음이 부족한지 보고 싶다.
US-507	사용자로서, 작업 중 "사이드체인은 어떻게 걸어?"처럼 궁금한 걸 하단 패널의 Nyxi에게 바로 묻고 싶다.
US-508	사용자로서, Nyxi가 내 프로젝트(BPM, 트랙, 디바이스 구성)를 알고 그에 맞게 답해 주길 원한다.

4. 기능 요구사항

4.1 레퍼런스 입력

ID	요구사항	우선순위
FR-501	입력 방식 3가지: ① 링크 붙여넣기 ② 파일 드롭/선택 ③ 내 라이브러리 샘플 선택	필수
FR-502	장르: 텍스트 입력(`genre_text`) 또는 시스템 장르 태그(`tags.category='GENRE'`) 클릭 선택, 둘 다 가능	필수
FR-503	링크 처리: 서버에서 링크 유효성·길이 확인 후 분석용 오디오를 임시로 가져와 분석. **분석 후 원본 오디오는 삭제**하고 특징값만 저장	필수
FR-504	링크로 오디오를 가져올 수 없거나 정책상 불가한 경우, 파일 업로드를 안내	필수
FR-505	레퍼런스는 프로젝트에 연결(`project_id`)하거나 개인 레퍼런스로 보관	권장

4.2 분석

ID	요구사항	우선순위
FR-511	분석은 백그라운드 작업(`ai_analyses.status`)으로 처리, 진행 상태 표시	필수
FR-512	**신호 처리 기반 특징 추출**: BPM, 키, 통합 LUFS, 트루피크, 다이내믹 레인지, 대역별 에너지(예: Sub/Low/Mid/High-mid/High), 스테레오 폭	필수
FR-513	**AI 모델 기반 해석**: 추출한 특징 + 장르 정보 → 음색 특성 설명(`summary`), 악기 구성 추정(`features`)	필수
FR-514	(선택) Phase 4 분리 기능을 이용해 악기별 stem 단위로 특징 추출 → 트랙별 믹싱 추천 정확도 향상	권장
FR-515	분석 결과 화면: 수치 카드(BPM, 키, LUFS 등), 스펙트럼 밸런스 차트, 요약 문장	필수

4.3 추천

ID	요구사항	우선순위
FR-521	추천값은 **Studio Nx 플러그인의 `param_schema` 범위 안의 값**으로만 생성 (적용 불가능한 값 금지). 서버에서 스키마 검증 후 저장	필수
FR-522	Synth 프리셋 추천(`SYNTH_PRESET`): 대상 가상악기 + 파라미터 값. 역할별(베이스, 리드, 패드 등) 여러 개	필수

ID	요구사항	우선순위
FR-523	Plugin 프리셋 추천(`PLUGIN_PRESET`): EQ·컴프·리버브 등 이펙트별 설정	필수
FR-524	믹싱 추천(`MIXING`): 트랙 역할별 상대 볼륨, 팬, 추천 이펙트 체인	필수
FR-525	마스터링 추천(`MASTERING`): 마스터 체인(EQ → Comp → Limiter 등), 목표 LUFS, 트루피크 한계	필수
FR-526	각 추천에 **이유 설명(`rationale`)** 표시. 예: "레퍼런스의 저역 에너지가 높아 베이스 트랙 80Hz 부근을 +2dB 제안"	필수
FR-527	적용: 대상 트랙 선택 → 미리듣기(A/B 토글) → [적용]. 적용은 Undo 이력에 포함, `is_applied`·`applied_at` 기록	필수
FR-528	추천 프리셋을 "내 프리셋으로 저장" → `plugin_presets`에 `source='AI'`	권장
FR-529	레퍼런스 비교: 현재 프로젝트를 빠르게 렌더링(Phase 3 렌더러 재사용)해 같은 특징을 추출, 레퍼런스와 차이 표시	권장
FR-530	**레퍼런스 기반 새 프로젝트**: 분석 결과의 BPM·키·스케일, 추정 악기 구성으로 트랙을 만들고 추천 체인·마스터링을 적용한 새 프로젝트 생성. **적용 전 확인창**에 생성될 내용 요약 표시(PRD 02 FR-2143)	권장

4.3.1 AI 사용량

ID	요구사항	우선순위
FR-551	사용자별 일일 한도: 레퍼런스 분석 N회, Nyxi 질문 M회 (설정값). 한도 초과 시 안내 + 초기화 시각 표시	필수
FR-552	마이페이지 대시보드에 오늘/이번 달 AI 사용량 표시 (PRD 01 §4.4)	필수
FR-553	사용량 집계는 `ai_analyses`, `assistant_messages`의 생성 기록으로 계산 (별도 테이블 없음)	필수

4.4 Nyxi 도우미 — AI 도우미 (FL Studio Gopher식)

> 시퀀서의 **하단 패널(토글형, PRD 02 §4.8)** 의 [Nyxi] 탭에서 동작하는 대화형 AI 도우미입니다. FL Studio의 서비스명 "Gopher"는 쓰지 않고, 시퀀서 캐릭터 **Nyxi(닉시)**(PRD 02 §4.9)를 도우미의 아이콘·얼굴로 씁니다. 캐릭터 외형은 별도 AI 플랫폼에서 제작합니다.

ID	요구사항	우선순위
FR-531	상단바 **Nyxi 아이콘** 버튼(또는 Ctrl+K) → 하단 패널이 올라오며 Nyxi 탭 표시. 입력창에 질문 → 답변 스트리밍 표시	필수
FR-532	답변 범위: ① Studio Nx 기능 사용법(도움말 문서 기반) ② 작곡·사운드 디자인·믹싱 일반 지식 ③ 현재 프로젝트에 대한 조언	필수
FR-533	**프로젝트 문맥 제공**: 질문 시 현재 프로젝트 요약(BPM, 키, 트랙 목록, 선택 트랙의 디바이스 체인/패처 구성, 최근 분석 결과)을 함께 보내 맞춤 답변. 오디오 원본은 보내지 않음	필수
FR-534	도움말 근거: Studio Nx 사용법 질문은 `docs/help/` 도움말 문서를 검색해 근거로 사용(RAG), 답변에 해당 도움말 링크 표시	권장
FR-535	**액션 버튼**: 답변이 화면 조작으로 이어질 수 있으면 버튼 제공 (예: "Bass 트랙에 Compressor 추가", "패처 뷰 열기", "레퍼런스 분석 시작"). 실행 전 확인, 실행 후 Undo 가능	권장
FR-536	대화 기록 저장(`assistant_messages`), 프로젝트별로 이어서 보기, [대화 지우기]	필수
FR-537	Nyxi 대화는 **개인 전용** (협업 멤버에게 보이지 않음). 협업자에게 공유하고 싶으면 [톡방에 공유] 버튼으로 답변을 채팅에 붙여넣기 (Phase 6 이후)	권장
FR-538	사용량 제한(사용자별 일일 질문 수) 및 실패 시 "잠시 후 다시 시도" 안내	필수

ID	요구사항	우선순위
FR-539	**캐릭터 연동**: 답변 생성 중엔 Nyxi가 "생각 중" 동작, 패널이 닫혀 있으면 답변 완료 시 Nyxi 아이콘 배지 + 캐릭터가 답변 요약을 말풍선으로 말함. 말풍선 클릭 → [Nyxi] 탭 열림	필수
FR-540	**빠른 질문**: 캐릭터 클릭 → 캐릭터 옆 짧은 입력창. 답변은 말풍선(요약) + 전체 내용은 [Nyxi] 탭에 기록	권장
FR-541	튜토리얼(PRD 02 §4.15) 진행 중 질문하면 현재 튜토리얼 단계를 문맥으로 함께 전달	권장

4.5 Nyxi 팁 — 상황 인지 (먼저 알려주기)

> 질문형 도우미(§4.4)와 달리, Nyxi가 작업 상태를 보고 **먼저** 말풍선으로 알려줍니다. 잔소리가 되지 않도록 기본값은 조용하게 두고, **사용자가 익숙해지면(프로젝트 5개 무렵) 계속 받을지 확인창으로 묻습니다.** 상세 배경: Ai_feedback_moreExplain.md §1

팁 종류

구분	팁 (감지 조건)	판단 방식	버튼 예시
중요	마스터 클리핑 (재생 중 마스터 피크 0 dBFS 초과 3회 이상)	규칙 (미터 값)	[리미터 추가] [볼륨 -3dB]
중요	트랙 클리핑 (특정 트랙 미터가 계속 빨간색)	규칙	[게인 -6dB]
중요	오디오 끊김 (언더런 연속 발생)	규칙 (Sentry 지표와 같은 데이터)	[무거운 트랙 보기]
중요	저장 안 함 (30분 이상, 자동 저장 꺼짐)	규칙	[지금 저장]
중요	저장 용량 90% 초과	규칙	[용량 정리]
작곡 조언	킥·베이스 저역 겹침 (50~100Hz 동시 강세)	클라이언트 주파수 분석	[사이드체인 설정 열기]
작곡 조언	음량 목표 벗어남 (출력 직전 LUFS > -8)	LUFS 측정	[마스터링 추천 보기]
작곡 조언	스케일 밖 음 다수 (키 설정 시)	프로젝트 데이터	[스케일 밖 노트 선택]
작곡 조언	곡 앞 빈 구간 4마디 이상 (출력 시)	프로젝트 데이터	[출력 범위 조정]

ID	요구사항	우선순위
FR-561	팁 레벨: **끄기 / 중요한 것만(기본) / 모두**(작곡 조언 포함). 환경 설정(PRD 02 FR-2166a)에서 **언제든 수동 변경**	필수
FR-562	**익숙해짐 확인창**: 사용자의 프로젝트가 **5개**(설정값)에 도달한 뒤 시퀀서를 처음 열 때 1회 표시 — "Studio Nx에 꽤 익숙해지셨네요! Nyxi 팁을 계속 받을까요?" [계속 받기] [중요한 것만] [끄기]. 팁 레벨이 이미 '끄기'면 표시하지 않음. 표시 여부는 `tips_state`에 기록해 다시 묻지 않음	필수
FR-563	표시 타이밍: **재생을 멈췄을 때만** 표시(재생·드래그 중엔 대기열에 보관), **녹음 중에는 절대 표시 안 함**	필수
FR-564	빈도 제한: 같은 팁은 프로젝트당 1회(또는 24시간 1회), 전체 팁은 10분에 최대 1개	필수
FR-565	말풍선 메뉴: [이 팁 다시 보지 않기] [팁 끄기] [왜 그런가요?] — "왜 그런가요?"는 Nyxi 도우미(§4.4)에게 설명을 요청(AI 사용량에 포함)	필수
FR-566	조용한 모드: 캐릭터 창이 숨겨져 있으면 말풍선 대신 상단바 Nyxi 아이콘에 작은 점(●) → 클릭 시 팁 표시	필수
FR-567	판단은 **브라우저에서 규칙·오디오 분석으로 처리**(LLM 호출 없음). 문장은 팁별 템플릿 사용	필수
FR-568	협업 중에는 팁을 **현재 사용자의 행동과 관련된 경우에만** 표시(예: 내가 담당·잠금한 트랙, 내가 누른 출력). 다른 멤버에게는 보내지 않음	권장
FR-569	팁 규칙은 목록(레지스트리)으로 관리해 추가·수정이 쉽도록 하고, 각 규칙의 오탐 여부를 회귀 테스트 프로젝트로 검증	권장

5. 비기능 요구사항 / 정책

구분	요구사항
저작권·약관	외부 링크의 오디오는 **분석 목적으로만 임시 사용 후 삭제**, 특징값(수치)만 저장. 각 플랫폼 이용약관을 확인해 허용되지 않는 방식이면 파일 업로드 방식만 제공
AI 모델	특징 추출은 오디오 분석 라이브러리(예: librosa, pyloudnorm, essentia 등 후보), 해석·추천은 LLM 또는 규칙 기반 + LLM 조합. 사용 모델/버전은 `model_version`에 기록
신뢰성	LLM 출력은 JSON 스키마로 강제하고, 범위 밖 값은 클램프 또는 재요청. 실패 시 규칙 기반 기본 추천으로 대체
비용	같은 레퍼런스 재분석 시 기존 결과 재사용(캐시)
시간	3분 곡 기준 분석+추천 1분 이내 목표 (분리 사용 시 제외)
개인정보	외부 AI API로는 오디오 원본이 아닌 **추출된 특징값만** 전송하는 것을 기본으로 함

6. 화면 구성

- `studioNx.html` 우측 **AI 패널** (토글)
  1. 레퍼런스 입력: 링크 입력창 / 파일 드롭존 / [라이브러리에서 선택]
  2. 장르: 텍스트 입력 + 장르 태그 칩(클릭 선택)
  3. [분석하기] → 진행 표시
  4. 분석 결과: 수치 카드, 스펙트럼 차트, 요약
  5. 추천 탭: Synth | Plugin | Mixing | Mastering — 카드별 [미리듣기] [적용] [프리셋 저장] + 이유 설명
  6. 비교 탭: 내 곡 vs 레퍼런스 (LUFS, 대역 밸런스)
- `studioNx.html` **하단 패널 [Nyxi] 탭** (Nyxi 아이콘 버튼 클릭 시에만 표시)
  - 대화 목록(질문/답변, 답변 안 액션 버튼, 도움말 링크), 입력창, [대화 지우기]
  - 입력창 위 "프로젝트 문맥 포함" 토글 (기본 켜짐)

7. API 초안 (FastAPI)

Method	Path	설명
POST	`/references`	레퍼런스 등록 (링크 / 파일 multipart / sample_id, 장르 텍스트, tag_ids)
GET	`/references?project_id=`	레퍼런스 목록
DELETE	`/references/{id}`	삭제
POST	`/references/{id}/analyses`	분석 시작
GET	`/analyses/{analysis_id}`	분석 상태·결과
GET	`/analyses/{analysis_id}/recommendations?type=`	추천 목록
POST	`/recommendations/{id}/apply`	적용 기록 (body: project_id, 대상 track_id 등)
POST	`/recommendations/{id}/save-preset`	내 프리셋으로 저장
GET	`/analyses/{analysis_id}/project-preview`	레퍼런스 기반 새 프로젝트 생성 내용 요약 (**확인창 내용**)
POST	`/analyses/{analysis_id}/create-project`	새 프로젝트 생성 (`confirmed=true` 필수)
GET	`/me/ai-usage`	오늘/이번 달 AI 사용량·남은 한도
POST	`/projects/{id}/compare`	내 프로젝트 렌더링 결과 업로드 → 레퍼런스와 비교

Method	Path	설명
GET	`/tags?category=GENRE`	장르 태그 목록 (Phase 2 API 재사용)
POST	`/assistant/messages`	Nyxi 질문 (body: project_id, content, context) → 답변 스트리밍 (SSE)
GET	`/assistant/messages?project_id=&before_id=`	Nyxi 대화 기록
DELETE	`/assistant/messages?project_id=`	대화 지우기

8. 사용 테이블

> 컬럼 상세는 DB.md §3.5 참고

- `reference_tracks`, `reference_tags`
- `ai_analyses`, `ai_recommendations`
- `tags` (장르 태그), `plugins` (`param_schema` 검증), `plugin_presets` (`source='AI'` 저장)
- `tracks`, `track_plugins`, `projects.master_settings` (추천 적용 대상)
- `assistant_messages` (Nyxi 대화 기록)
- `user_settings.tips_level`, `tips_state` (Nyxi 팁 레벨·다시 보지 않기·익숙해짐 확인창 표시 여부)
- `projects` (사용자의 프로젝트 수 — 익숙해짐 확인창 조건)
- `projects`, `tracks`, `track_plugins` (레퍼런스 기반 새 프로젝트 생성)

9. AI 역할 구분

영역	이 Phase에서 AI가 하는 일
HTML / CSS	AI 패널, 태그 칩, 결과 카드, 추천 카드, Nyxi 대화 UI **작성**
JavaScript	패널 상태 관리, A/B 미리듣기, 적용 로직 **템플릿**만
Backend	레퍼런스/분석/추천 라우터, 특징 추출·추천 생성 서비스 **템플릿**만. 분석 파이프라인·프롬프트·Nyxi(대화/RAG) 서비스는 요청 시 AI 구현
C++	해당 없음 (분석 DSP를 C++로 만들 경우 인터페이스만 문서화)
DB	Phase 5 테이블 생성, 장르 태그 시드 데이터, DB.md 갱신
문서	분석 지표 정의, 추천 생성 로직, 프롬프트, 사용 모델을 `docs/ai-notes/`에 상세히 기록 (면접·발표 대비 핵심 파트)

10. 완료 기준 (Acceptance Criteria)

- [ ] 파일 업로드 + 장르 태그 선택으로 분석하면 BPM·키·LUFS·스펙트럼 밸런스가 표시된다.
- [ ] 장르를 텍스트로만 입력해도 분석·추천이 동작한다.
- [ ] 4종(Synth / Plugin / Mixing / Mastering) 추천이 각각 1개 이상 생성되고 이유 설명이 붙는다.
- [ ] 모든 추천값이 해당 플러그인의 `param_schema` 범위 안에 있다.
- [ ] 추천을 적용하면 소리에 즉시 반영되고 Undo로 되돌릴 수 있다.
- [ ] 마스터링 추천 적용 후 출력(Phase 3)한 결과의 LUFS가 목표값 ±1 LU 안에 든다.
- [ ] 링크 분석 후 서버에 원본 오디오 파일이 남아 있지 않다.

- [ ] [이 레퍼런스로 새 프로젝트]를 누르면 확인창에 생성 내용이 요약되고, [적용]해야만 프로젝트가 만들어진다.
- [ ] 일일 AI 사용 한도를 넘으면 분석·질문이 막히고 남은 시간이 안내된다.
- [ ] Nyxi 답변 중엔 캐릭터가 생각 중 동작을 하고, 패널이 닫혀 있으면 답변 요약을 말풍선으로 보여준다.
- [ ] 마스터가 클리핑되면 재생을 멈춘 뒤 Nyxi가 팁을 보여주고, [리미터 추가]가 동작한다. 재생·녹음 중에는 팁이 뜨지 않는다.
- [ ] 다섯 번째 프로젝트를 만든 뒤 시퀀서를 열면 팁 유지 여부 확인창이 한 번만 뜨고, 선택이 설정에 저장된다.
- [ ] 환경 설정에서 팁을 끄면 어떤 팁도 표시되지 않는다.
- [ ] 하단 패널의 Nyxi에게 Studio Nx 사용법을 물으면 관련 도움말 근거와 함께 답한다.
- [ ] "이 프로젝트 BPM이 몇이야?"처럼 프로젝트 관련 질문에 현재 프로젝트 정보로 정확히 답한다.
- [ ] Nyxi 대화는 새로고침 후에도 남아 있고, 협업 멤버에게는 보이지 않는다.

Studio Nx — PRD 06. Phase 6: 실시간 협업 · 단체톡

항목	내용
기획서 기능	6) 다른 회원을 초대하여 Google Docs와 비슷한 방식으로 협업하는 기능 (with 단체 톡)
선행 Phase	Phase 1 (회원, 프로젝트), Phase 2 (프로젝트 데이터 구조)
관련 페이지	`studioNx.html` (참여자 표시, 하단 패널 [톡방] 탭), `myProject.html` (공유된 프로젝트), `myPage.html` (받은 초대)
전체 개요	00_PRD_Overview.md

---

1. 목표 / 배경

- 파일을 주고받지 않고, **하나의 프로젝트를 여러 회원이 동시에 열어 편집**할 수 있게 합니다. 한 사람이 클립을 옮기면 다른 사람 화면에도 곧바로 반영됩니다.
- 프로젝트마다 **단체 채팅방**을 두어 작업하면서 바로 의견을 나눌 수 있게 합니다.

2. 범위

In scope

- 초대: 이메일(또는 닉네임)로 초대, 초대 링크, 수락/거절, 권한 지정(EDITOR / VIEWER)
- 멤버 관리: 권한 변경, 내보내기, 나가기
- 실시간 동시 편집: 트랙·클립·노트·플러그인 파라미터·프로젝트 설정 변경을 실시간 동기화
- 접속자 표시(Presence): 접속 중인 멤버 아바타, 각자의 재생 헤드/선택 영역 색상 표시
- 프로젝트별 단체 채팅: 실시간 메시지, 이전 메시지 불러오기, 시스템 메시지(입장, 초대 수락 등), 안 읽은 메시지 수
- 변경 이력 저장(누가 언제 무엇을 바꿨는지)
- **타임라인 코멘트**: 마디·트랙·클립 위치에 댓글 핀, 스레드 답글, 해결 처리
- **버전 스냅샷 · 복원**: 수동/자동 스냅샷, 미리듣기, 복원, 새 프로젝트로 복제
- **트랙 잠금 · 담당자 표시**

Out of scope

- 음성/영상 통화 → 지원하지 않음
- 재생 동기화(모두가 같이 듣기) → 추후 검토
- 채팅 파일 첨부, 이모지 반응 → 추후 검토

3. 사용자 스토리

ID	사용자 스토리
US-601	프로젝트 소유자로서, 친구 이메일로 초대해 편집 권한을 주고 싶다.
US-602	초대받은 회원으로서, 마이페이지에서 초대를 보고 수락하면 프로젝트가 내 목록에 나타나길 원한다.
US-603	편집자로서, 다른 사람이 옮긴 클립이 내 화면에도 바로 보이길 원한다.

ID	사용자 스토리
US-604	편집자로서, 지금 누가 접속해 있고 어디를 보고 있는지 알고 싶다.
US-605	멤버로서, 작업하면서 채팅으로 "베이스 좀 줄여줘" 같은 이야기를 나누고 싶다.
US-606	뷰어로서, 프로젝트를 듣고 채팅은 할 수 있지만 편집은 할 수 없어야 한다.
US-607	소유자로서, 특정 멤버의 권한을 바꾸거나 내보내고 싶다.
US-608	멤버로서, "32마디 베이스가 너무 커요"처럼 정확한 위치에 코멘트를 남기고 답글로 논의한 뒤 해결 처리하고 싶다.
US-609	멤버로서, 어제 버전이 더 좋았다면 그 시점 소리를 미리 들어보고 되돌리거나 복제해 비교하고 싶다.
US-610	멤버로서, 내가 작업 중인 트랙을 다른 사람이 건드리지 않도록 잠그고, 트랙별 담당자를 표시하고 싶다.

4. 기능 요구사항

4.1 초대 · 멤버

ID	요구사항	우선순위
FR-601	권한: **OWNER**(모든 권한, 1명) / **EDITOR**(편집·채팅) / **VIEWER**(재생·채팅만)	필수
FR-602	초대: OWNER(설정에 따라 EDITOR도)가 이메일·권한 입력 → `invitations` 생성(토큰, 7일 만료)	필수
FR-603	초대받은 이메일이 회원이면 `invitee_id` 연결 + 마이페이지 "받은 초대"에 표시. 비회원이면 초대 링크로 가입 후 수락	필수
FR-604	초대 링크 공유: 토큰 URL 복사 기능	권장
FR-605	수락 시 `project_members` 추가 + 채팅에 시스템 메시지 "OO님이 참여했습니다". 거절/만료/취소 상태 관리	필수
FR-606	멤버 관리: 권한 변경, 내보내기(OWNER), 나가기(본인, OWNER 제외)	필수
FR-607	`myProject.html`에 "공유받은 프로젝트" 탭 추가	필수

4.2 실시간 동시 편집

ID	요구사항	우선순위
FR-611	프로젝트를 열면 **WebSocket**(`/ws/projects/{id}`)으로 접속. 인증 토큰 + 멤버 권한 확인	필수
FR-612	편집은 **변경 연산(operation)** 단위로 전송. 예: `clip.move {clip_id, start_tick}`, `note.add {...}`, `param.set {track_plugin_id, index, value}`, 패처 연결 `link.add` / `link.remove`, `track.routing_mode`	필수
FR-613	**서버가 순서를 정하는 방식(중앙 서버 순서화)**: 서버가 연산을 받아 검증 → DB 반영 → `projects.revision` +1 → `project_change_logs` 기록 → 모든 접속자에게 브로드캐스트	필수
FR-614	충돌 처리: 클라이언트는 자기 연산을 먼저 화면에 반영(낙관적 적용)하고, 서버 확정 순서와 다르면 서버 기준으로 재적용. 같은 속성 동시 수정은 **마지막에 도착한 값이 우선(LWW)**	필수
FR-615	삭제된 대상에 대한 연산(예: 다른 사람이 지운 클립 이동)은 서버에서 거부하고 해당 클라이언트에 알림	필수
FR-616	재접속: 마지막으로 받은 `revision` 이후 변경분을 `project_change_logs`에서 받아 따라잡기. 차이가 너무 크면 전체 상태 다시 로드	필수
FR-617	연속 변경(노브 돌리기, 드래그 중)은 일정 간격(예: 50ms)으로 묶어 전송, 드래그 종료 시 최종값 확정	필수
FR-618	Undo/Redo는 **자기 연산만** 되돌림 (다른 사람 작업을 되돌리지 않음)	권장
FR-619	VIEWER의 편집 연산은 서버에서 거부, UI에서도 편집 도구 비활성화	필수

ID	요구사항	우선순위
FR-620	샘플 공유: 멤버가 타임라인에 올린 샘플은 다른 멤버도 재생 가능해야 함(프로젝트에 사용된 샘플은 멤버에게 읽기 권한)	필수

> 참고: 더 정교한 방식으로 CRDT(Yjs 등)나 OT를 검토할 수 있으나, 시퀀서 데이터는 대부분 "객체 속성 변경"이라 **서버 순서화 + LWW**로 시작하고 필요 시 확장합니다.

4.3 접속자 표시 (Presence)

ID	요구사항	우선순위
FR-631	접속 중인 멤버 아바타 목록 (멤버별 고유 색상)	필수
FR-632	다른 멤버의 선택 영역(선택한 클립/트랙)을 해당 색상 테두리로 표시	권장
FR-633	다른 멤버의 재생 헤드/마우스 위치(커서) 표시	권장
FR-634	Presence 정보는 DB에 저장하지 않고 서버 메모리에서만 관리	필수

4.4 단체 채팅

ID	요구사항	우선순위
FR-641	프로젝트 = 채팅방. 같은 WebSocket 연결로 채팅 메시지 송수신. 화면은 **하단 패널 [톡방] 탭**(버튼 클릭 시 표시)	필수
FR-642	메시지 저장(`chat_messages`), 패널 열 때 최근 50개 로드, 위로 스크롤 시 이전 메시지 추가 로드	필수
FR-643	시스템 메시지: 입장/퇴장, 초대 수락, 권한 변경	필수
FR-644	안 읽은 메시지 수: `project_members.last_read_message_id` 기준, 💬 톡방 버튼 배지와 `myProject.html` 카드에 표시	권장
FR-645	본인 메시지 삭제(소프트 삭제, "삭제된 메시지입니다" 표시)	권장
FR-646	메시지 길이 제한(예: 2000자), 짧은 시간 연속 전송 제한	필수
FR-647	메시지 내용은 화면에 표시할 때 HTML 이스케이프 (XSS 방지)	필수

4.5 타임라인 코멘트

ID	요구사항	우선순위
FR-651	타임라인 눈금자 또는 클립 우클릭 → [코멘트 추가]. 위치 = `tick`(+선택적으로 `track_id`/`clip_id`)	필수
FR-652	타임라인 위에 코멘트 핀 표시(작성자 색), 클릭 시 스레드(답글) 팝오버	필수
FR-653	답글, 수정(본인), 삭제(본인·OWNER, 소프트 삭제), **해결(Resolve)**/다시 열기. 해결된 코멘트는 숨김 필터	필수
FR-654	우측 [코멘트] 사이드 패널: 전체 목록(미해결 우선), 클릭 시 해당 위치로 이동·재생	필수
FR-655	새 코멘트·답글 시 톡방에 시스템 메시지 "OO님이 32마디에 코멘트" + Nyxi 리액션, `@닉네임` 멘션 시 해당 멤버에게 배지	권장
FR-656	VIEWER도 코멘트 작성 가능(편집 권한과 분리)	필수
FR-657	코멘트가 붙은 클립이 이동·삭제되면: 이동 → 핀도 따라감(클립 기준 상대 위치), 삭제 → 클립 연결만 해제되고 tick 위치 유지	권장

4.6 버전 스냅샷 · 복원

> Undo(세션 내 명령 단위)와 변경 이력(연산 로그)으로는 "며칠 전 상태로 통째로 돌아가기"가 어렵습니다. 스냅샷은 **프로젝트 전체 상태의 저장본**입니다. 상세 설명: Ai_feedback_moreExplain.md §2

ID	요구사항	우선순위
FR-661	**수동 스냅샷**: [스냅샷 저장] → 이름 입력(예: "믹스 v1"). OWNER/EDITOR 가능	필수
FR-662	**자동 스냅샷**: ① 하루 1회(그날 변경이 있을 때) ② 위험 작업 직전 — 스냅샷 복원 직전(`BEFORE_RESTORE`), 템플릿 교체 적용 직전(`BEFORE_TEMPLATE`), AI 추천 일괄 적용 직전(`BEFORE_AI_APPLY`)	필수
FR-663	저장 내용: 프로젝트 전체 상태(트랙·디바이스·연결선·패턴·노트·오토메이션·설정)를 압축 JSON 파일로 저장(`data_path`). **샘플 오디오 파일은 복사하지 않고 참조**만 함	필수
FR-664	스냅샷 목록: 이름, 종류, 만든 사람, 시각, revision. 고정(📌) 가능	필수
FR-665	**미리듣기**: 스냅샷 시점 상태를 오프라인 렌더링해 짧은 미리듣기 생성(요청 시 생성 후 캐시, `preview_path`)	권장
FR-666	**복원**: 확인창 → 현재 상태를 `BEFORE_RESTORE`로 자동 저장 → 스냅샷 상태로 교체. 협업 중이면 서버가 `project.restore` 연산으로 처리해 **모든 접속자에게 동시에 반영**, revision 증가. OWNER/EDITOR만	필수
FR-667	**새 프로젝트로 복제**: 스냅샷 시점 상태로 새 프로젝트 생성(원본은 그대로) — 두 버전 비교용	필수
FR-668	**보관 규칙**: 수동·고정 스냅샷은 삭제 전까지 보관 / 자동 스냅샷은 최근 7일치 일일 + 최근 4주치 주간 1개씩 + 위험 작업 직전 스냅샷 30일 보관 후 정리. 스냅샷 파일도 저장 용량에 포함	필수
FR-669	스냅샷이 참조하는 샘플은 삭제(소프트) 후에도 파일을 지우지 않음 — 참조하는 스냅샷이 모두 정리된 뒤에 실제 파일 삭제	필수

4.7 트랙 잠금 · 담당자 표시

ID	요구사항	우선순위
FR-671	트랙 헤더 🔒 버튼: **잠금** — 잠근 사람만 해당 트랙(클립·노트·디바이스·오토메이션) 편집 가능. 다른 멤버 화면에는 잠금 아이콘 + 잠근 사람 아바타	필수
FR-672	잠금 해제: 본인, 또는 OWNER가 강제 해제(확인창 + 톡방 시스템 메시지)	필수
FR-673	자동 해제: 잠근 사람이 접속을 끊고 일정 시간(예: 30분) 지나면 자동 해제	권장
FR-674	**담당자 지정**: 트랙별 담당 멤버(`assignee_id`) 지정, 트랙 헤더에 담당자 아바타. 잠금과 별개(담당자는 표시만)	필수
FR-675	서버에서 잠긴 트랙에 대한 다른 사람의 연산은 거부(`op_reject`, reason=`TRACK_LOCKED`)	필수

5. 비기능 요구사항

구분	요구사항
지연	같은 지역 기준 한 사람의 편집이 다른 사람 화면에 반영되기까지 300ms 이내 목표
동시 접속	프로젝트당 동시 편집자 최대 8명 (설정값)
일관성	모든 접속자의 최종 상태가 서버 DB 상태와 동일하게 수렴할 것
확장성	서버 1대 기준으로 시작. 다중 서버 확장 시 메시지 브로커(예: Redis Pub/Sub) 도입 검토
보안	WebSocket 연결·모든 연산마다 권한 검사. 초대 토큰은 추측 불가능한 랜덤 64자, 1회 사용
이력 보관	`project_change_logs`는 용량이 커지므로 일정 기간/개수 이후 정리 정책 마련

6. 화면 구성

- `studioNx.html`
  - 상단바: 접속자 아바타 목록 + **[공유]** 버튼
  - 공유 모달: 이메일 입력 + 권한 선택 + [초대], 초대 링크 복사, 멤버 목록(권한 변경/내보내기), 대기 중 초대(취소)
  - **하단 패널 [톡방] 탭** (PRD 02 §4.8): 평소엔 숨김, 톡방 버튼(또는 Ctrl+J)을 누르면 올라옴. 메시지 목록, 입력창. 닫혀 있을 때는 버튼에 안 읽은 수 배지
  - 타임라인: 다른 멤버의 선택 영역/커서를 색상으로 표시
- `myPage.html`: "받은 초대" 위젯 (수락/거절)
- `myProject.html`: "내 프로젝트 | 공유받은 프로젝트" 탭, 카드에 멤버 수·안 읽은 메시지 수·미해결 코멘트 수
- `studioNx.html` 우측 사이드 패널(토글): **[코멘트]** · **[스냅샷]** 탭
- 타임라인: 코멘트 핀, 트랙 헤더의 🔒 잠금·담당자 아바타

7. API 초안 (FastAPI)

REST

Method	Path	설명
GET	`/projects/{id}/members`	멤버 목록
PATCH	`/projects/{id}/members/{user_id}`	권한 변경 (OWNER)
DELETE	`/projects/{id}/members/{user_id}`	내보내기 / 나가기
POST	`/projects/{id}/invitations`	초대 생성
GET	`/projects/{id}/invitations`	보낸 초대 목록
DELETE	`/invitations/{id}`	초대 취소
GET	`/me/invitations`	받은 초대 목록
GET	`/invitations/token/{token}`	초대 링크 정보 확인
POST	`/invitations/{id}/accept` · `/decline`	수락 / 거절
GET	`/projects/shared`	공유받은 프로젝트 목록
GET	`/projects/{id}/changes?since_revision=`	재접속 따라잡기용 변경분
GET	`/projects/{id}/messages?before_id=&size=50`	이전 채팅 메시지
PUT	`/projects/{id}/messages/read`	읽음 위치 갱신
DELETE	`/messages/{id}`	내 메시지 삭제
GET	`/projects/{id}/comments?resolved=`	코멘트 목록
POST	`/projects/{id}/comments`	코멘트·답글 작성 (tick, track_id, clip_id, parent_comment_id)
PATCH	`/comments/{id}`	수정 / 해결·다시 열기
DELETE	`/comments/{id}`	삭제 (소프트)
GET	`/projects/{id}/snapshots`	스냅샷 목록
POST	`/projects/{id}/snapshots`	수동 스냅샷 저장
PATCH	`/snapshots/{id}`	이름 변경·고정
POST	`/snapshots/{id}/preview`	미리듣기 생성 요청

Method	Path	설명
POST	`/snapshots/{id}/restore`	복원 (`confirmed=true` 필수)
POST	`/snapshots/{id}/duplicate`	새 프로젝트로 복제
DELETE	`/snapshots/{id}`	삭제
PATCH	`/tracks/{id}/assignee`	담당자 지정

WebSocket `/ws/projects/{id}?token=...`

방향	type	내용
C → S	`op`	`{client_op_id, base_revision, op_type, target_type, target_id, payload}`
S → C	`op_ack`	`{client_op_id, revision}` — 내 연산 확정
S → C	`op_reject`	`{client_op_id, reason}` — 거부(권한/대상 없음 등)
S → C	`op_broadcast`	`{revision, user_id, op_type, ...}` — 다른 사람 연산
C ↔ S	`presence`	`{user_id, selection, cursor_tick}` / 입장·퇴장
C → S	`op` (잠금)	`track.lock` / `track.unlock` — 성공 시 모두에게 broadcast
S → C	`comment_event`	코멘트 생성·수정·해결 알림
S → C	`project_restored`	스냅샷 복원됨 → 전체 상태 다시 로드
C → S	`chat_send`	`{content}`
S → C	`chat_message`	`{message_id, user_id, message_type, content, created_at}`

8. 사용 테이블

> 컬럼 상세는 DB.md §3.6 참고

- `project_members` (권한, 안 읽은 메시지 기준)
- `invitations`
- `project_change_logs`
- `chat_messages`
- `projects.revision` (문서 버전 번호)
- `timeline_comments` (코멘트), `project_snapshots` (스냅샷)
- `tracks.assignee_id` / `locked_by` / `locked_at` (담당자·잠금)
- Phase 2 편집 대상 테이블: `tracks`, `track_plugins`, `track_plugin_links`, `patterns`, `clips`, `midi_notes`, `automation_lanes`, `automation_points`, `launcher_scenes`

9. AI 역할 구분

영역	이 Phase에서 AI가 하는 일
HTML / CSS	공유 모달, 멤버 목록, 하단 패널 톡방 탭, 접속자 아바타, 받은 초대 위젯 **작성**
JavaScript	WebSocket 클라이언트, 연산 송수신·재적용, Presence 렌더링, 채팅 UI 로직 **템플릿**만
Backend	초대/멤버/채팅 라우터, WebSocket 연결 관리자, 연산 검증·적용 서비스 **템플릿**만
C++	해당 없음

영역	이 Phase에서 AI가 하는 일
DB	`invitations`, `project_change_logs`, `chat_messages` 생성, DB.md 갱신
문서	동기화 방식(서버 순서화 + LWW), 메시지 규격, 충돌 시나리오를 `docs/ai-notes/`에 정리

10. 완료 기준 (Acceptance Criteria)

- [ ] 이메일로 초대 → 상대가 마이페이지에서 수락 → 공유받은 프로젝트 목록에 나타난다.
- [ ] 두 브라우저에서 같은 프로젝트를 열고 한쪽에서 클립을 옮기면 다른 쪽에 즉시 반영된다.
- [ ] 두 명이 같은 노브를 동시에 돌려도 최종적으로 양쪽 값이 같아진다.
- [ ] 한쪽이 지운 클립을 다른 쪽이 옮기려 하면 거부되고 화면이 서버 상태로 맞춰진다.
- [ ] 네트워크를 끊었다 다시 연결하면 그 사이 변경분을 따라잡는다.
- [ ] VIEWER는 재생·채팅만 가능하고, 편집 연산을 강제로 보내도 서버가 거부한다.
- [ ] 채팅 메시지가 실시간으로 오가고, 새로고침 후에도 이전 메시지가 보인다.
- [ ] 접속 중인 멤버 아바타가 표시되고, 나가면 사라진다.
- [ ] 32마디에 코멘트를 달면 다른 멤버 화면에 핀이 나타나고, 답글·해결 처리가 동작한다. VIEWER도 코멘트를 달 수 있다.
- [ ] 수동 스냅샷을 만들고 편집을 계속한 뒤 복원하면, 모든 접속자 화면이 스냅샷 상태로 바뀌고 복원 직전 상태가 자동 스냅샷으로 남는다.
- [ ] 스냅샷을 새 프로젝트로 복제하면 원본은 그대로다.
- [ ] 잠긴 트랙은 잠근 사람 외에는 편집할 수 없고(서버도 거부), OWNER는 강제 해제할 수 있다.

Studio Nx — DB 설계 문서

> 이 문서는 Studio Nx의 데이터베이스 구조(테이블, 컬럼, 관계, 인덱스)를 정리한 문서입니다. 각 PRD 문서(`docs/PRD/*.md`)의 "사용 테이블" 항목은 이 문서의 절을 가리킵니다.

---

1. 개요

1.1 DBMS 선택

항목	내용
DBMS	**MySQL 8.0**
선택 이유	기획서 규칙: "Oracle DB를 새로 설치해야 하면 MySQL로 대체". 개발 PC에 Oracle은 없고 MySQL 8.0(서비스 `MySQL80-scit`)이 이미 실행 중
Database 이름	`studionx` (기존 DB를 쓰지 않고 새로 만듦)
문자셋	`utf8mb4` / `utf8mb4_0900_ai_ci` (한글·이모지 지원)
스토리지 엔진	InnoDB (트랜잭션, FK 지원)

1.2 Database 생성

```
CREATE DATABASE studionx
  DEFAULT CHARACTER SET utf8mb4
  DEFAULT COLLATE utf8mb4_0900_ai_ci;

-- 애플리케이션 전용 계정 (비밀번호는 실제 값으로 교체)
CREATE USER 'studionx_app'@'localhost' IDENTIFIED BY '<비밀번호>';
GRANT SELECT, INSERT, UPDATE, DELETE ON studionx.* TO 'studionx_app'@'localhost';
```

1.3 설계 규칙

규칙	내용
테이블명	소문자 `snake_case`, 복수형 (`users`, `projects`)
PK	`<단수형>_id` `BIGINT UNSIGNED AUTO_INCREMENT` (연결 테이블은 복합 PK)
FK	참조 대상 PK와 같은 이름 사용 (예외: `owner_id`, `inviter_id`처럼 의미가 분명할 때)
시간	`DATETIME(3)`, **UTC 기준**으로 저장
공통 컬럼	`created_at` (기본값 현재시각), `updated_at` (수정 시 자동 갱신)
소프트 삭제	복구가 필요한 데이터(`users`, `projects`, `samples`, `chat_messages`)는 `deleted_at`으로 표시만 함
상태값	`ENUM` 사용, 값은 대문자
유연한 설정값	플러그인 파라미터, AI 추천 설정 등 구조가 자주 바뀌는 값은 `JSON`
예약어	MySQL 예약어(`references`, `key` 등)는 테이블/컬럼명으로 쓰지 않음 → `reference_tracks`, `music_key`

1.4 시간 단위(Tick) 규칙

시퀀서의 위치·길이는 초(sec)가 아닌 **tick** 단위로 저장합니다. BPM이 바뀌어도 음표 위치가 틀어지지 않기 위함입니다.

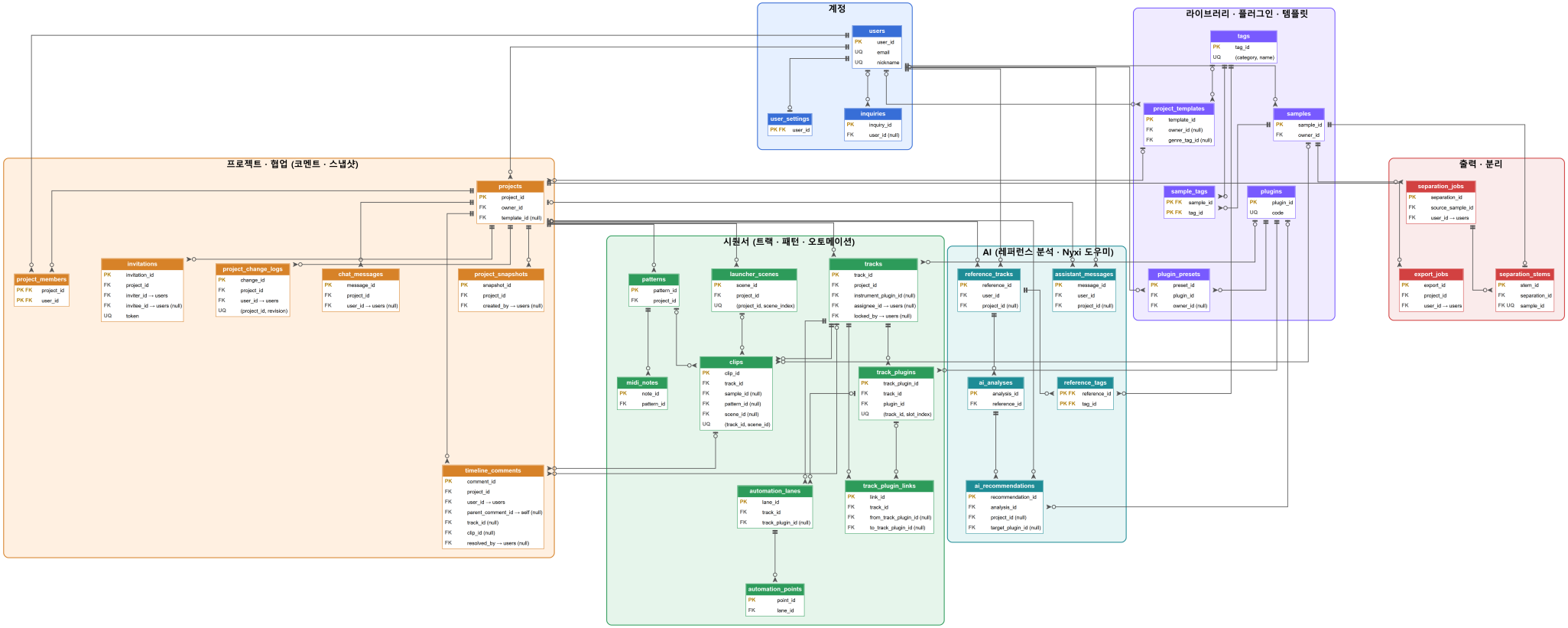
- **PPQ(4분음표 1개당 tick 수) = 960**
- 예: 4/4 박자 1마디 = 960 × 4 = 3840 tick

1.5 파일 저장 정책

- 오디오/MIDI/이미지 파일은 **DB에 BLOB으로 저장하지 않습니다.**
- 파일은 서버 파일 스토리지(개발: 로컬 `storage/` 디렉토리, 추후 S3 등으로 교체 가능)에 저장하고, DB에는 **상대 경로(`file_path`)만** 저장합니다.
- 경로 규칙 예시: `samples/{user_id}/{uuid}.wav`, `exports/{project_id}/{export_id}.mp3`

---

2. ERD


계정
users
PK user_id
UQ email
UQ nickname
user_settings
PK FK user_id
inquiries
PK inquiry_id
FK user_id (null)
라이브러리 · 플러그인 · 템플릿
tags
PK tag_id
UQ (category, name)
project_templates
PK template_id
FK owner_id (null)
FK genre_tag_id (null)
samples
PK sample_id
FK owner_id
sample_tags
PK FK sample_id
PK FK tag_id
plugins
PK plugin_id
UQ code
plugin_presets
PK preset_id
FK plugin_id
FK owner_id (null)
프로젝트 · 협업 (코멘트 · 스냅샷)
projects
PK project_id
FK owner_id
FK template_id (null)
project_members
PK FK project_id
PK FK user_id
invitations
PK invitation_id
FK project_id
FK inviter_id → users
FK invitee_id → users (null)
UQ token
project_change_logs
PK change_id
FK project_id
FK user_id → users
UQ (project_id, revision)
chat_messages
PK message_id
FK project_id
FK user_id → users (null)
project_snapshots
PK snapshot_id
FK project_id
FK created_by → users (null)
timeline_comments
PK comment_id
FK project_id
FK user_id → users
FK parent_comment_id → self (null)
FK track_id (null)
FK clip_id (null)
FK resolved_by → users (null)
시퀀서 (트랙 · 패턴 · 오토메이션)
patterns
PK pattern_id
FK project_id
launcher_scenes
PK scene_id
FK project_id
UQ (project_id, scene_index)
tracks
PK track_id
FK project_id
FK instrument_plugin_id (null)
FK assignee_id → users (null)
FK locked_by → users (null)
midi_notes
PK note_id
FK pattern_id
clips
PK clip_id
FK track_id
FK sample_id (null)
FK pattern_id (null)
FK scene_id (null)
UQ (track_id, scene_id)
track_plugins
PK track_plugin_id
FK track_id
FK plugin_id
UQ (track_id, slot_index)
automation_lanes
PK lane_id
FK track_id
FK track_plugin_id (null)
track_plugin_links
PK link_id
FK track_id
FK from_track_plugin_id (null)
FK to_track_plugin_id (null)
automation_points
PK point_id
FK lane_id
AI (레퍼런스 분석 · Nyxi 도우미)
reference_tracks
PK reference_id
FK user_id
FK project_id (null)
assistant_messages
PK message_id
FK user_id
FK project_id (null)
ai_analyses
PK analysis_id
FK reference_id
reference_tags
PK FK reference_id
PK FK tag_id
ai_recommendations
PK recommendation_id
FK analysis_id
FK project_id (null)
FK target_plugin_id (null)
출력 · 분리
separation_jobs
PK separation_id
FK source_sample_id
FK user_id → users
export_jobs
PK export_id
FK project_id
FK user_id → users
separation_stems
PK stem_id
FK separation_id
FK UQ sample_id

> - 이미지 파일: `docs/ERD.png` (고해상도)
> - 관계를 글로 정리한 문서: `docs/ERD.md`
> - 표기: 박스 안에는 **PK·FK·UQ 컬럼만** 표시 (전체 컬럼은 아래 §3). 선 끝 `|` = 1, 까마귀발 = N, `o` = 0 가능(선택).

2.1 테이블 목록

#	테이블	설명	Phase
1	`users`	회원 (+ 저장 용량)	1
2	`inquiries`	홈 화면 문의	1
3	`projects`	시퀀서 프로젝트	1 (목록) / 2 (편집)
4	`project_members`	프로젝트 참여자·권한 (생성 시 OWNER 등록)	1 / 6
5	`samples`	개인 샘플 라이브러리 (업로드·녹음·출력·분리 결과)	2
6	`tags`	태그(장르·악기·분위기)	2
7	`sample_tags`	샘플-태그 연결	2
8	`plugins`	웹 기반 가상악기/이펙트 목록	2
9	`plugin_presets`	플러그인 프리셋	2
10	`tracks`	트랙 (AUDIO / MIDI / MASTER)	2
11	`track_plugins`	트랙 디바이스 — 체인 순서 + 패처 노드 위치	2
12	`track_plugin_links`	패처 모드의 디바이스 연결선(라우팅)	2
13	`patterns`	MIDI 패턴 (링크 클립이 공유하는 노트 묶음)	2
14	`launcher_scenes`	클립 런처의 씬(행)	2
15	`clips`	타임라인/런처 위의 오디오·MIDI 클립	2
16	`midi_notes`	패턴의 노트	2
17	`automation_lanes`	오토메이션 레인 (대상 파라미터)	2
18	`automation_points`	오토메이션 포인트	2
19	`project_templates`	프로젝트 템플릿 (시스템/내 템플릿)	2
20	`user_settings`	사용자 UI 설정 (테마·단축키·캐릭터·하단 패널·튜토리얼)	2
21	`export_jobs`	오디오 출력 작업	3
22	`separation_jobs`	악기 분리 작업	4
23	`separation_stems`	분리 결과 stem	4
24	`reference_tracks`	레퍼런스 트랙(링크/파일)	5
25	`reference_tags`	레퍼런스-태그(장르) 연결	5
26	`ai_analyses`	AI 분석 결과	5
27	`ai_recommendations`	AI 추천 설정(프리셋/믹싱/마스터링)	5
28	`assistant_messages`	Nyxi 도우미(AI) 대화 기록	5
29	`invitations`	프로젝트 초대	6
30	`project_change_logs`	실시간 편집 변경 이력	6
31	`chat_messages`	프로젝트 단체 채팅(톡방)	6

#	테이블	설명	Phase
32	`timeline_comments`	타임라인 코멘트 (위치 댓글·답글·해결)	6
33	`project_snapshots`	버전 스냅샷	6

---|---|---|---| | 1 | `users` | 회원 | 1 | | 2 | `inquiries` | 홈 화면 문의 | 1 | | 3 | `projects` | 시퀀서 프로젝트 | 1 (목록) / 2 (편집) | | 4 | `samples` | 개인 샘플 라이브러리 | 2 | | 5 | `tags` | 태그(장르·악기·분위기) | 2 | | 6 | `sample_tags` | 샘플-태그 연결 | 2 | | 7 | `plugins` | 웹 기반 가상악기/이펙트 목록 | 2 | | 8 | `plugin_presets` | 플러그인 프리셋 | 2 | | 9 | `tracks` | 프로젝트의 트랙 | 2 | | 10 | `track_plugins` | 트랙 디바이스(이펙트) — 체인 순서 + 패처 노드 위치 | 2 | | 11 | `track_plugin_links` | 패처 모드의 디바이스 연결선(라우팅) | 2 | | 12 | `clips` | 트랙 위의 오디오/MIDI 클립 | 2 | | 13 | `midi_notes` | MIDI 클립의 노트 | 2 | | 14 | `user_settings` | 사용자 UI 설정 (캐릭터, 하단 패널 등) | 2 | | 15 | `export_jobs` | 오디오 출력 작업 | 3 | | 16 | `separation_jobs` | 악기 분리 작업 | 4 | | 17 | `separation_stems` | 분리 결과 stem | 4 | | 18 | `reference_tracks` | 레퍼런스 트랙(링크/파일) | 5 | | 19 | `reference_tags` | 레퍼런스-태그(장르) 연결 | 5 | | 20 | `ai_analyses` | AI 분석 결과 | 5 | | 21 | `ai_recommendations` | AI 추천 설정(프리셋/믹싱/마스터링) | 5 | | 22 | `assistant_messages` | Nyxi 도우미(AI) 대화 기록 | 5 | | 23 | `project_members` | 프로젝트 참여자·권한 | 6 | | 24 | `invitations` | 프로젝트 초대 | 6 | | 25 | `project_change_logs` | 실시간 편집 변경 이력 | 6 | | 26 | `chat_messages` | 프로젝트 단체 채팅 | 6 |

---

3. 테이블 정의

> 표기: **PK** 기본키, **FK** 외래키, **UQ** 유니크, **IDX** 인덱스, `CURRENT` = `CURRENT_TIMESTAMP(3)`

3.1 Phase 1 — 계정 · 기본

`users` — 회원

컬럼	타입	NULL	기본값	키	설명
user_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	회원 ID
email	VARCHAR(255)	N		UQ	로그인 ID
password_hash	VARCHAR(255)	N			bcrypt/argon2 해시 (평문 저장 금지)
nickname	VARCHAR(50)	N		UQ	화면 표시 이름
profile_image_path	VARCHAR(500)	Y	NULL		프로필 이미지 경로
role	ENUM('USER','ADMIN')	N	'USER'		권한
status	ENUM('ACTIVE','SUSPENDED','DELETED')	N	'ACTIVE'		계정 상태
last_login_at	DATETIME(3)	Y	NULL		마지막 로그인
storage_quota_bytes	BIGINT UNSIGNED	N	5368709120		저장 한도 (기본 5GB)
storage_used_bytes	BIGINT UNSIGNED	N	0		현재 사용량 (파일 생성·삭제 시 증감, 하루 1회 재계산)

컬럼	타입	NULL	기본값	키	설명
created_at	DATETIME(3)	N	CURRENT		가입일
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		수정일
deleted_at	DATETIME(3)	Y	NULL		탈퇴일(소프트 삭제)

`inquiries` — 문의 (home.html 문의 창)

컬럼	타입	NULL	기본값	키	설명
inquiry_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
user_id	BIGINT UNSIGNED	Y	NULL	FK → users	비회원 문의면 NULL
name	VARCHAR(50)	N			작성자 이름
email	VARCHAR(255)	N			답변 받을 이메일
title	VARCHAR(200)	N			제목
content	TEXT	N			내용
status	ENUM('RECEIVED','ANSWERED','CLOSED')	N	'RECEIVED'	IDX	처리 상태
answer	TEXT	Y	NULL		관리자 답변
answered_at	DATETIME(3)	Y	NULL		답변 일시
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

`projects` — 시퀀서 프로젝트

컬럼	타입	NULL	기본값	키	설명
project_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
owner_id	BIGINT UNSIGNED	N		FK → users, IDX	생성자(소유자)
title	VARCHAR(100)	N			프로젝트 이름
description	VARCHAR(500)	Y	NULL		설명
bpm	DECIMAL(6,2)	N	120.00		템포
time_sig_num	TINYINT UNSIGNED	N	4		박자 분자 (4/4의 앞 4)
time_sig_den	TINYINT UNSIGNED	N	4		박자 분모
music_key	VARCHAR(10)	Y	NULL		조성 (예: `C#m`)
scale_mode	VARCHAR(20)	Y	NULL		스케일 (`major`, `minor`, `dorian`, `pentatonic_minor` 등) — 스케일 도우미용

컬럼	타입	NULL	기본값	키	설명
sample_rate	INT UNSIGNED	N	48000		프로젝트 샘플레이트
master_settings	JSON	Y	NULL		마스터 부가 설정(목표 LUFS, 디더링 등). **마스터 디바이스는 MASTER 트랙의 `track_plugins`**
revision	INT UNSIGNED	N	0		협업용 문서 버전 번호 (Phase 6)
thumbnail_path	VARCHAR(500)	Y	NULL		목록용 썸네일
template_id	BIGINT UNSIGNED	Y	NULL	FK → project_templates	시작한 템플릿 (기록용)
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)	IDX	최근 수정순 정렬용
deleted_at	DATETIME(3)	Y	NULL		소프트 삭제

3.2 Phase 2 — 시퀀서 코어

`samples` — 개인 샘플 라이브러리

컬럼	타입	NULL	기본값	키	설명
sample_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
owner_id	BIGINT UNSIGNED	N		FK → users, IDX	소유자
name	VARCHAR(200)	N			표시 이름 (기본값: 원본 파일명)
original_filename	VARCHAR(255)	N			업로드 원본 파일명
file_path	VARCHAR(500)	N			스토리지 상대 경로
mime_type	VARCHAR(100)	N			`audio/wav` 등
file_size	BIGINT UNSIGNED	N			바이트
duration_ms	INT UNSIGNED	Y	NULL		길이(ms)
sample_rate	INT UNSIGNED	Y	NULL		
channels	TINYINT UNSIGNED	Y	NULL		1=mono, 2=stereo
bpm	DECIMAL(6,2)	Y	NULL		감지/입력된 BPM
music_key	VARCHAR(10)	Y	NULL		감지/입력된 키
waveform_path	VARCHAR(500)	Y	NULL		파형 미리보기 데이터(JSON/PNG) 경로
source	ENUM('UPLOAD','SEPARATION','EXPORT','RECORDING')	N	'UPLOAD'		생성 경로
created_at	DATETIME(3)	N	CURRENT		

컬럼	타입	NULL	기본값	키	설명
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		
deleted_at	DATETIME(3)	Y	NULL		

`tags` — 태그

컬럼	타입	NULL	기본값	키	설명
tag_id	INT UNSIGNED	N	AUTO_INCREMENT	PK	
name	VARCHAR(50)	N		UQ(category, name)	태그명 (예: `Lo-fi`, `Kick`)
category	ENUM('GENRE','INSTRUMENT','MOOD','ETC')	N	'ETC'		분류
is_system	TINYINT(1)	N	1		1=기본 제공 태그(클릭 선택용), 0=사용자 생성
created_at	DATETIME(3)	N	CURRENT		

`sample_tags` — 샘플-태그 연결

컬럼	타입	NULL	기본값	키	설명
sample_id	BIGINT UNSIGNED	N		PK, FK → samples	
tag_id	INT UNSIGNED	N		PK, FK → tags, IDX	

`plugins` — 웹 기반 가상악기 / 이펙트

컬럼	타입	NULL	기본값	키	설명
plugin_id	INT UNSIGNED	N	AUTO_INCREMENT	PK	
code	VARCHAR(50)	N		UQ	프론트에서 모듈을 찾는 키 (예: `nx-subsynth`, `nx-eq3`)
name	VARCHAR(100)	N			표시 이름
plugin_type	ENUM('INSTRUMENT','EFFECT')	N			가상악기 / 이펙트
engine	ENUM('WEBAUDIO','WASM')	N	'WEBAUDIO'		구현 방식 (Web Audio 노드 / C++→WASM)
version	VARCHAR(20)	N	'1.0.0'		
param_schema	JSON	N			파라미터 정의 배열: 이름, 범위, 기본값, 단위, **`primary`**(카드에 노출할 핵심 파라미터 여부), **`group`**(상세 모달의 탭/섹션)
card_asset	VARCHAR(100)	Y	NULL		디바이스 카드 아트 키 (Nyxi 에셋 `manifest.json`의 항목명, 외부 제작). NULL이면 플레이스홀더
is_active	TINYINT(1)	N	1		사용 가능 여부

컬럼	타입	NULL	기본값	키	설명
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

`plugin_presets` — 플러그인 프리셋

컬럼	타입	NULL	기본값	키	설명
preset_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
plugin_id	INT UNSIGNED	N		FK → plugins, IDX	
owner_id	BIGINT UNSIGNED	Y	NULL	FK → users	NULL이면 시스템 기본 프리셋
name	VARCHAR(100)	N			
params	JSON	N			파라미터 값
source	ENUM('SYSTEM','USER','AI')	N	'USER'		AI 추천을 저장한 경우 'AI'
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

`tracks` — 트랙

컬럼	타입	NULL	기본값	키	설명
track_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, IDX(project_id, order_index)	
name	VARCHAR(100)	N			
track_type	ENUM('AUDIO', 'MIDI','MASTER')	N			MASTER는 프로젝트당 1개, 생성 시 자동 생성·삭제 불가
order_index	INT UNSIGNED	N	0		화면 위→아래 순서
color	CHAR(7)	N	'#7C5CFF'		HEX 색상
volume_db	DECIMAL(5,2)	N	0.00		페이더 (-60 ~ +6 dB)
pan	DECIMAL(3,2)	N	0.00		-1.00(L) ~ 1.00(R)
is_muted	TINYINT(1)	N	0		
is_solo	TINYINT(1)	N	0		
instrument_plugin_id	INT UNSIGNED	Y	NULL	FK → plugins	MIDI 트랙의 가상악기
instrument_params	JSON	Y	NULL		가상악기 파라미터 값

컬럼	타입	NULL	기본값	키	설명
routing_mode	ENUM('CHAIN','PATCHER')	N	'CHAIN'		디바이스 라우팅 방식. CHAIN: `slot_index` 순서대로 직렬 연결 / PATCHER: `track_plugin_links` 연결선대로 처리
assignee_id	BIGINT UNSIGNED	Y	NULL	FK → users	담당 멤버 (표시용, Phase 6)
locked_by	BIGINT UNSIGNED	Y	NULL	FK → users	잠근 멤버. NULL이면 잠금 없음 (Phase 6)
locked_at	DATETIME(3)	Y	NULL		잠근 시각 (자동 해제 판단)
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

> 트랙 아코디언의 펼침/접힘 상태는 사용자마다 다르므로 DB에 저장하지 않고 브라우저(localStorage)에 저장합니다.

`track_plugins` — 트랙 디바이스 (체인 + 패처 노드)

컬럼	타입	NULL	기본값	키	설명
track_plugin_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
track_id	BIGINT UNSIGNED	N		FK → tracks, UQ(track_id, slot_index)	
plugin_id	INT UNSIGNED	N		FK → plugins	`plugin_type = 'EFFECT'` 만 허용(애플리케이션에서 검증)
slot_index	TINYINT UNSIGNED	N			체인 뷰에서의 순서 (0부터). CHAIN 모드에서는 이 순서가 곧 신호 흐름
node_x	INT	Y	NULL		패처 뷰에서 노드의 X 좌표(px)
node_y	INT	Y	NULL		패처 뷰에서 노드의 Y 좌표(px)
params	JSON	N			파라미터 값
is_bypassed	TINYINT(1)	N	0		
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

`track_plugin_links` — 패처 연결선

> `tracks.routing_mode = 'PATCHER'` 일 때만 사용. CHAIN → PATCHER 전환 시 현재 체인 순서대로 연결선을 자동 생성합니다.

컬럼	타입	NULL	기본값	키	설명
link_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
track_id	BIGINT UNSIGNED	N		FK → tracks, IDX	
from_track_plugin_id	BIGINT UNSIGNED	Y	NULL	FK → track_plugins	출발 노드. **NULL = 트랙 소스**(오디오 클립 또는 가상악기 출력)
from_port	TINYINT UNSIGNED	N	0		출발 노드의 출력 포트 번호
to_track_plugin_id	BIGINT UNSIGNED	Y	NULL	FK → track_plugins	도착 노드. **NULL = 트랙 출력**(페이더 → 마스터)
to_port	TINYINT UNSIGNED	N	0		도착 노드의 입력 포트 번호 (사이드체인 등은 1번 이상)
gain_db	DECIMAL(5,2)	N	0.00		연결선 게인 (병렬 처리 시 비율 조절)
created_at	DATETIME(3)	N	CURRENT		

> 순환(사이클) 연결과, 같은 노드·포트 쌍의 중복 연결은 애플리케이션에서 막습니다.

`clips` — 클립

컬럼	타입	NULL	기본값	키	설명
clip_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
track_id	BIGINT UNSIGNED	N		FK → tracks, IDX(track_id, start_tick)	
clip_type	ENUM('AUDIO','MIDI')	N			트랙 타입과 일치해야 함
sample_id	BIGINT UNSIGNED	Y	NULL	FK → samples	AUDIO 클립일 때만 사용
pattern_id	BIGINT UNSIGNED	Y	NULL	FK → patterns, IDX	**MIDI 클립은 필수.** 같은 패턴을 가리키는 클립끼리 링크(하나를 고치면 모두 바뀜)
scene_id	BIGINT UNSIGNED	Y	NULL	FK → launcher_scenes, UQ(track_id, scene_id)	NULL = 타임라인 클립 / 값 있음 = 클립 런처 슬롯 (이때 `start_tick`=0)
name	VARCHAR(100)	Y	NULL		
start_tick	INT UNSIGNED	N			타임라인 시작 위치 (런처 클립은 0)
length_tick	INT UNSIGNED	N			클립 길이
offset_tick	INT UNSIGNED	N	0		샘플 앞부분 잘라낸 양 (트리밍)
gain_db	DECIMAL(5,2)	N	0.00		클립 게인

컬럼	타입	NULL	기본값	키	설명
is_looped	TINYINT(1)	N	0		
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

`midi_notes` — MIDI 노트

컬럼	타입	NULL	기본값	키	설명
note_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
pattern_id	BIGINT UNSIGNED	N		FK → patterns, IDX(pattern_id, start_tick)	노트는 클립이 아니라 **패턴**에 속함
pitch	TINYINT UNSIGNED	N			0~127 (60 = C4)
velocity	TINYINT UNSIGNED	N	100		1~127
start_tick	INT UNSIGNED	N			**패턴 시작 기준** 상대 위치
length_tick	INT UNSIGNED	N			
channel	TINYINT UNSIGNED	N	0		0~15

`patterns` — MIDI 패턴

> MIDI 클립의 노트는 패턴에 저장됩니다. 클립 복제 시 **링크 복제**는 같은 `pattern_id`, **독립 복제**는 패턴을 복사해 새 `pattern_id`를 씁니다.

컬럼	타입	NULL	기본값	키	설명
pattern_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, IDX	
name	VARCHAR(100)	N			패턴 이름 (예: `Drum A`)
color	CHAR(7)	Y	NULL		링크 클립 표시 색
length_tick	INT UNSIGNED	N	3840		패턴 길이 (기본 1마디)
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

`launcher_scenes` — 클립 런처 씬

컬럼	타입	NULL	기본값	키	설명
scene_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, UQ(project_id, scene_index)	
scene_index	SMALLINT UNSIGNED	N			위→아래 순서
name	VARCHAR(100)	Y	NULL		씬 이름 (예: `Verse`, `Drop`)
created_at	DATETIME(3)	N	CURRENT		

`automation_lanes` — 오토메이션 레인

컬럼	타입	NULL	기본값	키	설명
lane_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
track_id	BIGINT UNSIGNED	N		FK → tracks, IDX	대상 트랙 (MASTER 트랙 포함)
target_type	ENUM('VOLUME','PAN','INSTRUMENT_PARAM','DEVICE_PARAM')	N			대상 종류
track_plugin_id	BIGINT UNSIGNED	Y	NULL	FK → track_plugins	`DEVICE_PARAM` 일 때 대상 디바이스
param_index	SMALLINT UNSIGNED	Y	NULL		`*_PARAM` 일 때 `param_schema` 순번
is_enabled	TINYINT(1)	N	1		레인 켜기/끄기
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

> 같은 대상에 레인이 두 개 생기지 않도록 애플리케이션에서 검사합니다 (NULL이 섞여 DB UNIQUE로는 막기 어려움).

`automation_points` — 오토메이션 포인트

컬럼	타입	NULL	기본값	키	설명
point_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
lane_id	BIGINT UNSIGNED	N		FK → automation_lanes, IDX(lane_id, tick)	
tick	INT UNSIGNED	N			타임라인 위치
value	DECIMAL(12,4)	N			파라미터 **실제 단위** 값 (dB, Hz 등)

컬럼	타입	NULL	기본값	키	설명
curve	DECIMAL(3,2)	N	0.00		다음 포인트까지 곡선 정도 (-1.00 ~ 1.00, 0 = 직선)

`project_templates` — 프로젝트 템플릿

컬럼	타입	NULL	기본값	키	설명
template_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
owner_id	BIGINT UNSIGNED	Y	NULL	FK → users, IDX	NULL = 시스템 템플릿
name	VARCHAR(100)	N			
description	VARCHAR(500)	Y	NULL		
genre_tag_id	INT UNSIGNED	Y	NULL	FK → tags	장르
data	JSON	N			BPM·박자·키, 트랙 구성, 디바이스 체인, (선택) 패턴·샘플 참조
preview_path	VARCHAR(500)	Y	NULL		미리듣기 오디오
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

`user_settings` — 사용자 UI 설정 (users와 1:1)

컬럼	타입	NULL	기본값	키	설명
user_id	BIGINT UNSIGNED	N		PK, FK → users	
theme	ENUM('DARK','LIGHT','SYSTEM')	N	'DARK'		테마
accent_color	CHAR(7)	N	'#7C5CFF'		강조 색
ui_scale	DECIMAL(3,2)	N	1.00		UI 배율 (0.80 ~ 1.50)
keybindings	JSON	Y	NULL		사용자 지정 단축키 `{명령ID: 키}` (기본값과 다른 것만 저장)
tutorial_state	JSON	Y	NULL		튜토리얼 코스별 진행 단계·완료 여부
character_enabled	TINYINT(1)	N	1		시퀀서 화면 캐릭터 표시 여부
character_code	VARCHAR(50)	N	'nyxi'		선택한 캐릭터/스킨 (에셋 폴더 `static/characters/{code}/`)
character_state	JSON	Y	NULL		캐릭터 창 위치·크기, 춤 on/off, 말풍선 on/off

컬럼	타입	NULL	기본값	키	설명
tips_level	ENUM('OFF','IMPORTANT','ALL')	N	'IMPORTANT'		Nyxi 팁 레벨 (끄기 / 중요한 것만 / 모두)
tips_state	JSON	Y	NULL		다시 보지 않기 팁 목록, 팁별 마지막 표시 시각, **익숙해짐 확인창(프로젝트 5개) 표시 시각**
bottom_panel_state	JSON	Y	NULL		하단 패널 높이, 마지막으로 연 탭(톡방/Nyxi)
ui_settings	JSON	Y	NULL		기타 UI 설정 (글자 크기, 고대비, 패널 위치·너비, 파형·미터 스타일, 스냅·자동저장 기본값, 오디오 버퍼, 녹음 레이턴시 보정값, 로컬 캐시 상한 등)
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		

3.3 Phase 3 — 오디오 출력

`export_jobs` — 출력 작업

컬럼	타입	NULL	기본값	키	설명
export_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, IDX	
user_id	BIGINT UNSIGNED	N		FK → users	요청자
format	ENUM('WAV','MP3','FLAC')	N			
sample_rate	INT UNSIGNED	N	44100		
bit_depth	TINYINT UNSIGNED	Y	NULL		WAV/FLAC: 16/24/32
bitrate_kbps	SMALLINT UNSIGNED	Y	NULL		MP3: 128/192/320
range_start_tick	INT UNSIGNED	N	0		출력 구간 시작
range_end_tick	INT UNSIGNED	Y	NULL		NULL이면 프로젝트 끝까지
render_mode	ENUM('CLIENT','SERVER')	N	'CLIENT'		렌더링 위치
status	ENUM('PENDING','PROCESSING','DONE','FAILED')	N	'PENDING'	IDX	
progress	TINYINT UNSIGNED	N	0		0~100
result_path	VARCHAR(500)	Y	NULL		결과 파일 경로
file_size	BIGINT UNSIGNED	Y	NULL		
error_message	VARCHAR(1000)	Y	NULL		
created_at	DATETIME(3)	N	CURRENT		
completed_at	DATETIME(3)	Y	NULL		

3.4 Phase 4 — 악기 분리

`separation_jobs` — 분리 작업

컬럼	타입	NULL	기본값	키	설명
separation_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
user_id	BIGINT UNSIGNED	N		FK → users, IDX	요청자
source_sample_id	BIGINT UNSIGNED	N		FK → samples	분리할 원본
model_name	VARCHAR(50)	N			사용 모델 (예: `htdemucs`)
stem_mode	ENUM('2STEM','4STEM','6STEM')	N	'4STEM'		2: 보컬/반주, 4: 보컬/드럼/베이스/기타, 6: +기타/피아노
status	ENUM('PENDING','PROCESSING','DONE','FAILED')	N	'PENDING'	IDX	
progress	TINYINT UNSIGNED	N	0		
error_message	VARCHAR(1000)	Y	NULL		
created_at	DATETIME(3)	N	CURRENT		
completed_at	DATETIME(3)	Y	NULL		

`separation_stems` — 분리 결과

컬럼	타입	NULL	기본값	키	설명
stem_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
separation_id	BIGINT UNSIGNED	N		FK → separation_jobs, UQ(separation_id, stem_type)	
stem_type	ENUM('VOCALS','DRUMS','BASS','GUITAR','PIANO','OTHER','ACCOMPANIMENT')	N			
sample_id	BIGINT UNSIGNED	N		FK → samples, UQ	결과 파일은 `samples`에 `source='SEPARATION'`으로 등록됨
created_at	DATETIME(3)	N	CURRENT		

3.5 Phase 5 — AI 레퍼런스 분석

`reference_tracks` — 레퍼런스 트랙

> `references`는 MySQL 예약어라 `reference_tracks`로 이름 지음

컬럼	타입	NULL	기본값	키	설명
reference_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
user_id	BIGINT UNSIGNED	N		FK → users, IDX	등록자
project_id	BIGINT UNSIGNED	Y	NULL	FK → projects, IDX	연결된 프로젝트 (없을 수 있음)
source_type	ENUM('LINK','FILE')	N			
source_url	VARCHAR(1000)	Y	NULL		LINK일 때 (YouTube 등)
file_path	VARCHAR(500)	Y	NULL		FILE일 때 업로드 경로
title	VARCHAR(200)	Y	NULL		곡 제목 (링크 메타데이터 또는 직접 입력)
genre_text	VARCHAR(100)	Y	NULL		사용자가 직접 입력한 주 장르
created_at	DATETIME(3)	N	CURRENT		

`reference_tags` — 레퍼런스-태그(장르) 연결

컬럼	타입	NULL	기본값	키	설명
reference_id	BIGINT UNSIGNED	N		PK, FK → reference_tracks	
tag_id	INT UNSIGNED	N		PK, FK → tags	태그 클릭으로 선택한 장르

`ai_analyses` — AI 분석 결과

컬럼	타입	NULL	기본값	키	설명
analysis_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
reference_id	BIGINT UNSIGNED	N		FK → reference_tracks, IDX	
status	ENUM('PENDING','PROCESSING','DONE','FAILED')	N	'PENDING'		
model_version	VARCHAR(50)	Y	NULL		분석에 사용한 모델/버전
bpm	DECIMAL(6,2)	Y	NULL		
music_key	VARCHAR(10)	Y	NULL		
loudness_lufs	DECIMAL(5,2)	Y	NULL		통합 라우드니스
true_peak_db	DECIMAL(5,2)	Y	NULL		
dynamic_range_db	DECIMAL(5,2)	Y	NULL		
spectral_profile	JSON	Y	NULL		대역별 에너지 분포

컬럼	타입	NULL	기본값	키	설명
features	JSON	Y	NULL		기타 특징(악기 구성, 스테레오 폭 등)
summary	TEXT	Y	NULL		사람이 읽을 분석 요약
error_message	VARCHAR(1000)	Y	NULL		
created_at	DATETIME(3)	N	CURRENT		
completed_at	DATETIME(3)	Y	NULL		

`ai_recommendations` — AI 추천 설정

컬럼	타입	NULL	기본값	키	설명
recommendation_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
analysis_id	BIGINT UNSIGNED	N		FK → ai_analyses, IDX	
project_id	BIGINT UNSIGNED	Y	NULL	FK → projects	적용 대상 프로젝트
rec_type	ENUM('SYNTH_PRESET','PLUGIN_PRESET','MIXING','MASTERING')	N			추천 종류
target_plugin_id	INT UNSIGNED	Y	NULL	FK → plugins	프리셋 추천일 때 대상 플러그인
settings	JSON	N			추천 설정값
rationale	TEXT	Y	NULL		추천 이유 설명
is_applied	TINYINT(1)	N	0		프로젝트에 적용했는지
applied_at	DATETIME(3)	Y	NULL		
created_at	DATETIME(3)	N	CURRENT		

`assistant_messages` — Nyxi 도우미(AI) 대화

컬럼	타입	NULL	기본값	키	설명
message_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
user_id	BIGINT UNSIGNED	N		FK → users, IDX(user_id, project_id, message_id)	대화 주인 (Nyxi 대화는 개인별, 협업 멤버와 공유하지 않음)
project_id	BIGINT UNSIGNED	Y	NULL	FK → projects	대화가 일어난 프로젝트
role	ENUM('USER','ASSISTANT')	N			질문 / 답변
content	TEXT	N			메시지 내용
context	JSON	Y	NULL		질문 시점의 프로젝트 요약 (BPM, 트랙·디바이스 구성 등)
model_version	VARCHAR(50)	Y	NULL		답변 생성 모델

컬럼	타입	NULL	기본값	키	설명
created_at	DATETIME(3)	N	CURRENT		

3.6 Phase 6 — 협업 · 단체톡

`project_members` — 프로젝트 참여자

컬럼	타입	NULL	기본값	키	설명
project_id	BIGINT UNSIGNED	N		PK, FK → projects	
user_id	BIGINT UNSIGNED	N		PK, FK → users, IDX	
role	ENUM('OWNER','EDITOR','VIEWER')	N	'EDITOR'		권한
last_read_message_id	BIGINT UNSIGNED	Y	NULL		채팅 안 읽은 수 계산용
joined_at	DATETIME(3)	N	CURRENT		

> 프로젝트 생성 시 소유자도 `role='OWNER'`로 함께 INSERT 합니다. (Phase 1부터 넣어 두면 Phase 6에서 마이그레이션이 필요 없음)

`invitations` — 초대

컬럼	타입	NULL	기본값	키	설명
invitation_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, IDX	
inviter_id	BIGINT UNSIGNED	N		FK → users	초대한 사람
invitee_email	VARCHAR(255)	N		IDX	초대받는 이메일
invitee_id	BIGINT UNSIGNED	Y	NULL	FK → users	이미 회원이면 채움
role	ENUM('EDITOR','VIEWER')	N	'EDITOR'		부여할 권한
token	CHAR(64)	N		UQ	초대 링크 토큰
status	ENUM('PENDING','ACCEPTED','DECLINED','EXPIRED','CANCELED')	N	'PENDING'		
expires_at	DATETIME(3)	N			만료 시각 (기본 7일)
created_at	DATETIME(3)	N	CURRENT		
responded_at	DATETIME(3)	Y	NULL		

`project_change_logs` — 편집 변경 이력

컬럼	타입	NULL	기본값	키	설명
change_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, UQ(project_id, revision)	
user_id	BIGINT UNSIGNED	N		FK → users	변경한 사람
revision	INT UNSIGNED	N			이 변경이 적용된 후의 `projects.revision`
op_type	VARCHAR(50)	N			예: `clip.move`, `note.add`, `track.update`
target_type	VARCHAR(30)	N			`track`, `clip`, `note` 등
target_id	BIGINT UNSIGNED	Y	NULL		대상 행 ID
payload	JSON	N			변경 내용 (이전값/새값)
created_at	DATETIME(3)	N	CURRENT		

`chat_messages` — 단체 채팅

컬럼	타입	NULL	기본값	키	설명
message_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, IDX(project_id, message_id)	프로젝트 = 채팅방
user_id	BIGINT UNSIGNED	Y	NULL	FK → users	SYSTEM 메시지면 NULL
message_type	ENUM('TEXT','SYSTEM')	N	'TEXT'		SYSTEM: "OO님이 입장했습니다" 등
content	TEXT	N			
created_at	DATETIME(3)	N	CURRENT		
deleted_at	DATETIME(3)	Y	NULL		

`timeline_comments` — 타임라인 코멘트

컬럼	타입	NULL	기본값	키	설명
comment_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, IDX(project_id, resolved_at)	
user_id	BIGINT UNSIGNED	N		FK → users	작성자

컬럼	타입	NULL	기본값	키	설명
parent_comment_id	BIGINT UNSIGNED	Y	NULL	FK → timeline_comments	답글이면 원 코멘트 ID
tick	INT UNSIGNED	Y	NULL		위치 (답글은 NULL)
track_id	BIGINT UNSIGNED	Y	NULL	FK → tracks	특정 트랙에 붙인 경우
clip_id	BIGINT UNSIGNED	Y	NULL	FK → clips	특정 클립에 붙인 경우 (클립 이동 시 따라감)
content	TEXT	N			`@닉네임` 멘션 포함 가능
resolved_at	DATETIME(3)	Y	NULL		해결 처리 시각
resolved_by	BIGINT UNSIGNED	Y	NULL	FK → users	해결 처리한 사람
created_at	DATETIME(3)	N	CURRENT		
updated_at	DATETIME(3)	N	CURRENT (ON UPDATE)		
deleted_at	DATETIME(3)	Y	NULL		소프트 삭제

`project_snapshots` — 버전 스냅샷

컬럼	타입	NULL	기본값	키	설명
snapshot_id	BIGINT UNSIGNED	N	AUTO_INCREMENT	PK	
project_id	BIGINT UNSIGNED	N		FK → projects, IDX(project_id, created_at)	
created_by	BIGINT UNSIGNED	Y	NULL	FK → users	NULL = 자동 스냅샷
snapshot_type	ENUM('MANUAL','AUTO_DAILY','BEFORE_RESTORE','BEFORE_TEMPLATE','BEFORE_AI_APPLY')	N			생성 이유
name	VARCHAR(100)	Y	NULL		수동 스냅샷 이름 (예: `믹스 v1`)
revision	INT UNSIGNED	N			스냅샷 시점의 `projects.revision`
data_path	VARCHAR(500)	N			전체 상태 압축 JSON 파일 경로 (샘플 오디오는 복사하지 않고 참조)
preview_path	VARCHAR(500)	Y	NULL		미리듣기 오디오 (요청 시 생성)
size_bytes	BIGINT UNSIGNED	N	0		파일 크기 합 (저장 용량에 포함)
is_pinned	TINYINT(1)	N	0		고정 — 자동 정리 제외
created_at	DATETIME(3)	N	CURRENT		

> 스냅샷 데이터를 DB의 JSON 컬럼이 아닌 **파일**로 두는 이유: 프로젝트 전체 상태(노트 수천 개)는 수 MB가 될 수 있어 DB 백업·조회를 무겁게 만들기 때문입니다.

---

4. 삭제(ON DELETE) 정책

관계	정책	이유
projects → tracks → clips	CASCADE	프로젝트 구성요소는 프로젝트와 생명주기가 같음
projects → patterns → midi_notes	CASCADE	
patterns → clips	RESTRICT	클립이 쓰는 패턴은 지울 수 없음 (마지막 클립이 지워지면 애플리케이션이 패턴 정리)
projects → launcher_scenes → clips(scene_id)	CASCADE	씬을 지우면 그 행의 런처 클립도 삭제
tracks → automation_lanes → automation_points	CASCADE	
track_plugins → automation_lanes	CASCADE	디바이스를 지우면 그 파라미터 오토메이션도 삭제
projects → timeline_comments, project_snapshots	CASCADE	
tracks / clips → timeline_comments	SET NULL	트랙·클립이 지워져도 코멘트는 tick 위치로 남음
timeline_comments → 답글	CASCADE	원 코멘트 삭제 시 답글도 삭제 (실제로는 소프트 삭제 사용)
project_templates → projects.template_id	SET NULL	템플릿을 지워도 프로젝트는 유지
tracks → track_plugins, track_plugin_links	CASCADE	
track_plugins → track_plugin_links	CASCADE	디바이스를 지우면 연결된 선도 삭제
users → user_settings	CASCADE	1:1 부속 데이터
clips.sample_id → samples	RESTRICT	사용 중인 샘플은 바로 지울 수 없음 (samples는 소프트 삭제)
sample_tags, reference_tags	CASCADE	연결 테이블
separation_jobs → separation_stems	CASCADE	
ai_analyses → ai_recommendations	CASCADE	
projects → project_members / invitations / change_logs / chat_messages	CASCADE	
users → (각종 FK)	RESTRICT	회원은 소프트 삭제만 하므로 실제 DELETE가 일어나지 않음

관계	정책	이유
*.project_id (NULL 허용: reference_tracks, ai_recommendations, assistant_messages)	SET NULL	프로젝트가 지워져도 분석 기록은 남김

5. DDL 초안

> 아래 SQL은 **문서용 초안**입니다. 실제 실행 전 검토가 필요하며, 이 문서 작성 시점에는 DB에 실행하지 않았습니다. FK 의존 순서대로 정렬되어 있어 위에서부터 차례로 실행하면 됩니다.

```
USE studionx;

-- ========== Phase 1 ==========
CREATE TABLE users (
  user_id            BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  email              VARCHAR(255)    NOT NULL,
  password_hash      VARCHAR(255)    NOT NULL,
  nickname           VARCHAR(50)     NOT NULL,
  profile_image_path VARCHAR(500)    NULL,
  role               ENUM('USER','ADMIN') NOT NULL DEFAULT 'USER',
  status             ENUM('ACTIVE','SUSPENDED','DELETED') NOT NULL DEFAULT 'ACTIVE',
  last_login_at      DATETIME(3)     NULL,
  storage_quota_bytes BIGINT UNSIGNED NOT NULL DEFAULT 5368709120,
  storage_used_bytes BIGINT UNSIGNED NOT NULL DEFAULT 0,
  created_at         DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at         DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP
(3),
  deleted_at         DATETIME(3)     NULL,
  PRIMARY KEY (user_id),
  UNIQUE KEY uq_users_email (email),
  UNIQUE KEY uq_users_nickname (nickname)
) ENGINE=InnoDB;

CREATE TABLE inquiries (
  inquiry_id  BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id     BIGINT UNSIGNED NULL,
  name        VARCHAR(50)     NOT NULL,
  email       VARCHAR(255)    NOT NULL,
  title       VARCHAR(200)    NOT NULL,
  content     TEXT            NOT NULL,
  status      ENUM('RECEIVED','ANSWERED','CLOSED') NOT NULL DEFAULT 'RECEIVED',
  answer      TEXT            NULL,
  answered_at DATETIME(3)     NULL,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (inquiry_id),
  KEY idx_inquiries_status (status),
  CONSTRAINT fk_inquiries_user FOREIGN KEY (user_id) REFERENCES users (user_id)
) ENGINE=InnoDB;

CREATE TABLE projects (
  project_id      BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  owner_id        BIGINT UNSIGNED  NOT NULL,
```

```
  title           VARCHAR(100)     NOT NULL,
  description     VARCHAR(500)     NULL,
  bpm             DECIMAL(6,2)     NOT NULL DEFAULT 120.00,
  time_sig_num    TINYINT UNSIGNED NOT NULL DEFAULT 4,
  time_sig_den    TINYINT UNSIGNED NOT NULL DEFAULT 4,
  music_key       VARCHAR(10)      NULL,
  scale_mode      VARCHAR(20)      NULL,
  sample_rate     INT UNSIGNED     NOT NULL DEFAULT 48000,
  master_settings JSON             NULL,
  revision        INT UNSIGNED     NOT NULL DEFAULT 0,
  thumbnail_path  VARCHAR(500)     NULL,
  template_id     BIGINT UNSIGNED  NULL,  -- FK는 project_templates 생성 후 ALTER로 추가
  created_at      DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at      DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  deleted_at      DATETIME(3)      NULL,
  PRIMARY KEY (project_id),
  KEY idx_projects_owner (owner_id),
  KEY idx_projects_updated (updated_at),
  CONSTRAINT fk_projects_owner FOREIGN KEY (owner_id) REFERENCES users (user_id)
) ENGINE=InnoDB;

-- project_members는 Phase 6 기능이지만, 프로젝트 생성 시 OWNER 행을 넣기 위해 Phase 1에서 함께 생성
CREATE TABLE project_members (
  project_id           BIGINT UNSIGNED NOT NULL,
  user_id              BIGINT UNSIGNED NOT NULL,
  role                 ENUM('OWNER','EDITOR','VIEWER') NOT NULL DEFAULT 'EDITOR',
  last_read_message_id BIGINT UNSIGNED NULL,
  joined_at            DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (project_id, user_id),
  KEY idx_project_members_user (user_id),
  CONSTRAINT fk_pm_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASCADE,
  CONSTRAINT fk_pm_user    FOREIGN KEY (user_id)    REFERENCES users (user_id)
) ENGINE=InnoDB;

-- ========== Phase 2 ==========
CREATE TABLE samples (
  sample_id         BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  owner_id          BIGINT UNSIGNED  NOT NULL,
  name              VARCHAR(200)     NOT NULL,
  original_filename VARCHAR(255)     NOT NULL,
  file_path         VARCHAR(500)     NOT NULL,
  mime_type         VARCHAR(100)     NOT NULL,
  file_size         BIGINT UNSIGNED  NOT NULL,
  duration_ms       INT UNSIGNED     NULL,
  sample_rate       INT UNSIGNED     NULL,
  channels          TINYINT UNSIGNED NULL,
  bpm               DECIMAL(6,2)     NULL,
  music_key         VARCHAR(10)      NULL,
  waveform_path     VARCHAR(500)     NULL,
  source            ENUM('UPLOAD','SEPARATION','EXPORT','RECORDING') NOT NULL DEFAULT 'UPLOAD',
  created_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at        DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP
(3),
  deleted_at        DATETIME(3)      NULL,
  PRIMARY KEY (sample_id),
  KEY idx_samples_owner (owner_id),
  CONSTRAINT fk_samples_owner FOREIGN KEY (owner_id) REFERENCES users (user_id)
```

```
) ENGINE=InnoDB;

CREATE TABLE tags (
  tag_id     INT UNSIGNED NOT NULL AUTO_INCREMENT,
  name       VARCHAR(50)  NOT NULL,
  category   ENUM('GENRE','INSTRUMENT','MOOD','ETC') NOT NULL DEFAULT 'ETC',
  is_system  TINYINT(1)   NOT NULL DEFAULT 1,
  created_at DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (tag_id),
  UNIQUE KEY uq_tags_category_name (category, name)
) ENGINE=InnoDB;

CREATE TABLE sample_tags (
  sample_id BIGINT UNSIGNED NOT NULL,
  tag_id    INT UNSIGNED    NOT NULL,
  PRIMARY KEY (sample_id, tag_id),
  KEY idx_sample_tags_tag (tag_id),
  CONSTRAINT fk_st_sample FOREIGN KEY (sample_id) REFERENCES samples (sample_id) ON DELETE CASCADE,
  CONSTRAINT fk_st_tag    FOREIGN KEY (tag_id)    REFERENCES tags (tag_id)       ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE plugins (
  plugin_id    INT UNSIGNED NOT NULL AUTO_INCREMENT,
  code         VARCHAR(50)  NOT NULL,
  name         VARCHAR(100) NOT NULL,
  plugin_type  ENUM('INSTRUMENT','EFFECT') NOT NULL,
  engine       ENUM('WEBAUDIO','WASM') NOT NULL DEFAULT 'WEBAUDIO',
  version      VARCHAR(20)  NOT NULL DEFAULT '1.0.0',
  param_schema JSON         NOT NULL,
  card_asset   VARCHAR(100) NULL,
  is_active    TINYINT(1)   NOT NULL DEFAULT 1,
  created_at   DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3)  NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (plugin_id),
  UNIQUE KEY uq_plugins_code (code)
) ENGINE=InnoDB;

CREATE TABLE plugin_presets (
  preset_id  BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  plugin_id  INT UNSIGNED    NOT NULL,
  owner_id   BIGINT UNSIGNED NULL,
  name       VARCHAR(100)    NOT NULL,
  params     JSON            NOT NULL,
  source     ENUM('SYSTEM','USER','AI') NOT NULL DEFAULT 'USER',
  created_at DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (preset_id),
  KEY idx_presets_plugin (plugin_id),
  CONSTRAINT fk_presets_plugin FOREIGN KEY (plugin_id) REFERENCES plugins (plugin_id),
  CONSTRAINT fk_presets_owner  FOREIGN KEY (owner_id)  REFERENCES users (user_id)
) ENGINE=InnoDB;

CREATE TABLE tracks (
  track_id             BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  project_id           BIGINT UNSIGNED NOT NULL,
  name                 VARCHAR(100)    NOT NULL,
  track_type           ENUM('AUDIO','MIDI','MASTER') NOT NULL,
```

```
  order_index          INT UNSIGNED    NOT NULL DEFAULT 0,
  color                CHAR(7)         NOT NULL DEFAULT '#7C5CFF',
  volume_db            DECIMAL(5,2)    NOT NULL DEFAULT 0.00,
  pan                  DECIMAL(3,2)    NOT NULL DEFAULT 0.00,
  is_muted             TINYINT(1)      NOT NULL DEFAULT 0,
  is_solo              TINYINT(1)      NOT NULL DEFAULT 0,
  instrument_plugin_id INT UNSIGNED    NULL,
  instrument_params    JSON            NULL,
  routing_mode         ENUM('CHAIN','PATCHER') NOT NULL DEFAULT 'CHAIN',
  assignee_id          BIGINT UNSIGNED NULL,
  locked_by            BIGINT UNSIGNED NULL,
  locked_at            DATETIME(3)     NULL,
  created_at           DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at           DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP
(3),
  PRIMARY KEY (track_id),
  KEY idx_tracks_project_order (project_id, order_index),
  CONSTRAINT fk_tracks_project    FOREIGN KEY (project_id)          REFERENCES projects (project_id) ON
DELETE CASCADE,
  CONSTRAINT fk_tracks_instrument FOREIGN KEY (instrument_plugin_id) REFERENCES plugins (plugin_id),
  CONSTRAINT fk_tracks_assignee   FOREIGN KEY (assignee_id)          REFERENCES users (user_id),
  CONSTRAINT fk_tracks_locked_by  FOREIGN KEY (locked_by)            REFERENCES users (user_id)
) ENGINE=InnoDB;

CREATE TABLE track_plugins (
  track_plugin_id BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  track_id        BIGINT UNSIGNED  NOT NULL,
  plugin_id       INT UNSIGNED     NOT NULL,
  slot_index      TINYINT UNSIGNED NOT NULL,
  node_x          INT              NULL,
  node_y          INT              NULL,
  params          JSON             NOT NULL,
  is_bypassed     TINYINT(1)       NOT NULL DEFAULT 0,
  created_at      DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at      DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (track_plugin_id),
  UNIQUE KEY uq_track_plugins_slot (track_id, slot_index),
  CONSTRAINT fk_tp_track  FOREIGN KEY (track_id)  REFERENCES tracks (track_id) ON DELETE CASCADE,
  CONSTRAINT fk_tp_plugin FOREIGN KEY (plugin_id) REFERENCES plugins (plugin_id)
) ENGINE=InnoDB;

CREATE TABLE track_plugin_links (
  link_id              BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  track_id             BIGINT UNSIGNED  NOT NULL,
  from_track_plugin_id BIGINT UNSIGNED  NULL,
  from_port            TINYINT UNSIGNED NOT NULL DEFAULT 0,
  to_track_plugin_id   BIGINT UNSIGNED  NULL,
  to_port              TINYINT UNSIGNED NOT NULL DEFAULT 0,
  gain_db              DECIMAL(5,2)     NOT NULL DEFAULT 0.00,
  created_at           DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (link_id),
  KEY idx_links_track (track_id),
  CONSTRAINT fk_links_track FOREIGN KEY (track_id)             REFERENCES tracks (track_id)
ON DELETE CASCADE,
  CONSTRAINT fk_links_from  FOREIGN KEY (from_track_plugin_id) REFERENCES track_plugins (track_plugin_i
d) ON DELETE CASCADE,
  CONSTRAINT fk_links_to    FOREIGN KEY (to_track_plugin_id)   REFERENCES track_plugins (track_plugin_i
```

```
d) ON DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE patterns (
  pattern_id  BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  project_id  BIGINT UNSIGNED NOT NULL,
  name        VARCHAR(100)    NOT NULL,
  color       CHAR(7)         NULL,
  length_tick INT UNSIGNED    NOT NULL DEFAULT 3840,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (pattern_id),
  KEY idx_patterns_project (project_id),
  CONSTRAINT fk_patterns_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CAS
CADE
) ENGINE=InnoDB;

CREATE TABLE launcher_scenes (
  scene_id    BIGINT UNSIGNED   NOT NULL AUTO_INCREMENT,
  project_id  BIGINT UNSIGNED   NOT NULL,
  scene_index SMALLINT UNSIGNED NOT NULL,
  name        VARCHAR(100)      NULL,
  created_at  DATETIME(3)       NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (scene_id),
  UNIQUE KEY uq_scenes_index (project_id, scene_index),
  CONSTRAINT fk_scenes_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASCA
DE
) ENGINE=InnoDB;

CREATE TABLE clips (
  clip_id     BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  track_id    BIGINT UNSIGNED NOT NULL,
  clip_type   ENUM('AUDIO','MIDI') NOT NULL,
  sample_id   BIGINT UNSIGNED NULL,
  pattern_id  BIGINT UNSIGNED NULL,
  scene_id    BIGINT UNSIGNED NULL,
  name        VARCHAR(100)    NULL,
  start_tick  INT UNSIGNED    NOT NULL,
  length_tick INT UNSIGNED    NOT NULL,
  offset_tick INT UNSIGNED    NOT NULL DEFAULT 0,
  gain_db     DECIMAL(5,2)    NOT NULL DEFAULT 0.00,
  is_looped   TINYINT(1)      NOT NULL DEFAULT 0,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (clip_id),
  KEY idx_clips_track_start (track_id, start_tick),
  CONSTRAINT fk_clips_track  FOREIGN KEY (track_id)  REFERENCES tracks (track_id) ON DELETE CASCADE,
  UNIQUE KEY uq_clips_launcher_slot (track_id, scene_id),
  KEY idx_clips_pattern (pattern_id),
  CONSTRAINT fk_clips_sample  FOREIGN KEY (sample_id)  REFERENCES samples (sample_id),
  CONSTRAINT fk_clips_pattern FOREIGN KEY (pattern_id) REFERENCES patterns (pattern_id),
  CONSTRAINT fk_clips_scene   FOREIGN KEY (scene_id)   REFERENCES launcher_scenes (scene_id) ON DELETE C
ASCADE,
  CONSTRAINT chk_clips_kind CHECK (
    (clip_type = 'MIDI'  AND pattern_id IS NOT NULL AND sample_id IS NULL) OR
    (clip_type = 'AUDIO' AND sample_id  IS NOT NULL AND pattern_id IS NULL)
  )
```

```
) ENGINE=InnoDB;

CREATE TABLE midi_notes (
  note_id     BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  pattern_id  BIGINT UNSIGNED  NOT NULL,
  pitch       TINYINT UNSIGNED NOT NULL,
  velocity    TINYINT UNSIGNED NOT NULL DEFAULT 100,
  start_tick  INT UNSIGNED     NOT NULL,
  length_tick INT UNSIGNED     NOT NULL,
  channel     TINYINT UNSIGNED NOT NULL DEFAULT 0,
  PRIMARY KEY (note_id),
  KEY idx_notes_pattern_start (pattern_id, start_tick),
  CONSTRAINT fk_notes_pattern FOREIGN KEY (pattern_id) REFERENCES patterns (pattern_id) ON DELETE CASCAD
E,
  CONSTRAINT chk_notes_pitch    CHECK (pitch <= 127),
  CONSTRAINT chk_notes_velocity CHECK (velocity BETWEEN 1 AND 127)
) ENGINE=InnoDB;

CREATE TABLE automation_lanes (
  lane_id         BIGINT UNSIGNED   NOT NULL AUTO_INCREMENT,
  track_id        BIGINT UNSIGNED   NOT NULL,
  target_type     ENUM('VOLUME','PAN','INSTRUMENT_PARAM','DEVICE_PARAM') NOT NULL,
  track_plugin_id BIGINT UNSIGNED   NULL,
  param_index     SMALLINT UNSIGNED NULL,
  is_enabled      TINYINT(1)        NOT NULL DEFAULT 1,
  created_at      DATETIME(3)       NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at      DATETIME(3)       NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP
(3),
  PRIMARY KEY (lane_id),
  KEY idx_lanes_track (track_id),
  CONSTRAINT fk_lanes_track  FOREIGN KEY (track_id)        REFERENCES tracks (track_id)               ON
DELETE CASCADE,
  CONSTRAINT fk_lanes_device FOREIGN KEY (track_plugin_id) REFERENCES track_plugins (track_plugin_id) ON
DELETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE automation_points (
  point_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  lane_id  BIGINT UNSIGNED NOT NULL,
  tick     INT UNSIGNED    NOT NULL,
  value    DECIMAL(12,4)   NOT NULL,
  curve    DECIMAL(3,2)    NOT NULL DEFAULT 0.00,
  PRIMARY KEY (point_id),
  KEY idx_points_lane_tick (lane_id, tick),
  CONSTRAINT fk_points_lane FOREIGN KEY (lane_id) REFERENCES automation_lanes (lane_id) ON DELETE CASCAD
E,
  CONSTRAINT chk_points_curve CHECK (curve BETWEEN -1.00 AND 1.00)
) ENGINE=InnoDB;

CREATE TABLE project_templates (
  template_id  BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  owner_id     BIGINT UNSIGNED NULL,
  name         VARCHAR(100)    NOT NULL,
  description  VARCHAR(500)    NULL,
  genre_tag_id INT UNSIGNED    NULL,
  data         JSON            NOT NULL,
  preview_path VARCHAR(500)    NULL,
```

```
  created_at   DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at   DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP(3),
  PRIMARY KEY (template_id),
  KEY idx_templates_owner (owner_id),
  CONSTRAINT fk_templates_owner FOREIGN KEY (owner_id)     REFERENCES users (user_id),
  CONSTRAINT fk_templates_genre FOREIGN KEY (genre_tag_id) REFERENCES tags (tag_id) ON DELETE SET NULL
) ENGINE=InnoDB;

ALTER TABLE projects
  ADD CONSTRAINT fk_projects_template FOREIGN KEY (template_id) REFERENCES project_templates (template_i
d) ON DELETE SET NULL;

CREATE TABLE user_settings (
  user_id            BIGINT UNSIGNED NOT NULL,
  theme              ENUM('DARK','LIGHT','SYSTEM') NOT NULL DEFAULT 'DARK',
  accent_color       CHAR(7)         NOT NULL DEFAULT '#7C5CFF',
  ui_scale           DECIMAL(3,2)    NOT NULL DEFAULT 1.00,
  keybindings        JSON            NULL,
  tutorial_state     JSON            NULL,
  character_enabled  TINYINT(1)      NOT NULL DEFAULT 1,
  character_code     VARCHAR(50)     NOT NULL DEFAULT 'nyxi',
  character_state    JSON            NULL,
  tips_level         ENUM('OFF','IMPORTANT','ALL') NOT NULL DEFAULT 'IMPORTANT',
  tips_state         JSON            NULL,
  bottom_panel_state JSON            NULL,
  ui_settings        JSON            NULL,
  updated_at         DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP
(3),
  PRIMARY KEY (user_id),
  CONSTRAINT fk_settings_user FOREIGN KEY (user_id) REFERENCES users (user_id) ON DELETE CASCADE
) ENGINE=InnoDB;

-- ========== Phase 3 ==========
CREATE TABLE export_jobs (
  export_id        BIGINT UNSIGNED   NOT NULL AUTO_INCREMENT,
  project_id       BIGINT UNSIGNED   NOT NULL,
  user_id          BIGINT UNSIGNED   NOT NULL,
  format           ENUM('WAV','MP3','FLAC') NOT NULL,
  sample_rate      INT UNSIGNED      NOT NULL DEFAULT 44100,
  bit_depth        TINYINT UNSIGNED  NULL,
  bitrate_kbps     SMALLINT UNSIGNED NULL,
  range_start_tick INT UNSIGNED      NOT NULL DEFAULT 0,
  range_end_tick   INT UNSIGNED      NULL,
  render_mode      ENUM('CLIENT','SERVER') NOT NULL DEFAULT 'CLIENT',
  status           ENUM('PENDING','PROCESSING','DONE','FAILED') NOT NULL DEFAULT 'PENDING',
  progress         TINYINT UNSIGNED  NOT NULL DEFAULT 0,
  result_path      VARCHAR(500)      NULL,
  file_size        BIGINT UNSIGNED   NULL,
  error_message    VARCHAR(1000)     NULL,
  created_at       DATETIME(3)       NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  completed_at     DATETIME(3)       NULL,
  PRIMARY KEY (export_id),
  KEY idx_export_project (project_id),
  KEY idx_export_status (status),
  CONSTRAINT fk_export_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASCA
DE,
  CONSTRAINT fk_export_user    FOREIGN KEY (user_id)    REFERENCES users (user_id)
```

```
) ENGINE=InnoDB;

-- ========== Phase 4 ==========
CREATE TABLE separation_jobs (
  separation_id    BIGINT UNSIGNED  NOT NULL AUTO_INCREMENT,
  user_id          BIGINT UNSIGNED  NOT NULL,
  source_sample_id BIGINT UNSIGNED  NOT NULL,
  model_name       VARCHAR(50)      NOT NULL,
  stem_mode        ENUM('2STEM','4STEM','6STEM') NOT NULL DEFAULT '4STEM',
  status           ENUM('PENDING','PROCESSING','DONE','FAILED') NOT NULL DEFAULT 'PENDING',
  progress         TINYINT UNSIGNED NOT NULL DEFAULT 0,
  error_message    VARCHAR(1000)    NULL,
  created_at       DATETIME(3)      NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  completed_at     DATETIME(3)      NULL,
  PRIMARY KEY (separation_id),
  KEY idx_sep_user (user_id),
  KEY idx_sep_status (status),
  CONSTRAINT fk_sep_user   FOREIGN KEY (user_id)          REFERENCES users (user_id),
  CONSTRAINT fk_sep_source FOREIGN KEY (source_sample_id) REFERENCES samples (sample_id)
) ENGINE=InnoDB;

CREATE TABLE separation_stems (
  stem_id       BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  separation_id BIGINT UNSIGNED NOT NULL,
  stem_type     ENUM('VOCALS','DRUMS','BASS','GUITAR','PIANO','OTHER','ACCOMPANIMENT') NOT NULL,
  sample_id     BIGINT UNSIGNED NOT NULL,
  created_at    DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (stem_id),
  UNIQUE KEY uq_stems_job_type (separation_id, stem_type),
  UNIQUE KEY uq_stems_sample (sample_id),
  CONSTRAINT fk_stems_job    FOREIGN KEY (separation_id) REFERENCES separation_jobs (separation_id) ON D
ELETE CASCADE,
  CONSTRAINT fk_stems_sample FOREIGN KEY (sample_id)     REFERENCES samples (sample_id)
) ENGINE=InnoDB;

-- ========== Phase 5 ==========
CREATE TABLE reference_tracks (
  reference_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id      BIGINT UNSIGNED NOT NULL,
  project_id   BIGINT UNSIGNED NULL,
  source_type  ENUM('LINK','FILE') NOT NULL,
  source_url   VARCHAR(1000)   NULL,
  file_path    VARCHAR(500)    NULL,
  title        VARCHAR(200)    NULL,
  genre_text   VARCHAR(100)    NULL,
  created_at   DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (reference_id),
  KEY idx_ref_user (user_id),
  KEY idx_ref_project (project_id),
  CONSTRAINT fk_ref_user    FOREIGN KEY (user_id)    REFERENCES users (user_id),
  CONSTRAINT fk_ref_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE SET NUL
L,
  CONSTRAINT chk_ref_source CHECK (
    (source_type = 'LINK' AND source_url IS NOT NULL) OR
    (source_type = 'FILE' AND file_path  IS NOT NULL)
  )
) ENGINE=InnoDB;
```

```
CREATE TABLE reference_tags (
  reference_id BIGINT UNSIGNED NOT NULL,
  tag_id       INT UNSIGNED    NOT NULL,
  PRIMARY KEY (reference_id, tag_id),
  KEY idx_ref_tags_tag (tag_id),
  CONSTRAINT fk_rt_ref FOREIGN KEY (reference_id) REFERENCES reference_tracks (reference_id) ON DELETE C
ASCADE,
  CONSTRAINT fk_rt_tag FOREIGN KEY (tag_id)       REFERENCES tags (tag_id)                  ON DELETE C
ASCADE
) ENGINE=InnoDB;

CREATE TABLE ai_analyses (
  analysis_id      BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  reference_id     BIGINT UNSIGNED NOT NULL,
  status           ENUM('PENDING','PROCESSING','DONE','FAILED') NOT NULL DEFAULT 'PENDING',
  model_version    VARCHAR(50)     NULL,
  bpm              DECIMAL(6,2)    NULL,
  music_key        VARCHAR(10)     NULL,
  loudness_lufs    DECIMAL(5,2)    NULL,
  true_peak_db     DECIMAL(5,2)    NULL,
  dynamic_range_db DECIMAL(5,2)    NULL,
  spectral_profile JSON            NULL,
  features         JSON            NULL,
  summary          TEXT            NULL,
  error_message    VARCHAR(1000)   NULL,
  created_at       DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  completed_at     DATETIME(3)     NULL,
  PRIMARY KEY (analysis_id),
  KEY idx_analyses_ref (reference_id),
  CONSTRAINT fk_analyses_ref FOREIGN KEY (reference_id) REFERENCES reference_tracks (reference_id) ON DE
LETE CASCADE
) ENGINE=InnoDB;

CREATE TABLE ai_recommendations (
  recommendation_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  analysis_id       BIGINT UNSIGNED NOT NULL,
  project_id        BIGINT UNSIGNED NULL,
  rec_type          ENUM('SYNTH_PRESET','PLUGIN_PRESET','MIXING','MASTERING') NOT NULL,
  target_plugin_id  INT UNSIGNED    NULL,
  settings          JSON            NOT NULL,
  rationale         TEXT            NULL,
  is_applied        TINYINT(1)      NOT NULL DEFAULT 0,
  applied_at        DATETIME(3)     NULL,
  created_at        DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (recommendation_id),
  KEY idx_rec_analysis (analysis_id),
  CONSTRAINT fk_rec_analysis FOREIGN KEY (analysis_id)      REFERENCES ai_analyses (analysis_id) ON DELE
TE CASCADE,
  CONSTRAINT fk_rec_project  FOREIGN KEY (project_id)       REFERENCES projects (project_id)     ON DELE
TE SET NULL,
  CONSTRAINT fk_rec_plugin   FOREIGN KEY (target_plugin_id) REFERENCES plugins (plugin_id)
) ENGINE=InnoDB;

CREATE TABLE assistant_messages (
  message_id    BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  user_id       BIGINT UNSIGNED NOT NULL,
```

```
  project_id    BIGINT UNSIGNED NULL,
  role          ENUM('USER','ASSISTANT') NOT NULL,
  content       TEXT            NOT NULL,
  context       JSON            NULL,
  model_version VARCHAR(50)     NULL,
  created_at    DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (message_id),
  KEY idx_assist_user_project (user_id, project_id, message_id),
  CONSTRAINT fk_assist_user    FOREIGN KEY (user_id)    REFERENCES users (user_id),
  CONSTRAINT fk_assist_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE SET N
ULL
) ENGINE=InnoDB;

-- ========== Phase 6 ==========
-- (project_members는 Phase 1 영역에서 이미 생성)
CREATE TABLE invitations (
  invitation_id BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  project_id    BIGINT UNSIGNED NOT NULL,
  inviter_id    BIGINT UNSIGNED NOT NULL,
  invitee_email VARCHAR(255)    NOT NULL,
  invitee_id    BIGINT UNSIGNED NULL,
  role          ENUM('EDITOR','VIEWER') NOT NULL DEFAULT 'EDITOR',
  token         CHAR(64)        NOT NULL,
  status        ENUM('PENDING','ACCEPTED','DECLINED','EXPIRED','CANCELED') NOT NULL DEFAULT 'PENDING',
  expires_at    DATETIME(3)     NOT NULL,
  created_at    DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  responded_at  DATETIME(3)     NULL,
  PRIMARY KEY (invitation_id),
  UNIQUE KEY uq_invitations_token (token),
  KEY idx_invitations_project (project_id),
  KEY idx_invitations_email (invitee_email),
  CONSTRAINT fk_inv_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASCADE,
  CONSTRAINT fk_inv_inviter FOREIGN KEY (inviter_id) REFERENCES users (user_id),
  CONSTRAINT fk_inv_invitee FOREIGN KEY (invitee_id) REFERENCES users (user_id)
) ENGINE=InnoDB;

CREATE TABLE project_change_logs (
  change_id   BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  project_id  BIGINT UNSIGNED NOT NULL,
  user_id     BIGINT UNSIGNED NOT NULL,
  revision    INT UNSIGNED    NOT NULL,
  op_type     VARCHAR(50)     NOT NULL,
  target_type VARCHAR(30)     NOT NULL,
  target_id   BIGINT UNSIGNED NULL,
  payload     JSON            NOT NULL,
  created_at  DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (change_id),
  UNIQUE KEY uq_changes_project_rev (project_id, revision),
  CONSTRAINT fk_changes_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASC
ADE,
  CONSTRAINT fk_changes_user    FOREIGN KEY (user_id)    REFERENCES users (user_id)
) ENGINE=InnoDB;

CREATE TABLE chat_messages (
  message_id   BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  project_id   BIGINT UNSIGNED NOT NULL,
  user_id      BIGINT UNSIGNED NULL,
```

```
  message_type ENUM('TEXT','SYSTEM') NOT NULL DEFAULT 'TEXT',
  content      TEXT            NOT NULL,
  created_at   DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  deleted_at   DATETIME(3)     NULL,
  PRIMARY KEY (message_id),
  KEY idx_chat_project_msg (project_id, message_id),
  CONSTRAINT fk_chat_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CASCAD
E,
  CONSTRAINT fk_chat_user    FOREIGN KEY (user_id)    REFERENCES users (user_id)
) ENGINE=InnoDB;

CREATE TABLE timeline_comments (
  comment_id        BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  project_id        BIGINT UNSIGNED NOT NULL,
  user_id           BIGINT UNSIGNED NOT NULL,
  parent_comment_id BIGINT UNSIGNED NULL,
  tick              INT UNSIGNED    NULL,
  track_id          BIGINT UNSIGNED NULL,
  clip_id           BIGINT UNSIGNED NULL,
  content           TEXT            NOT NULL,
  resolved_at       DATETIME(3)     NULL,
  resolved_by       BIGINT UNSIGNED NULL,
  created_at        DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  updated_at        DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3) ON UPDATE CURRENT_TIMESTAMP
(3),
  deleted_at        DATETIME(3)     NULL,
  PRIMARY KEY (comment_id),
  KEY idx_comments_project (project_id, resolved_at),
  CONSTRAINT fk_comments_project  FOREIGN KEY (project_id)        REFERENCES projects (project_id)
ON DELETE CASCADE,
  CONSTRAINT fk_comments_user     FOREIGN KEY (user_id)           REFERENCES users (user_id),
  CONSTRAINT fk_comments_parent   FOREIGN KEY (parent_comment_id) REFERENCES timeline_comments (comment_
id) ON DELETE CASCADE,
  CONSTRAINT fk_comments_track    FOREIGN KEY (track_id)          REFERENCES tracks (track_id)
ON DELETE SET NULL,
  CONSTRAINT fk_comments_clip     FOREIGN KEY (clip_id)           REFERENCES clips (clip_id)
ON DELETE SET NULL,
  CONSTRAINT fk_comments_resolver FOREIGN KEY (resolved_by)       REFERENCES users (user_id)
) ENGINE=InnoDB;

CREATE TABLE project_snapshots (
  snapshot_id   BIGINT UNSIGNED NOT NULL AUTO_INCREMENT,
  project_id    BIGINT UNSIGNED NOT NULL,
  created_by    BIGINT UNSIGNED NULL,
  snapshot_type ENUM('MANUAL','AUTO_DAILY','BEFORE_RESTORE','BEFORE_TEMPLATE','BEFORE_AI_APPLY') NOT NUL
L,
  name          VARCHAR(100)    NULL,
  revision      INT UNSIGNED    NOT NULL,
  data_path     VARCHAR(500)    NOT NULL,
  preview_path  VARCHAR(500)    NULL,
  size_bytes    BIGINT UNSIGNED NOT NULL DEFAULT 0,
  is_pinned     TINYINT(1)      NOT NULL DEFAULT 0,
  created_at    DATETIME(3)     NOT NULL DEFAULT CURRENT_TIMESTAMP(3),
  PRIMARY KEY (snapshot_id),
  KEY idx_snapshots_project (project_id, created_at),
  CONSTRAINT fk_snapshots_project FOREIGN KEY (project_id) REFERENCES projects (project_id) ON DELETE CA
SCADE,
```

```
  CONSTRAINT fk_snapshots_user    FOREIGN KEY (created_by) REFERENCES users (user_id)
) ENGINE=InnoDB;
```

---

6. 변경 이력

날짜	내용
2026-10-01	최초 작성 (Phase 1~6 전체 테이블 초안)
2026-10-01	트랙 아코디언(체인+패처): `tracks.routing_mode`, `track_plugins.node_x/node_y`, `track_plugin_links` 추가 · 캐릭터/하단 패널: `user_settings` 추가 · Gopher: `assistant_messages` 추가 · ERD를 직선 이미지(`ERD.png`)로 교체 (총 26개 테이블)
2026-10-01	AI 제안 채택 반영: `patterns`·`launcher_scenes`(패턴·클립 런처, **`midi_notes.clip_id` → `pattern_id`**), `automation_lanes`·`automation_points`, `project_templates`, `timeline_comments`, `project_snapshots` 추가 / `tracks`: MASTER 타입·담당자·잠금 / `projects`: `scale_mode`·`template_id` / `users`: 저장 용량 / `user_settings`: 테마·배율·단축키·튜토리얼 / Gopher → Nyxi(가칭) (총 33개 테이블)
2026-10-01	이름 **Nyxi** 확정 · `user_settings.tips_level/tips_state`(Nyxi 팁) · `plugins.card_asset` 및 `param_schema`의 `primary`/`group`(카드 UI·상세 모달) 추가

Studio Nx — ERD (관계 설명)

> 그림: ERD.png · 컬럼 상세: DB.md §3 · DBMS: MySQL 8.0 (`studionx`) · 테이블 33개

(ERD 그림은 앞의 가로 페이지 참고)

---

1. 읽는 법

표기	의미
`||` (선 끝 두 줄)	반드시 1개
`|o` (선 끝 줄 + 원)	0 또는 1개 (FK가 NULL 허용)
까마귀발 + 원	0개 이상 (N)
`PK` / `FK` / `UQ`	기본키 / 외래키 / 유니크
`(null)`	NULL 허용 FK
`→ users`	요청자·작성자를 가리키는 FK. 선이 너무 많아져 **그림에서는 선을 생략**하고 박스 안에만 표기

2. 한 줄 요약

회원(users)이 프로젝트(projects)를 갖고, 프로젝트는 트랙 → 클립으로 내려가며, MIDI 클립은 패턴(노트 묶음)을 가리킨다. 트랙에는 디바이스가 체인/패처로 연결되고 오토메이션 레인이 붙는다. 샘플·플러그인·태그·템플릿은 공용 자원이고, 출력·분리·AI·협업(코멘트·스냅샷)은 프로젝트나 샘플에 매달린 부가 기록이다.

3. 영역별 구성

영역	테이블	핵심 역할
계정	`users`, `user_settings`, `inquiries`	회원(+저장 용량), UI 설정(테마·단축키·캐릭터·튜토리얼), 문의
프로젝트 · 협업	`projects`, `project_members`, `invitations`, `project_change_logs`, `chat_messages`, `timeline_comments`, `project_snapshots`	프로젝트와 참여자·초대·편집이력·톡방·코멘트·스냅샷
시퀀서	`tracks`, `track_plugins`, `track_plugin_links`, `patterns`, `launcher_scenes`, `clips`, `midi_notes`, `automation_lanes`, `automation_points`	곡의 실제 내용 (트랙·디바이스·라우팅·패턴·런처·클립·노트·오토메이션)
라이브러리 · 플러그인 · 템플릿	`samples`, `tags`, `sample_tags`, `plugins`, `plugin_presets`, `project_templates`	샘플 라이브러리, 태그, 웹 플러그인 목록·프리셋, 프로젝트 템플릿
출력 · 분리	`export_jobs`, `separation_jobs`, `separation_stems`	오디오 출력 작업, 악기 분리 작업과 결과
AI	`reference_tracks`, `reference_tags`, `ai_analyses`, `ai_recommendations`, `assistant_messages`	레퍼런스 분석·추천, Nyxi 도우미 대화

4. 관계 목록

> 형식: **부모 → 자식** (관계) · 연결 컬럼 · 설명

4.1 계정

부모 → 자식	관계	연결 컬럼	설명
users → user_settings	1 : 0..1	`user_settings.user_id` (PK=FK)	회원당 UI 설정 1행
users → inquiries	0..1 : N	`inquiries.user_id` (null)	비회원 문의는 NULL
users → projects	1 : N	`projects.owner_id`	프로젝트 소유자
users → samples	1 : N	`samples.owner_id`	개인 샘플 라이브러리
users → plugin_presets	0..1 : N	`plugin_presets.owner_id` (null)	NULL = 시스템 프리셋

4.2 프로젝트 · 협업

부모 → 자식	관계	연결 컬럼	설명
projects ↔ users	N : M	`project_members(project_id, user_id)`	참여자와 권한(OWNER/EDITOR/VIEWER). 소유자도 OWNER로 들어감
projects → invitations	1 : N	`invitations.project_id`	초대 (inviter·invitee → users)
projects → project_change_logs	1 : N	`project_change_logs.project_id`	실시간 편집 이력, `(project_id, revision)` 유일
projects → chat_messages	1 : N	`chat_messages.project_id`	프로젝트 = 톡방
projects → timeline_comments	1 : N	`timeline_comments.project_id`	위치(tick) 코멘트. `parent_comment_id`로 답글(자기 참조)
tracks / clips → timeline_comments	0..1 : N	`track_id`, `clip_id` (null)	트랙·클립에 붙인 코멘트 (지워지면 NULL)
projects → project_snapshots	1 : N	`project_snapshots.project_id`	버전 스냅샷 (데이터는 압축 JSON 파일)
users → tracks (담당·잠금)	0..1 : N	`tracks.assignee_id`, `locked_by` (null)	그림에서는 선 생략 (`→ users` 표기)

4.3 시퀀서

부모 → 자식	관계	연결 컬럼	설명
projects → tracks	1 : N	`tracks.project_id`	트랙 목록
tracks → clips	1 : N	`clips.track_id`	트랙 위 클립
projects → patterns	1 : N	`patterns.project_id`	MIDI 패턴 (노트 묶음)
patterns → midi_notes	1 : N	`midi_notes.pattern_id`	노트는 **패턴**에 속함
patterns → clips	0..1 : N	`clips.pattern_id` (null)	MIDI 클립은 패턴 필수. 같은 패턴을 쓰는 클립 = 링크 클립
projects → launcher_scenes	1 : N	`launcher_scenes.project_id`	클립 런처 씬(행)
launcher_scenes → clips	0..1 : N	`clips.scene_id` (null)	NULL = 타임라인 클립, 값 = 런처 슬롯 `(track_id, scene_id)` 유일

부모 → 자식	관계	연결 컬럼	설명
tracks → automation_lanes	1 : N	`automation_lanes.track_id`	트랙별 오토메이션 레인 (MASTER 포함)
track_plugins → automation_lanes	0..1 : N	`automation_lanes.track_plugin_id` (null)	디바이스 파라미터 오토메이션
automation_lanes → automation_points	1 : N	`automation_points.lane_id`	포인트 (tick, 값, 곡선)
samples → clips	0..1 : N	`clips.sample_id` (null)	오디오 클립만 샘플 참조
tracks → track_plugins	1 : N	`track_plugins.track_id`	트랙의 이펙트 디바이스 (`slot_index` = 체인 순서)
plugins → track_plugins	1 : N	`track_plugins.plugin_id`	어떤 플러그인의 인스턴스인지
plugins → tracks	0..1 : N	`tracks.instrument_plugin_id` (null)	MIDI 트랙의 가상악기
tracks → track_plugin_links	1 : N	`track_plugin_links.track_id`	패처 연결선
track_plugins → track_plugin_links	0..1 : N	`from_track_plugin_id`, `to_track_plugin_id` (null)	NULL = 트랙 소스 / 트랙 출력

4.4 라이브러리 · 플러그인

부모 → 자식	관계	연결 컬럼	설명
samples ↔ tags	N : M	`sample_tags(sample_id, tag_id)`	샘플 태그
plugins → plugin_presets	1 : N	`plugin_presets.plugin_id`	플러그인별 프리셋
users → project_templates	0..1 : N	`project_templates.owner_id` (null)	NULL = 시스템 템플릿
tags → project_templates	0..1 : N	`project_templates.genre_tag_id` (null)	템플릿 장르
project_templates → projects	0..1 : N	`projects.template_id` (null)	어떤 템플릿에서 시작했는지

4.5 출력 · 분리

부모 → 자식	관계	연결 컬럼	설명
projects → export_jobs	1 : N	`export_jobs.project_id`	출력 작업 (요청자 → users)
samples → separation_jobs	1 : N	`separation_jobs.source_sample_id`	분리할 원본 (요청자 → users)
separation_jobs → separation_stems	1 : N	`separation_stems.separation_id`	작업당 stem 여러 개, `(separation_id, stem_type)` 유일
samples → separation_stems	1 : 0..1	`separation_stems.sample_id` (UQ)	결과 stem도 하나의 샘플로 저장

4.6 AI

부모 → 자식	관계	연결 컬럼	설명
users → reference_tracks	1 : N	`reference_tracks.user_id`	레퍼런스 등록자
projects → reference_tracks	0..1 : N	`reference_tracks.project_id` (null)	프로젝트 연결은 선택
reference_tracks ↔ tags	N : M	`reference_tags(reference_id, tag_id)`	장르 태그
reference_tracks → ai_analyses	1 : N	`ai_analyses.reference_id`	분석 결과 (재분석 가능)

부모 → 자식	관계	연결 컬럼	설명
ai_analyses → ai_recommendations	1 : N	`ai_recommendations.analysis_id`	프리셋/믹싱/마스터링 추천
projects → ai_recommendations	0..1 : N	`ai_recommendations.project_id` (null)	적용 대상 프로젝트
plugins → ai_recommendations	0..1 : N	`ai_recommendations.target_plugin_id` (null)	프리셋 추천 대상 플러그인
users → assistant_messages	1 : N	`assistant_messages.user_id`	Nyxi 도우미 대화 (개인 전용)
projects → assistant_messages	0..1 : N	`assistant_messages.project_id` (null)	대화가 일어난 프로젝트

5. 꼭 알아둘 규칙

1. **프로젝트를 지우면** 트랙·클립·패턴·노트·디바이스·연결선·오토메이션·씬·멤버·초대·이력·채팅·코멘트·스냅샷·출력이 함께 지워집니다(CASCADE). 레퍼런스·추천·Nyxi 대화는 `project_id`만 NULL이 됩니다.
2. **회원·샘플·프로젝트는 소프트 삭제**(`deleted_at`)입니다. 실제 DELETE가 거의 일어나지 않아 users FK는 RESTRICT입니다.
3. **N:M 연결 테이블은 3개**: `project_members`, `sample_tags`, `reference_tags` (모두 복합 PK).
4. **분리 결과와 출력 결과는 `samples`에 다시 들어갑니다** (`source = 'SEPARATION' / 'EXPORT'`). 그래서 타임라인에서 일반 샘플처럼 쓸 수 있습니다.
5. **라우팅**: `tracks.routing_mode = 'CHAIN'`이면 `track_plugins.slot_index` 순서로 직렬, `'PATCHER'`이면 `track_plugin_links` 연결선대로 처리합니다.
6. 시퀀서의 위치·길이는 모두 **tick (PPQ = 960)** 단위입니다. `midi_notes.start_tick`은 **패턴 시작 기준** 상대값입니다.
7. **마스터도 트랙입니다** (`tracks.track_type = 'MASTER'`, 프로젝트당 1개). 그래서 마스터 디바이스·오토메이션도 일반 트랙과 같은 테이블을 씁니다.
8. **클립 종류 규칙**: MIDI 클립 = `pattern_id` 필수·`sample_id` NULL / AUDIO 클립 = `sample_id` 필수·`pattern_id` NULL (CHECK 제약).